

AGENTE X

Plataforma SaaS Multi-Tenant de
Inteligencia de Negocios Conversacional

Integrante del grupo

- Mauricio Lillo

Índice

Índice	2
Introducción	3
Contexto del proyecto	4
Descripción del problema u oportunidad	6
Descripción del mercado objetivo	8
Objetivo general y objetivos específicos	10
Situación inicial del cliente	12
Planificación	14
Servicios Cloud a utilizar	16
Conceptualización	18
Alcance, supuestos y restricciones	21
Metodología de desarrollo y gestión del proyecto	23
Documentos y diagramas de diseño de la solución	26
Arquitectura de software y patrones de diseño	29
Tecnologías, lenguaje y dependencias	32
Configuración de servidor de producción	35
Estado de avance del desarrollo	39
Integraciones necesarias	43
Estrategia y enfoque de calidad	46
Plan de pruebas	49
Tipos de pruebas aplicadas	52
Herramientas y software utilizados	56
Ejecución de pruebas y resultados	60
Conclusión	68
Anexos	71

Introducción

La gestión de la información en el entorno empresarial contemporáneo presenta una paradoja crítica: las organizaciones generan más datos que nunca, pero sus líderes tardan demasiado tiempo en extraer valor estratégico de ellos. En la actualidad, cuando un dueño de negocio o ejecutivo necesita tomar una decisión operativa, se enfrenta a silos de información fragmentados entre reportes financieros estáticos, sistemas de gestión complejos y múltiples plataformas desconectadas entre sí. La extracción de respuestas concretas requiere navegar por paneles de analítica poco intuitivos o depender de intermediarios técnicos, lo que genera un costo de oportunidad altísimo y una latencia inaceptable en la toma de decisiones. Esta fricción administrativa y la alta carga cognitiva que implica cruzar datos manualmente es la situación fundamental que motivó el presente proyecto.

Para resolver este cuello de botella, se construyó AgenteX: una plataforma web SaaS multi-tenant de Inteligencia de Negocios Conversacional basada en una arquitectura de Agentes de Inteligencia Artificial especializados. En lugar de obligar al usuario a interpretar gráficos crudos o navegar por sistemas ERP complejos, la plataforma actúa como un intermediario cognitivo que se conecta directamente a los endpoints de datos de cada empresa. A través de agentes especializados por dominio operativo —bodega, ventas, analítica y logística— el sistema permite a los equipos interactuar en lenguaje natural con sus propios datos en tiempo real, recibiendo respuestas sintetizadas y fundamentadas exclusivamente en información real de la empresa, eliminando el riesgo de alucinaciones genéricas de la IA.

En las siguientes páginas, el lector encontrará un desglose estructurado de esta solución. Se detallará el contexto del proyecto y una radiografía del problema identificando sus causas y el impacto directo en el mercado objetivo. Posteriormente, se definirán los objetivos generales y específicos que guiaron el desarrollo, seguidos de un análisis de la situación inicial del cliente. El documento expone además la planificación metodológica, la justificación de los servicios cloud seleccionados, la conceptualización de la arquitectura implementada, y delimita claramente el alcance, supuestos y restricciones del sistema. Finalmente, se documenta el estado de avance del desarrollo con evidencias del producto funcional, las integraciones implementadas y las conclusiones del proyecto.

Contexto del proyecto

Antecedentes, motivación y origen del problema

En el ecosistema empresarial actual, la velocidad en la toma de decisiones define la supervivencia de una compañía. Sin embargo, existe una desconexión crítica entre la cantidad de datos que genera una empresa y la capacidad de su directiva para consumirlos de forma ágil. La motivación de este proyecto surge de la observación directa y la experiencia empírica en la gestión operativa de negocios de retail, e-commerce y servicios. Se detectó recurrentemente que los dueños de negocios y gerentes invierten una cantidad desproporcionada de tiempo intentando extraer métricas específicas —ventas trimestrales, rotación de inventario, estado de órdenes— cruzando manualmente hojas de cálculo, leyendo reportes en PDF o navegando por sistemas ERP complejos que no fueron diseñados para responder preguntas en lenguaje natural.

Hoy en día, la información estratégica está secuestrada detrás de interfaces técnicas. Cuando un directivo necesita una respuesta rápida antes de una reunión, el flujo actual lo obliga a solicitar un reporte a un analista o a buscarlo él mismo, generando latencia operativa. Esta fricción convierte la toma de decisiones en un proceso reactivo en lugar de proactivo, con consecuencias directas en la competitividad del negocio.

Público objetivo general

El usuario principal de esta solución está compuesto por dueños de negocios de PyMEs y medianas empresas, ejecutivos C-Level y gerentes operativos de áreas como ventas, bodega, logística y analítica, que requieren inmediatez en el acceso a la información. Son perfiles con un alto costo de oportunidad que no desean invertir tiempo en interpretar gráficos densos, sino que buscan respuestas sintetizadas y exactas fundamentadas en sus propios datos corporativos. La plataforma adopta un modelo B2B multi-tenant, donde cada empresa cliente opera en un entorno aislado con sus propios agentes, configuraciones y conexiones a endpoints.

Estado del arte y formas actuales de resolución

Actualmente, el mercado intenta resolver esta necesidad mediante dos extremos que no logran conectarse. Por un lado, se utilizan herramientas de Business Intelligence tradicionales como Power BI o Tableau, las cuales son potentes visualmente pero requieren conocimientos técnicos para su configuración y no responden a consultas conversacionales de forma nativa. Por otro lado, la irrupción de modelos de lenguaje genéricos —ChatGPT, Claude, Gemini— ha democratizado la inteligencia artificial, pero estas herramientas carecen de conocimiento sobre la realidad interna y privada de la empresa, lo que las lleva a generar respuestas imprecisas o alucinadas cuando no se les inyecta el contexto adecuado.

Esta desconexión evidencia que las soluciones actuales no bastan. Se requiere una plataforma que fusione la agilidad conversacional de los LLMs con la rigurosidad de los datos corporativos reales, conectándose directamente a los sistemas de cada empresa sin exponer información sensible ni depender de ecosistemas cerrados de alto costo.

Tabla de homologación:

Solución / Competidor	¿Qué hace?	Ventaja	Desventaja	¿Qué haré distinto?
 Herramientas de BI Tradicional (PowerBI, Tableau)	Extrae datos de bases de datos y los convierte en paneles de control y gráficos interactivos.	Altísima precisión en los datos. Visualización profunda y capacidades matemáticas avanzadas.	Curva de aprendizaje empinada. Interfaces rígidas. No responde al lenguaje natural de forma conversacional rápida.	Reemplaza los gráficos complejos por respuestas directas en lenguaje natural mediante agentes especializados por dominio operativo
 LLMs Genéricos y Públicos (ChatGPT, Claude, Gemini)	Asistentes de IA que responden preguntas en lenguaje natural	Flexibilidad total, lenguaje natural avanzado, fácil uso	Riesgo crítico de alucinaciones. Desconocen los datos privados de la empresa. No se conectan a endpoints en tiempo real	Los agentes están restringidos a responder exclusivamente basándose en datos reales extraídos de los endpoints de la empresa, con protocolo anti-alucinación implementado en el sistema prompt
 Chatbots de ERPs Corporativos (SAP CoPilot, Microsoft Dynamics)	Asistentes virtuales integrados nativamente en software de gestión empresarial de gran escala	Conexión directa con la base de datos de la corporación. Alta seguridad	Costos de licencia prohibitivos para PyMEs. Ecosistema cerrado, solo funcionan con su propio software	Arquitectura agnóstica: cualquier empresa puede conectar sus endpoints REST existentes sin cambiar su ERP, con un costo de implementación drásticamente inferior

Descripción del problema u oportunidad

Definición del problema central

Actualmente la extracción manual y visual de métricas desde silos de información corporativa y sistemas de gestión tradicionales exige una alta carga cognitiva a los directivos, lo que provoca una latencia inaceptable en la toma de decisiones estratégicas y un profundo desperdicio del capital de datos de la empresa.

Descripción del impacto y alcance

El impacto de esta fricción operativa se mide directamente en costos de oportunidad y tiempo. En un entorno corporativo o PyME, el dueño del negocio o ejecutivo pierde horas semanales cruzando datos entre facturación, inventario y reportes para obtener respuestas simples como "¿Cuál fue la línea de productos más rentable el trimestre pasado considerando el costo de almacenamiento?". Esta ineficiencia administrativa genera pérdidas financieras ocultas: se pagan horas de recursos humanos altamente calificados —o el tiempo del propio CEO— para realizar tareas de recopilación de datos, en lugar de utilizarlos en la ejecución comercial.

Adicionalmente, genera una mala experiencia para el directivo, quien sufre de fatiga por exceso de información, viéndose obligado a interpretar paneles visuales genéricos en lugar de recibir conclusiones ejecutivas precisas. El resultado es una organización que posee los datos pero no puede consumirlos con la velocidad que el mercado exige.

Análisis de Causas y Efectos

Para entender la anatomía de este fallo operativo, se estructuró un análisis basado en el modelo de Árbol de Problemas, desglosando las raíces técnicas que originan el problema y las consecuencias que impactan al negocio. El diagrama correspondiente se presenta en los Anexos [\(Figura 1\)](#).

Las Causas:

- 1. Silos de datos desconectados y estáticos:** La información de la empresa vive en formatos incompatibles. Los reportes cualitativos están en PDFs aislados, mientras que los datos cuantitativos residen en bases de datos y ERPs que requieren consultas complejas o navegación manual especializada. No existe un punto de acceso unificado que integre ambas fuentes.
- 2. Fricción en la Interfaz de Usuario:** Los sistemas actuales dependen de dashboards que exigen al usuario aprender a filtrar, cruzar y exportar información. No están diseñados para consultas naturales, directas y conversacionales, lo que eleva la barrera de acceso para perfiles no técnicos.
- 3. Limitaciones de la IA Pública:** Los modelos de lenguaje grandes genéricos carecen de acceso a la intranet, los documentos privados o las bases de datos de la empresa. Si un

usuario intenta usarlos sin un sistema de inyección de contexto estructurado, la IA alucina inventando datos, volviéndola inútil para la toma de decisiones críticas.

Los Efectos:

1. Toma de decisiones reactiva y demorada: Al carecer de respuestas en tiempo real, las decisiones se toman basadas en intuición o con días de retraso, perdiendo agilidad competitiva ante mercados que cambian en horas.

2. Subutilización del capital de datos: Gran parte del conocimiento operativo de la empresa —históricos de ventas, órdenes de trabajo, métricas de logística— se convierte en datos muertos porque el costo en tiempo para extraerlos y analizarlos es superior al beneficio percibido.

3. Sobrecarga administrativa y dependencia técnica: El dueño del negocio depende de analistas o personal de TI para generar reportes ad-hoc, creando un cuello de botella en los flujos de trabajo de la directiva y aumentando los costos operativos indirectos.

Conclusión del análisis

La solución desarrollada ataca directamente las tres causas identificadas: elimina los silos mediante la conexión directa a endpoints REST de la empresa, reemplaza la interfaz rígida con un chat conversacional en lenguaje natural, y mitiga las alucinaciones mediante un protocolo anti-alucinación jerárquico implementado en el sistema prompt de cada agente, que restringe las respuestas exclusivamente a los datos recibidos en tiempo real.

Descripción del mercado objetivo

Definición del usuario y cliente

El mercado objetivo de AgenteX opera bajo un modelo B2B multi-tenant. El cliente principal —quien toma la decisión de compra y autoriza la integración del sistema— es el dueño del negocio, el CEO o el Director de Operaciones de una PyME o empresa mediana. Sin embargo, el usuario final —quien utilizará la plataforma en el día a día— se divide en capas según su rol operativo: la alta dirección para consultas estratégicas y analíticas, y los equipos de bodega, ventas y logística para consultas tácticas y operativas en tiempo real.

La plataforma está diseñada para que una misma empresa pueda desplegar múltiples agentes especializados simultáneamente, cada uno conectado a los endpoints relevantes para su área. El administrador de cada empresa gestiona la configuración desde un panel propio, sin depender de soporte técnico externo para las operaciones cotidianas.

Necesidades principales

El usuario necesita lograr una sola cosa en el menor tiempo posible: certeza operativa. Necesita resolver la fricción entre la pregunta de negocio que tiene en la cabeza y el dato duro que vive en los sistemas de la empresa. Específicamente, necesita centralizar la consulta de información entre múltiples fuentes, eliminar la dependencia de analistas de datos o soporte de TI para obtener métricas diarias, e interactuar con sus propios datos de forma conversacional sin requerir conocimientos técnicos.

Descripción de dolores

Los dolores principales del usuario objetivo son:

- 1. La interfaz técnica:** El usuario necesita abrir un software ERP complejo, recordar cómo aplicar filtros específicos y exportar archivos para obtener datos básicos como el stock actual o las ventas de la semana. Esta fricción consume tiempo y energía cognitiva que debería destinarse a decisiones de negocio.
- 2. La fragmentación:** El contexto operativo vive en plataformas separadas — el inventario en un sistema, las órdenes en otro, los clientes en un tercero. Cruzar esa información manualmente para obtener una visión unificada es el cuello de botella más costoso del proceso.
- 3. La dependencia de terceros:** El directivo debe esperar horas o días a que su equipo procese la información y entregue un resumen ejecutivo antes de una toma de decisión urgente, creando una dependencia estructural que ralentiza toda la organización.

Escenario de uso

La plataforma será utilizada en escenarios de alta presión operativa. Un escenario típico es el de un jefe de bodega que necesita saber, antes de confirmar un despacho, si tiene stock

suficiente de un producto específico y cuál es el nivel de reposición crítico. Abre AgenteX, selecciona el Agente de Bodega y escribe: "¿Cuántas unidades de neumático 29x2.10 tenemos disponibles y cuál es el stock mínimo?". El sistema consulta el ERP en tiempo real y responde en segundos con datos exactos, sin que el usuario haya tenido que navegar por ningún panel.

Perfiles de Usuario

Usuario 1 (Principal): Dueño / Gerente de PyME con operaciones de inventario y ventas

- **Características:** Ejecutivo de 30 a 55 años que dirige una empresa con inventario físico, equipo de ventas y operaciones logísticas. Su día a día es acelerado y no tiene tiempo para aprender herramientas de analítica avanzada.
- **Necesidad:** Consultar métricas clave —stock crítico, ventas del período, estado de órdenes, rendimiento por técnico— a través de una pregunta en lenguaje natural, desde cualquier dispositivo, en cualquier momento del día.

Usuario 2 (Secundario): Vendedor o Coordinador Comercial

- **Características:** Profesional operativo que atiende clientes directamente y necesita información del inventario y del perfil del cliente en tiempo real durante una venta o negociación.
- **Necesidad:** Consultar disponibilidad de productos, historial de compras de un cliente específico y alternativas de stock, con recomendaciones personalizadas basadas en el perfil del cliente, sin interrumpir el flujo de la conversación comercial.

Objetivo general y objetivos específicos

Objetivo General

Desarrollar una plataforma web SaaS multi-tenant de Inteligencia de Negocios Conversacional, basada en una arquitectura de Agentes de Inteligencia Artificial especializados por dominio operativo, que permita a empresas PyME conectar sus sistemas de gestión existentes mediante endpoints REST y consultar sus datos en tiempo real a través de lenguaje natural, reduciendo la latencia en la toma de decisiones operativas y eliminando la dependencia de intermediarios técnicos para la extracción de información estratégica.

Objetivos Específicos

1. Diseñar e implementar una arquitectura multi-tenant en base de datos relacional que gestione el aislamiento de datos entre empresas, la jerarquía de roles de acceso (SUPER_ADMIN, ADMIN, USER) y la vinculación entre agentes, plantillas y sesiones de conversación, garantizando integridad referencial y trazabilidad completa de cada interacción.

- *Métrica de validación:* Despliegue de esquema relacional en Supabase con llaves foráneas activas, aislamiento verificado entre tenants y cero orfandad de datos en pruebas de inserción y eliminación en cascada.

2. Desarrollar un sistema de orquestación de motores especializados que enrute cada consulta al motor correspondiente según el tipo de agente configurado — búsqueda léxica de inventario, análisis temporal de datos, asesoría comercial con historial de cliente, y consulta de órdenes logísticas — conectándose en tiempo real a los endpoints REST de cada empresa.

- *Métrica de validación:* Ejecución exitosa de consultas a los cuatro motores (erp_search, analytics, ventas, logistica) con datos reales provenientes de los endpoints configurados, sin hardcoding de rutas ni lógica específica por empresa.

3. Implementar un protocolo anti-alucinación jerárquico en el sistema prompt de cada agente que restrinja las respuestas del modelo de lenguaje exclusivamente a los datos recibidos en tiempo real desde los endpoints del ERP, eliminando la posibilidad de que el modelo genere información no fundamentada en los datos reales de la empresa.

- *Métrica de validación:* Verificación empírica de que el agente devuelve "CERO RESULTADOS" o un mensaje de error controlado ante consultas fuera del alcance de los datos disponibles, en vez de generar respuestas inventadas.

4. Construir un panel de administración B2B con jerarquía de roles diferenciada que permita al SUPER_ADMIN aprovisionar nuevas empresas, configurar los agentes con sus instrucciones base, definir las conexiones a endpoints ERP y gestionar el estado de suscripción de cada tenant. El ADMIN de cada empresa cliente tendrá acceso limitado a personalizar las instrucciones conversacionales de sus agentes dentro de los parámetros

establecidos por el proveedor, y a gestionar los usuarios de su propia empresa según el tipo de suscripción contratada. La configuración técnica del ERP —URLs de endpoints, mapeo de campos y tokens de autenticación— es responsabilidad exclusiva del SUPER_ADMIN como parte del servicio de onboarding y soporte.

- *Métrica de validación:* Verificación de que un usuario con rol ADMIN no puede acceder ni modificar la configuración de endpoints ERP ni el `erp_mapping` de su empresa, y que el SUPER_ADMIN puede aprovisionar una empresa completa con agentes operativos desde el panel web sin intervención directa en la base de datos.

5. Desplegar la plataforma en infraestructura cloud con separación de servicios

utilizando Vercel para el frontend, Render para el backend Node.js y el microservicio Python, y Supabase como base de datos administrada, garantizando disponibilidad continua, escalabilidad independiente por servicio y comunicación segura entre componentes mediante secretos de entorno.

- *Métrica de validación:* Plataforma accesible públicamente con URL de producción, comunicación verificada entre los tres servicios desplegados, y variables de entorno configuradas sin exposición de secretos en el repositorio.

Situación inicial del cliente

Análisis del flujo operativo actual

El proceso actual de extracción de inteligencia de negocios y consulta de información en una PyME, e-commerce o departamento corporativo es manual, fragmentado y altamente dependiente de la intervención humana. La arquitectura de información del cliente opera bajo un modelo de "silos desconectados", donde los datos cuantitativos —inventario, ventas, órdenes de trabajo— residen en sistemas de gestión que no se comunican entre sí ni responden a consultas en lenguaje natural.

A continuación se detalla el proceso paso a paso:

- 1. El usuario formula una solicitud estratégica:** El flujo inicia cuando el director o dueño de la empresa se enfrenta a una decisión crítica y formula una pregunta de negocio compleja. Por ejemplo: "¿Por qué cayeron las ventas de la línea de movilidad eléctrica y tenemos suficiente stock en bodega para reactivarlo con una campaña de marketing hoy?".
- 2. Se inicia la recolección manual de datos:** Para obtener la respuesta, el usuario debe acceder a múltiples plataformas de forma independiente. Abre el sistema de gestión ERP para revisar el inventario actual, entra a la pasarela de pagos o al CRM para descargar las métricas de ventas, y paralelamente debe buscar en sus carpetas locales documentos con especificaciones técnicas de los productos para tener el contexto completo.
- 3. Se procesan los datos en herramientas intermedias:** La información extraída de los distintos sistemas se exporta en archivos CSV y se registra en una hoja de cálculo unificada o se intenta volcar en un panel de control. El usuario o analista debe aplicar fórmulas, limpiar datos incompatibles y redactar un reporte que cruce los números con el contexto operativo disponible.
- 4. El problema estalla cuando el tiempo anula la oportunidad:** Los puntos de dolor críticos se generan en la fase de cruce manual. El problema se hace evidente cuando el directivo necesita esta respuesta en cinco minutos antes de una reunión con inversores, pero el proceso de recolección y síntesis de datos tarda horas o incluso días. En ese punto, la decisión se toma por intuición o queda postergada, generando pérdidas directas de competitividad.

Identificación de Puntos de Dolor

- **Cuello de botella humano (Latencia):** El proceso depende de la velocidad a la que un humano pueda leer, descargar y cruzar datos entre sistemas desconectados. No existe automatización ni integración entre fuentes.
- **Ceguera de contexto:** Los sistemas de inventario o ventas muestran "qué" pasó —cayeron las ventas— pero no "por qué" pasó. La respuesta cualitativa suele estar escondida en documentos o sistemas que el ERP no lee ni integra.
- **Fatiga de decisión:** El directivo invierte su energía cognitiva intentando entender de dónde salió el dato y si puede confiar en él, en lugar de invertirla en la decisión estratégica que lo requiere.

- **Dependencia técnica:** La generación de reportes ad-hoc requiere la intervención de analistas o personal de TI, creando un cuello de botella estructural en los flujos de trabajo de la dirección y aumentando los costos operativos indirectos.

El diagrama del flujo operativo actual se presenta en los Anexos ([Figura 2](#)).

Planificación

El desarrollo de AgenteX se estructuró en un ciclo de 12 semanas dividido en 4 Sprints de 3 semanas cada uno, adoptando una metodología ágil iterativa que permitió ajustar el alcance técnico a medida que la arquitectura evolucionó desde la propuesta inicial. El proyecto fue ejecutado íntegramente por un único desarrollador, lo que implicó una gestión de prioridades estricta para mantener la estabilidad del núcleo funcional en cada entrega.

Distribución de Tareas por Sprint

Sprint 1 — Cimientos y Arquitectura (Semanas 1-3) ([Figura 3.1](#))

- Diseño e implementación del esquema relacional en Supabase con integridad referencial entre empresas, usuarios, agentes y sesiones de conversación.
- Configuración del entorno de desarrollo con Node.js, Express y conexión a Supabase.
- Implementación del sistema de autenticación JWT con roles diferenciados (SUPER_ADMIN, ADMIN, USER).
- Desarrollo de las rutas base de autenticación, pruebas con postman.

Sprint 2 — Motor de Orquestación e Integración IA (Semanas 4-6) ([Figura 3.2](#))

- Desarrollo del motor de búsqueda léxica de inventario (erp_search) con tokenización, fuzzy match y caché por empresa.
- Integración del microservicio Python con FastAPI como intermediario hacia la API de DeepSeek.
- Implementación del sistema de prompts jerárquico (NIVEL 0 al NIVEL 4) con protocolo anti-alucinación.
- Desarrollo de la interfaz de chat conversacional en React con historial de mensajes y persistencia de sesiones.

Sprint 3 — Motores Especializados y Panel B2B (Semanas 7-9) ([Figura 3.3](#))

- Implementación del motor analítico con LLM auxiliar para extracción de rangos temporales y análisis de datos por dimensiones dinámicas.
- Desarrollo del motor de ventas con identificación semántica de clientes, historial de compras y desambiguación conversacional.
- Desarrollo del motor de logística con normalización de campos y agregación dinámica de órdenes.
- Construcción del formulario de aprovisionamiento B2B con mapeo dinámico de campos ERP.
- **Primer despliegue en producción:** backend Node.js y microservicio Python en Render, frontend React en Vercel, con validación end-to-end del flujo completo en entorno real.

Sprint 4 — Seguridad, Panel de Administración y Cierre (Semanas 10-12) ([Figura 3.4](#))

- Implementación de medidas de seguridad: rate limiting, validación de ownership de sesiones, historial desde BD, JWT sin fallback, secret compartido Node→Python y caché por empresa.
- Construcción del panel de administración completo: Dashboard con métricas reales, historial de conversaciones, gestión de agentes, configuración de empresa y gestión de usuarios.
- Documentación de la API con Swagger/OpenAPI en archivo YAML centralizado.
- Implementación de logs estructurados con Pino y despliegue final en producción.

Backlog de Historias de Usuario

HU1 — Autenticación y Multi-tenancy Como SUPER_ADMIN, quiero aprovisionar nuevas empresas con sus usuarios administradores y agentes en una sola operación, para que cada tenant opere de forma aislada desde el primer acceso.

HU2 — Motor de Búsqueda de Inventario Como operario de bodega, quiero consultar stock, precios y disponibilidad de productos en lenguaje natural, para obtener respuestas exactas del ERP sin navegar por el sistema de gestión.

HU3 — Motor Analítico Como gerente, quiero consultar métricas de ventas, ingresos y rendimiento por período, técnico o categoría en lenguaje natural, para tomar decisiones estratégicas sin necesitar un analista de datos.

HU4 — Motor de Ventas con Perfil de Cliente Como vendedor, quiero que el agente identifique a un cliente por nombre o RUT y me muestre su perfil e historial de compras, para personalizar mi asesoría comercial en tiempo real.

HU5 — Motor de Logística Como coordinador de operaciones, quiero consultar el estado de órdenes de trabajo por responsable, estado o período, para tener visibilidad operativa sin acceder al sistema de gestión.

HU6 — Panel de Administración B2B Como SUPER_ADMIN, quiero gestionar todas las empresas de la plataforma desde un panel centralizado, y como ADMIN, quiero personalizar las instrucciones de mis agentes y gestionar los usuarios de mi empresa.

HU7 — Historial de Conversaciones Como usuario, quiero acceder al historial de mis conversaciones anteriores con cada agente, para retomar análisis previos sin perder el contexto de decisiones pasadas.

HU8 — Seguridad y Resiliencia Como desarrollador, quiero que el sistema valide el ownership de cada sesión, limite las peticiones por usuario y autentique la comunicación entre servicios internos, para garantizar que los datos de cada empresa no sean accesibles por otros tenants.

HU9 — Conexión a Endpoints Como dueño de mi negocio quiero tener información en tiempo real de mi base de datos, como el stock disponible, precio entre otros, sin perder la seguridad de mi base de datos.

Servicios Cloud a utilizar

Justificación estratégica del modelo PaaS

A diferencia de la propuesta inicial que contemplaba un Servidor Privado Virtual (VPS) con MySQL local, la arquitectura definitiva de AgenteX adoptó un modelo basado íntegramente en Plataforma como Servicio (PaaS). Esta decisión se fundamenta en tres ventajas competitivas para el modelo B2B multi-tenant:

- 1. Escalabilidad independiente por servicio:** Al separar el frontend, el backend Node.js, el microservicio Python y la base de datos en plataformas especializadas, cada componente puede escalar de forma autónoma según su carga sin afectar al resto del sistema. Esto es crítico en una plataforma multi-tenant donde el volumen de peticiones varía significativamente entre empresas.
- 2. Reducción de carga operativa:** Las plataformas PaaS seleccionadas gestionan automáticamente los certificados SSL, los reintentos ante fallos, los despliegues continuos desde GitHub y la disponibilidad del servicio. Esto libera al equipo de desarrollo de tareas de administración de infraestructura, permitiendo enfocarse en la lógica de negocio.
- 3. Costo optimizado para MVP:** El modelo de facturación de las plataformas seleccionadas permite operar en un plan gratuito o de bajo costo durante la fase de validación, con capacidad de migrar a planes de mayor capacidad sin cambios en la arquitectura ni en el código.

Arquitectura de despliegue en la nube

Frontend — Vercel (PaaS)

- **Tecnología:** React + Vite desplegado en Vercel con CDN global.
- **Justificación:** Vercel está optimizado para aplicaciones React con builds automáticos desde GitHub en cada push a la rama principal. Su CDN distribuido garantiza tiempos de carga mínimos independientemente de la ubicación geográfica del usuario. La configuración de `vercel.json` permite el manejo correcto de rutas SPA sin recargas de página.

Backend y API — Render (PaaS)

- **Tecnología:** Node.js + Express desplegado como Web Service en Render.
- **Justificación:** Render gestiona el ciclo de vida completo del servidor Node.js, incluyendo reinicio automático ante fallos, despliegue continuo desde GitHub y gestión de variables de entorno de forma segura. El backend expone la API REST que consume el frontend y orquesta la comunicación con el microservicio Python y Supabase.

Microservicio de IA – Render (PaaS)

- **Tecnología:** Python + FastAPI desplegado como Web Service independiente en Render.
- **Justificación:** La separación del motor de IA en un microservicio independiente permite actualizar el modelo de lenguaje o la lógica de procesamiento sin afectar al backend principal. La comunicación entre Node.js y Python se autentifica mediante un secret compartido (X-Internal-Secret) para garantizar que el endpoint no sea accesible públicamente.

Base de datos relacional – Supabase (DBaaS)

- **Tecnología:** PostgreSQL administrado en Supabase.
- **Justificación:** Supabase provee PostgreSQL con integridad referencial completa, backups automáticos, panel de administración visual y cliente JavaScript nativo. El esquema multi-tenant con llaves foráneas garantiza el aislamiento de datos entre empresas. Se descartó MySQL local en VPS por la complejidad operativa que implica su administración manual en un proyecto unipersonal.

Resumen de servicios utilizados

Componente	Plataforma	Tipo	Plan
Frontend React	Vercel	PaaS	Hobby (gratuito)
Backend Node.js	Render	PaaS	Free Web Service
Microservicio Python	Render	PaaS	Free Web Service
Base de datos	Supabase	DBaaS	Free tier
API de IA	DeepSeek API	SaaS	Pay per use

Conceptualización

Descripción de la solución

AgenteX es una plataforma web SaaS multi-tenant de Inteligencia de Negocios Conversacional que permite a empresas PyME interactuar con sus propios datos operativos en tiempo real mediante lenguaje natural. En el nuevo modelo operativo, el directivo o empleado deja de ser un recolector manual de datos para convertirse en un consultor de su propia información: formula una pregunta en lenguaje natural y el sistema orquesta automáticamente la consulta al motor correspondiente, extrae los datos reales desde los endpoints de la empresa y entrega una respuesta sintetizada, precisa y fundamentada exclusivamente en información verificable.

La plataforma opera bajo una arquitectura de tres capas de servicios cloud independientes — frontend en Vercel, backend Node.js en Render y microservicio Python en Render — conectados a una base de datos relacional PostgreSQL en Supabase. Cada empresa cliente opera en un tenant aislado con sus propios agentes, configuraciones de ERP y usuarios, sin compartir datos ni contexto con otros clientes de la plataforma.

El núcleo del sistema es el orquestador de motores en Node.js, que determina qué motor de procesamiento utilizar según el tipo de agente configurado para cada empresa, extrae los datos relevantes del ERP en tiempo real, construye un sistema prompt jerárquico con esos datos y delega la redacción de la respuesta al microservicio Python que se comunica con la API de DeepSeek.

Funcionalidades principales

1. Orquestación de motores especializados por dominio: La plataforma implementa cuatro motores de procesamiento independientes según el tipo de agente: motor de búsqueda léxica de inventario (erp_search) con tokenización, fuzzy match y caché por empresa; motor analítico con extracción de rangos temporales mediante LLM auxiliar y análisis por dimensiones dinámicas; motor de ventas con identificación semántica de clientes, historial de compras y flujo de desambiguación conversacional; y motor de logística con normalización de campos y agregación dinámica de órdenes de trabajo.

2. Protocolo anti-alucinación jerárquico: Cada agente opera bajo un sistema prompt estructurado en cinco niveles de prioridad (NIVEL 0 al NIVEL 4) que establece una jerarquía de instrucciones irrompible. El NIVEL 0 define la ley absoluta anti-alucinación: el agente sólo puede afirmar datos que aparezcan explícitamente en los datos recibidos del ERP. Los niveles posteriores definen la identidad del agente, el contexto del negocio, las restricciones globales y los datos operacionales en tiempo real.

3. Aprovisionamiento B2B multi-tenant: El SUPER_ADMIN puede crear nuevas empresas desde un formulario web que incluye mapeo dinámico de campos ERP mediante detección automática de claves desde JSON de ejemplo, configuración de endpoints secundarios, business context del rubro y asignación de agentes con instrucciones personalizadas. Cada empresa queda operativa inmediatamente tras el aprovisionamiento sin intervención en la base de datos.

4. Panel de administración por roles diferenciados: La plataforma implementa tres niveles de acceso con interfaces específicas para cada rol. El SUPER_ADMIN gestiona todas las empresas, sus suscripciones y configuraciones técnicas de ERP. El ADMIN de cada empresa personaliza las instrucciones conversacionales de sus agentes y gestiona los usuarios de su tenant. El USER accede únicamente al chat conversacional y su historial personal.

5. Historial de conversaciones con trazabilidad completa: Cada mensaje intercambiado entre el usuario y el agente se persiste en la base de datos con registro de tokens consumidos, tipo de remitente y timestamp. El usuario puede acceder al historial de sesiones anteriores y retomar conversaciones previas. El historial se reconstruye siempre desde la base de datos — nunca desde el cliente — eliminando el riesgo de inyección de mensajes falsos.

6. Gestión dinámica del mapeo de campos ERP: La plataforma implementa un sistema de mapeo agnóstico al rubro que permite a cualquier empresa conectar su ERP sin importar la nomenclatura de sus campos. El administrador define la equivalencia entre los nombres técnicos del JSON del ERP y los roles semánticos que el sistema entiende, configurando también endpoints secundarios para stock real, clientes, historial de compras y órdenes de trabajo.

Entregables

Al finalizar el desarrollo, se entregan los siguientes productos:

- **Plataforma web funcional:** Sistema desplegado en producción con URL pública, accesible desde cualquier navegador, con módulos de autenticación, chat conversacional, historial y panel de administración operativos.
- **Repositorio de código fuente:** Código base de backend Node.js, microservicio Python y frontend React alojado en GitHub con convenciones de nomenclatura kebab-case y estructura de carpetas documentada.
- **Documentación de API:** Swagger/OpenAPI completo en archivo YAML centralizado, accesible en /api/docs, documentando las 20 rutas de la API con schemas, ejemplos y códigos de respuesta.
- **Informe técnico:** El presente documento detalla la arquitectura, decisiones de diseño, planificación y estado de avance del sistema.

Consideraciones de calidad y seguridad

La plataforma implementa las siguientes medidas de seguridad en producción:

- **Autenticación JWT sin fallback:** Tokens firmados con secret aleatorio de 64 bytes almacenado en variables de entorno. El servidor rechaza cualquier request sin token válido y redirige automáticamente al login cuando el token expira.
- **Validación de ownership:** Cada acceso a sesiones, mensajes o recursos de empresa verifica que el recurso pertenece al usuario autenticado antes de procesarlo.

- **Historial desde base de datos:** El historial de conversaciones se reconstruye siempre desde Supabase, eliminando la posibilidad de que el cliente inyecte mensajes falsos en el contexto del agente.
- **Rate limiting diferenciado:** El endpoint `/api/chat` permite 20 peticiones por minuto por IP para proteger el consumo de tokens de DeepSeek. Las rutas de configuración permiten 100 peticiones por minuto.
- **Secret compartido entre servicios:** La comunicación entre Node.js y el microservicio Python se autentifica mediante el header `X-Internal-Secret`, impidiendo el acceso directo al motor de IA desde el exterior.
- **Caché aislado por empresa:** El caché de datos del ERP utiliza la clave `company_id:url`, garantizando que dos empresas con el mismo endpoint no compartan datos en memoria.
- **Logs estructurados:** Implementación de Pino para registro estructurado de eventos con campos indexables por empresa, motor y tiempo de respuesta, facilitando la correlación de errores en producción.

Diagrama de arquitectura de la solución

El diagrama de bloques de la arquitectura implementada se presenta en los Anexos [\(Figura 4\)](#).

Alcance, supuestos y restricciones

Alcance del Proyecto

El desarrollo de AgenteX delimita su entrega a un producto funcional y desplegado en producción que incluye las siguientes funcionalidades implementadas:

- **Módulo de autenticación multi-tenant:** Sistema de login con JWT, gestión de roles diferenciados (SUPER_ADMIN, ADMIN, USER) y aislamiento completo de datos entre empresas cliente.
- **Chat conversacional con cuatro motores especializados:** Interfaz de chat asíncrona que enruta cada consulta al motor correspondiente según el tipo de agente configurado — búsqueda de inventario, analítica temporal, asesoría comercial con perfil de cliente, y consulta de órdenes logísticas — conectándose en tiempo real a los endpoints REST de cada empresa.
- **Protocolo anti-alucinación:** Sistema prompt jerárquico de cinco niveles que restringe las respuestas del modelo de lenguaje exclusivamente a los datos reales recibidos desde los endpoints configurados.
- **Panel de aprovisionamiento B2B:** Formulario web para que el SUPER_ADMIN cree empresas con agentes, usuarios y configuración de ERP en una sola operación, incluyendo mapeo dinámico de campos mediante detección automática desde JSON de ejemplo.
- **Panel de administración por roles:** Vistas diferenciadas para SUPER_ADMIN (gestión de todas las empresas), ADMIN (personalización de instrucciones de agentes y gestión de usuarios de su empresa) y USER (chat e historial personal).
- **Historial de conversaciones:** Persistencia de sesiones y mensajes en base de datos con registro de tokens consumidos, accesible desde el panel de historial con capacidad de eliminación por sesión.
- **API REST documentada:** 20 rutas documentadas con Swagger/OpenAPI, incluyendo autenticación, chat, agentes, usuarios, sesiones, empresa y administración.
- **Infraestructura cloud en producción:** Frontend en Vercel, backend Node.js y microservicio Python en Render, base de datos PostgreSQL en Supabase, con variables de entorno configuradas y comunicación segura entre servicios.

No Alcance

Las siguientes funcionalidades quedan explícitamente fuera de esta entrega:

- **Motor RAG (Retrieval-Augmented Generation):** La ingesta, vectorización y consulta de documentos privados corporativos (PDFs, manuales) fue evaluada y postergada para una fase posterior. El esquema de base de datos contempla las tablas necesarias para su implementación futura (documents, company_agent_documents).
- **Aplicaciones móviles nativas:** El acceso es exclusivamente mediante navegador web responsivo. No se desarrollarán aplicaciones para iOS o Android.

- **Pasarelas de pago e integración de facturación:** El sistema no procesa cobros ni gestiona suscripciones de forma automatizada. La gestión del estado de suscripción es manual desde el panel SUPER_ADMIN.
- **Conexión directa a bases de datos de terceros:** El sistema no se conecta mediante credenciales SQL a los ERPs de los clientes. Los datos deben estar expuestos mediante endpoints REST que devuelvan JSON.

Entregables

1. Plataforma web desplegada y operativa con URL pública en Vercel y Render.
2. Repositorio de código fuente en GitHub con estructura kebab-case y commits documentados.
3. Documentación de API en Swagger/OpenAPI accesible en **/api/docs**.
4. Informe técnico completo (el presente documento).

Supuestos

- Las empresas usuarias poseen endpoints REST operativos que devuelven respuestas en formato JSON para sus datos de inventario, ventas, clientes u órdenes de trabajo.
- La API de DeepSeek mantiene su disponibilidad durante el período de operación y evaluación del sistema.
- El administrador de cada empresa cuenta con los datos de conexión a sus endpoints al momento del onboarding con el SUPER_ADMIN.
- Los navegadores web de los usuarios soportan JavaScript moderno y conexiones HTTPS, requisito mínimo para acceder a la plataforma.

Restricciones

- **Restricción de tiempo:** El ciclo de desarrollo estuvo limitado a 12 semanas con un único desarrollador full-stack, lo que obligó a priorizar la estabilidad del núcleo funcional y postergar el módulo RAG.
- **Restricción de recursos humanos:** Al tratarse de un proyecto académico unipersonal, el 100% del desarrollo — arquitectura, base de datos, backend, frontend, microservicio Python e integración de IA — fue ejecutado por un único desarrollador, limitando la paralelización de tareas complejas.
- **Restricción tecnológica (rate limits):** La velocidad de respuesta del sistema está condicionada a los límites de peticiones por minuto de la API de DeepSeek y a los tiempos de cold start de los servicios gratuitos de Render, que pueden introducir latencia adicional en la primera petición tras un período de inactividad.
- **Restricción de formato de datos:** Los endpoints ERP de las empresas cliente deben devolver datos en formato JSON. Formatos propietarios, conexiones SQL directas o archivos binarios no son compatibles con la arquitectura actual.

Metodología de desarrollo y gestión del proyecto

Metodología elegida

El proyecto adoptó una metodología ágil híbrida basada en Scrum con elementos de Kanban, adaptada a las condiciones de un proyecto unipersonal con entregas académicas periódicas. La elección de esta metodología se fundamenta en la naturaleza iterativa del desarrollo de sistemas de inteligencia artificial, donde los requisitos técnicos evolucionan con cada integración y los resultados de cada sprint retroalimentan las decisiones de diseño del siguiente.

A diferencia de una metodología en cascada, el enfoque ágil permitió pivotar decisiones arquitecturales críticas durante el desarrollo — como el abandono del VPS con MySQL a favor de una arquitectura cloud PaaS, o la sustitución del módulo RAG por motores especializados por dominio — sin comprometer la estabilidad de los componentes ya implementados. Cada sprint entregó un incremento funcional verificable, lo que garantizó que el sistema tuviera valor operativo en todo momento del desarrollo, no solo al final.

Forma de trabajo semana a semana

El ciclo de trabajo semanal se estructuró en tres fases:

- **Planificación (inicio de semana):** Definición de las tareas del sprint en curso, priorización según dependencias técnicas e identificación de riesgos de integración entre servicios.
- **Desarrollo y validación (mitad de semana):** Implementación de funcionalidades con validación incremental en entorno local. Cada cambio significativo en el backend fue verificado mediante pruebas directas con Postman o la interfaz de Swagger antes de integrarse al frontend.
- **Revisión y ajuste (fin de semana):** Evaluación del avance respecto a los objetivos del sprint, identificación de deuda técnica acumulada y ajuste de prioridades para la semana siguiente.

Roles del equipo

Dado que el proyecto fue ejecutado íntegramente por un único desarrollador, los roles de Scrum fueron asumidos de forma rotativa según la naturaleza de la tarea en curso:

Rol	Responsabilidad en el proyecto
Product Owner	Definición de historias de usuario, criterios de aceptación y priorización del backlog según valor de negocio
Scrum Master	Gestión del ritmo de sprints, identificación de bloqueos técnicos y ajuste de alcance ante restricciones de tiempo
Desarrollador Full-Stack	Implementación de arquitectura, base de datos, backend Node.js, microservicio Python y frontend React
DevOps	Configuración de entornos de despliegue en Render y Vercel, gestión de variables de entorno y secretos

Definición de "Hecho"

Una funcionalidad se considera completada cuando cumple los siguientes criterios:

- El código está implementado y funciona correctamente en el entorno local.
- La funcionalidad fue probada en el entorno de producción (Render/Vercel) con datos reales.
- Los endpoints involucrados están documentados en el archivo **swagger.yaml**.
- No introduce regresiones en funcionalidades previamente implementadas.
- Las variables de entorno necesarias están configuradas en Render sin exposición de secretos en el repositorio.

Gestión del repositorio

El código fuente se gestiona en un repositorio GitHub con dos ramas principales:

- **main**: Rama de producción. Cada merge a esta rama desencadena un despliegue automático en Render y Vercel mediante integración continua.
- **dev**: Rama de desarrollo activo donde se implementan y validan nuevas funcionalidades antes de su promoción a producción.

Las convenciones de nomenclatura adoptadas para el código incluyen kebab-case para nombres de archivos (**erp-search.service.js**, **chat.controller.js**), camelCase para variables y funciones JavaScript, y commits descriptivos con prefijos semánticos (**feat:**, **fix:**, **refactor:**).

Tabla de los Hitos más importantes con periodo.

Sprint	Hito principal	Período
Sprint 1	Diseño e implementación del esquema relacional en Supabase, sistema de autenticación JWT con roles diferenciados y configuración del entorno de desarrollo con Node.js y Express	Abril 27, 2026
Sprint 2	Motor de búsqueda léxica de inventario (erp_search) operativo, integración del microservicio Python con FastAPI y DeepSeek, sistema prompt jerárquico anti-alucinación e interfaz de chat conversacional en React	Mayo 6, 2026
Sprint 3	Motor analítico, motor de ventas con identificación semántica de clientes, motor de logística, formulario de aprovisionamiento B2B con mapeo dinámico de campos ERP y primer despliegue en producción (Render y Vercel)	Mayo 14, 2026
Sprint 4	Implementación de seguridad completa, panel de administración con vistas diferenciadas por rol, documentación Swagger/OpenAPI y logs estructurados con Pino	Mayo 22, 2026

Como evidencia de la gestión del proyecto, se presenta en la [Figura 5](#) el tablero de tareas actualizado en Trello, donde se visualizan las historias de usuario organizadas por estado — backlog, en progreso y completado — reflejando el avance real del desarrollo a lo largo de los cuatro sprints. Adicionalmente, la [Figura 6](#) muestra el historial de commits del repositorio GitHub, cuya progresión cronológica es consistente con los hitos definidos en la tabla anterior, evidenciando la cadencia de entregas iterativas y el avance incremental del sistema en cada sprint.

Documentos y diagramas de diseño de la solución

A continuación se describen los documentos y diagramas que formalizan el diseño de AgenteX, explicando el propósito de cada uno y su relación con el problema y los objetivos del proyecto.

1. Árbol de problema/oportunidad ([Figura 1](#))

El árbol de problema representa la anatomía del fallo operativo que da origen al proyecto. En la raíz se ubican las tres causas estructurales identificadas: los silos de datos desconectados, las interfaces de dashboards rígidas y las limitaciones de la IA pública sin contexto privado. El tronco representa el problema central: la extracción manual y lenta de datos corporativos genera latencia en la toma de decisiones. Las ramas superiores muestran los efectos: toma de decisiones reactiva, subutilización del capital de datos y sobrecarga administrativa con dependencia técnica.

Este diagrama fundamenta directamente los objetivos específicos del proyecto — cada causa identificada tiene un objetivo técnico que la ataca — y justifica la decisión de construir motores especializados conectados a endpoints reales en vez de depender de IA genérica.

2. Diagrama de flujo del proceso actual — AS-IS ([Figura 2](#))

El diagrama de flujo representa el proceso manual que sigue hoy un directivo o encargado para extraer información estratégica de la empresa. Muestra las dos rutas paralelas de búsqueda — acceso al ERP/CRM para datos duros y lectura de documentos para contexto cualitativo — que convergen en el punto de dolor central: el cruce manual de información y la redacción de reportes. La salida del proceso es un reporte estático entregado con horas o días de retraso.

Este diagrama contrasta directamente con el flujo TO-BE implementado en AgenteX, donde el usuario formula una pregunta y recibe la respuesta en segundos sin intermediarios.

3. Diagrama de arquitectura de la solución — TO-BE ([Figura 4](#))

El diagrama de bloques representa la arquitectura cloud implementada, mostrando los cinco componentes principales del sistema y sus relaciones de comunicación:

- **Usuario/Directivo** → accede mediante navegador al frontend React desplegado en Vercel.
- **Frontend React (Vercel)** → envía las consultas al backend Node.js mediante peticiones HTTP REST autenticadas con JWT.
- **Backend Node.js (Render)** → orquesta el flujo completo: autentica el request, determina el motor a usar, consulta los endpoints ERP de la empresa, construye el sistema prompt jerárquico con los datos reales e invoca al microservicio Python.
- **Microservicio Python/FastAPI (Render)** → recibe el sistema prompt con datos ya procesados y llama a la API de DeepSeek para generar la respuesta en lenguaje natural.
- **Supabase (PostgreSQL)** → persiste usuarios, empresas, agentes, sesiones y mensajes con integridad referencial completa.

- **Endpoints ERP de la empresa** → fuentes de datos reales consultadas en tiempo real por los motores especializados de Node.js.

4. Modelo de datos — Diagrama Entidad Relación ([Figura 7](#))

El modelo relacional implementado en Supabase define las siguientes entidades principales y sus relaciones:

- **companies:** Entidad central del modelo multi-tenant. Almacena nombre, `erp_mapping` (JSON con URLs y mapeo de campos), `business_context` y estado de suscripción.
- **users:** Vinculados a una empresa mediante `company_id`. Tienen rol diferenciado (`SUPER_ADMIN`, `ADMIN`, `USER`) y credenciales hasheadas con `bcrypt`.
- **agent_templates:** Catálogo de tipos de agentes disponibles en la plataforma con su motor asociado (`erp_search`, `analytics`, `ventas`, `logistica`) e instrucciones base.
- **company_agents:** Tabla pivote que vincula una empresa con un template de agente, almacenando las instrucciones personalizadas y temperatura configuradas para ese tenant específico.
- **session_chats:** Sesiones de conversación vinculadas a un usuario y a un `company_agent`, con registro de estado y visibilidad.
- **messages:** Mensajes individuales de cada sesión con contenido, tipo de remitente (`USER/IA`) y tokens consumidos (`prompt` y `completion`).

Las llaves foráneas garantizan que la eliminación de una empresa en cascada elimine sus agentes, usuarios, sesiones y mensajes asociados, manteniendo la integridad referencial del sistema.

5. Diagrama de casos de uso ([Figura 8](#))

El diagrama de casos de uso representa las interacciones de los tres actores del sistema con las funcionalidades de la plataforma:

- **SUPER_ADMIN:** Aprovisionar empresa, configurar agentes y ERP mapping, gestionar todas las empresas, modificar estado de suscripción, eliminar empresa.
- **ADMIN:** Personalizar instrucciones de agentes de su empresa, gestionar usuarios de su empresa, consultar configuración de empresa (solo lectura para ERP).
- **USER:** Interactuar con agentes mediante chat conversacional, consultar historial de sesiones, eliminar sesiones propias, actualizar perfil personal.

6. Diagrama de flujo del proceso nuevo — TO-BE ([Figura 9](#))

El diagrama de flujo TO-BE contrasta con el AS-IS mostrando el nuevo proceso simplificado con AgenteX:

1. El usuario formula una pregunta en lenguaje natural en el chat.
2. El backend identifica el motor correspondiente al agente seleccionado.
3. El motor consulta los endpoints ERP en tiempo real y procesa los datos.
4. El sistema `prompt` jerárquico inyecta los datos reales al contexto del LLM.

5. El microservicio Python genera la respuesta fundamentada exclusivamente en los datos recibidos.
6. El usuario recibe la respuesta en segundos, con los datos persistidos en la base de datos para trazabilidad.

Arquitectura de software y patrones de diseño

Arquitectura elegida

AgenteX implementa una arquitectura de **microservicios con separación de responsabilidades en tres capas**, donde cada servicio tiene un dominio técnico exclusivo y se comunica con los demás mediante contratos HTTP bien definidos:

- **Capa de presentación:** Frontend React en Vercel — responsable exclusivamente de la interfaz de usuario, el estado del chat y la navegación.
- **Capa de orquestación:** Backend Node.js en Render — responsable de la autenticación, la lógica de negocio, el enrutamiento de motores y la persistencia en Supabase.
- **Capa de procesamiento IA:** Microservicio Python/FastAPI en Render — responsable exclusivamente de la comunicación con la API de DeepSeek y la generación de respuestas en lenguaje natural.

Esta separación garantiza que cada capa pueda escalar, actualizarse o reemplazarse de forma independiente sin afectar a las demás. Por ejemplo, el modelo de lenguaje puede migrarse de DeepSeek a otro proveedor modificando únicamente el microservicio Python, sin tocar el backend ni el frontend.

Patrones de diseño implementados

1. Patrón Strategy — Motores especializados por dominio

El patrón Strategy se aplica en el sistema de orquestación de motores del backend. El `chat.controller.js` actúa como contexto que determina dinámicamente qué motor ejecutar según el atributo `motor` configurado en la base de datos para cada agente. Cada motor (`erp-search.service.js`, `analytics.service.js`, `sales-search.service.js`, `logistics-search.service.js`) implementa la misma interfaz conceptual pero con lógica de procesamiento completamente distinta.

`chat.controller.js` (Contexto)

```

├── motor: 'erp_search' → erp-search.service.js
├── motor: 'analytics' → analytics.service.js
├── motor: 'ventas' → sales-search.service.js
└── motor: 'logistica' → logistics-search.service.js

```

Este patrón permite agregar nuevos motores sin modificar el controlador principal — solo se registra el nuevo servicio y se asocia el motor en la base de datos.

2. Patrón Service Layer — Separación de lógica de negocio

Toda la lógica de negocio compleja reside en la capa de servicios, completamente separada de los controladores. Los controladores son responsables únicamente de recibir el request HTTP, invocar el servicio correspondiente y devolver la respuesta. Los servicios encapsulan

la lógica de búsqueda léxica, análisis temporal, identificación de clientes y consulta de órdenes.

Controlador → recibe HTTP, valida, delega

Service → lógica de negocio, fetch ERP, procesamiento

Controller → formatea respuesta HTTP

3. Patrón Multi-tenant con Row-Level Isolation

El aislamiento entre empresas se implementa a nivel de aplicación mediante el `company_id` extraído del JWT en cada request. Cada consulta a Supabase incluye obligatoriamente el filtro `eq('company_id', req.user.company_id)`, garantizando que un usuario nunca pueda acceder a datos de otro tenant aunque conozca el ID del recurso.

4. Patrón Proxy — Microservicio Python como intermediario de respuestas principales

El microservicio Python actúa como proxy entre el backend Node.js y la API de DeepSeek para la **generación de respuestas conversacionales finales**. Sin embargo, Node.js también se comunica directamente con DeepSeek para los **LLM auxiliares** — llamadas de baja latencia y temperatura 0 que realizan tareas específicas de extracción de datos: `extraerIntencion` para identificar el término de búsqueda y filtro del usuario, `extraerRango` para detectar períodos temporales en consultas analíticas, y `extraerCliente` para identificar semánticamente a un cliente en el motor de ventas.

La distribución de responsabilidades entre ambos servicios es la siguiente:

Node.js → DeepSeek directamente:

└─ `extraerIntencion()` → termino y filtro de búsqueda (temp: 0)

└─ `extraerRango()` → fechas inicio/fin para analítica (temp: 0)

└─ `extraerCliente()` → identificación semántica de cliente (temp: 0)

Node.js → Python → DeepSeek:

└─ Respuesta conversacional final al usuario (temp: configurable por empresa)

Esta separación permite que las tareas de extracción estructurada — que requieren respuestas JSON determinísticas — se ejecuten directamente sin la latencia adicional del microservicio, mientras que la generación de lenguaje natural se centraliza en Python.

5. Patrón Cache-Aside — Caché en memoria por empresa

Cada motor implementa un caché en memoria con TTL configurable, donde la clave es `company_id:url`. Los datos del ERP se almacenan en memoria tras el primer fetch y se sirven desde caché en requests subsecuentes dentro del TTL. Si los datos no están en caché o expiraron, se consulta el endpoint real. Este patrón reduce significativamente la latencia de respuesta y el número de peticiones a los ERPs de los clientes.

Justificación arquitectural

La arquitectura de microservicios con estos patrones es la más adecuada para AgenteX por tres razones fundamentales. Primero, el dominio del problema requiere especialización — la lógica de búsqueda léxica de inventario es radicalmente distinta a la lógica de análisis temporal, y mezclarlas en un monolito generaría un sistema difícil de mantener y escalar. Segundo, la separación del microservicio Python permite evolucionar la capa de IA de forma independiente, lo que es crítico en un campo que avanza tan rápido. Tercero, el patrón multi-tenant con aislamiento por `company_id` es la forma más pragmática de garantizar privacidad de datos en una plataforma SaaS sin la complejidad de esquemas de base de datos separados por cliente.

La estructura de comunicación entre los componentes del sistema se representa mediante dos niveles del modelo C4. La [Figura 10](#) presenta el diagrama de Contexto (C4 Nivel 1), mostrando las relaciones de alto nivel entre AgenteX, sus usuarios y los sistemas externos con los que interactúa — la API de DeepSeek y los endpoints REST de cada empresa cliente. La [Figura 11](#) presenta el diagrama de Contenedores (C4 Nivel 2), detallando los cuatro componentes internos del sistema — Frontend React, Backend Node.js, Microservicio Python y Base de Datos PostgreSQL — con sus tecnologías, plataformas de despliegue y relaciones de comunicación, incluyendo los protocolos y mecanismos de seguridad aplicados en cada integración.

Tecnologías, lenguaje y dependencias

El código fuente del proyecto está disponible en:

Lenguajes y frameworks

Capa	Lenguaje	Framework	Versión
Frontend	JavaScript (ES2022)	React 18 + Vite	Node 20+
Backend	JavaScript (ES2022)	Express 4	Node 20+
Microservicio IA	Python 3.11	FastAPI	0.110+

Dependencias principales por servicio

Backend Node.js:

Dependencia	Propósito
<code>express</code>	Framework HTTP para la API REST
<code>@supabase/supabase-js</code>	Cliente oficial de Supabase para operaciones CRUD
<code>jsonwebtoken</code>	Generación y verificación de tokens JWT
<code>bcrypt</code>	Hash de contraseñas con salt rounds
<code>express-rate-limit</code>	Rate limiting por IP diferenciado por ruta
<code>swagger-ui-express + ymljs</code>	Servicio de documentación OpenAPI en <code>/api/docs</code>
<code>pino + pino-pretty</code>	Logs estructurados en JSON para producción
<code>dotenv</code>	Gestión de variables de entorno en desarrollo local

Microservicio Python:

Dependencia	Propósito
<code>fastapi</code>	Framework HTTP asíncrono para el microservicio
<code>uvicorn</code>	Servidor ASGI para ejecutar FastAPI
<code>httpx</code>	Cliente HTTP asíncrono para llamadas a DeepSeek

pydantic	Validación de contratos de entrada y salida
python-dotenv	Gestión de variables de entorno

Frontend React:

Dependencia	Propósito
react-router-dom	Enrutamiento SPA con rutas protegidas
motion	Animaciones declarativas de componentes
lucide-react	Iconografía consistente en toda la interfaz
react-markdown + remark-gfm	Renderizado de Markdown con tablas en las respuestas del agente
tailwindcss	Utilidades CSS para el sistema de diseño

Estándares de código

- **Nomenclatura de archivos:** kebab-case para todos los archivos del backend (`erp-search.service.js`, `chat.controller.js`, `auth.middleware.js`) y PascalCase para componentes React (`ChatInterface.jsx`, `Dashboard.jsx`).
- **Estructura de carpetas backend:** separación estricta por responsabilidad en `controllers/`, `routes/`, `services/`, `middlewares/` y `config/`.
- **Estructura de carpetas frontend:** componentes organizados por dominio en `components/auth/`, `components/chat/`, `components/dashboard/`, `components/settings/`, `components/history/`, `components/admin/` y `components/provisioning/`.
- **Variables de entorno:** todos los secretos y URLs de servicios externos se gestionan mediante variables de entorno. Ningún secreto está hardcodeado en el repositorio. El archivo `.env` está incluido en `.gitignore`.
- **Convención de commits:** prefijos semánticos `feat:`, `fix:`, `refactor:`, `chore:` para mantener un historial legible y trazable.

Repositorio y flujo de trabajo

El proyecto se distribuye en tres repositorios independientes en GitHub, uno por cada servicio de la arquitectura. El repositorio del backend Node.js, disponible en [\[URL repositorio backend\]](#), contiene los controladores, rutas, servicios y middlewares de la API REST. El repositorio del frontend React, disponible en [\[URL repositorio frontend\]](#), contiene los componentes, vistas y la lógica de navegación de la interfaz de usuario. El repositorio del microservicio Python, disponible en [\[URL repositorio microservicio\]](#), contiene el motor de IA con FastAPI y la integración con la API de DeepSeek. Los tres repositorios siguen la misma convención de ramas — `main` para producción y `dev` para desarrollo activo — con despliegue automático en Render y Vercel en cada merge a `main`.

- **Repositorio:** GitHub con dos ramas principales — `main` para producción y `dev` para desarrollo activo.
- **CI/CD:** Render y Vercel están conectados al repositorio GitHub con despliegue automático en cada merge a `main`. No se requiere intervención manual para desplegar a producción.
- **Documentación de API:** archivo `swagger.yaml` en la raíz del backend, servido en `/api/docs` mediante `swagger-ui-express`.

El repositorio del proyecto se encuentra disponible en GitHub en la URL indicada anteriormente. La Figura X muestra la estructura de carpetas del proyecto, evidenciando la separación por responsabilidades adoptada. La [Figura 6](#) presenta el historial de commits del repositorio, cuya progresión es consistente con los sprints definidos en la planificación.

Configuración de servidor de producción

AgenteX se despliega en tres plataformas cloud independientes. A continuación se detallan las instrucciones de configuración para cada servicio.

1. Base de datos — Supabase

Requisitos: Cuenta en supabase.com.

Paso a paso:

1. Crear un nuevo proyecto en el dashboard de Supabase, seleccionando región más cercana al servidor de backend (US East para Render).
 2. En el editor SQL de Supabase, ejecutar el script DDL del proyecto que define las tablas `companies`, `users`, `agent_templates`, `company_agents`, `session_chats` y `messages` con sus llaves foráneas y relaciones en cascada.
 3. Copiar las credenciales desde **Settings** → **API**:
 - `SUPABASE_URL` → URL del proyecto
 - `SUPABASE_KEY` → anon/service key
 4. Insertar los templates de agentes iniciales en la tabla `agent_templates` con sus motores correspondientes (`erp_search`, `analytics`, `ventas`, `logistica`).
 5. Crear el primer usuario `SUPER_ADMIN` directamente en la tabla `users` con contraseña hasheada.
-

2. Microservicio Python — Render

Requisitos: Cuenta en render.com, repositorio GitHub con el código del microservicio.

Paso a paso:

1. En el dashboard de Render, crear un nuevo **Web Service** apuntando al repositorio GitHub del microservicio Python.
2. Configurar los parámetros del servicio:
 - **Runtime:** Python 3
 - **Build Command:** `pip install -r requirements.txt`
 - **Start Command:** `uvicorn main:app --host 0.0.0.0 --port $PORT`
3. En la sección **Environment**, agregar las variables de entorno:

`DEEPSEEK_API_KEY` = <clave de API de DeepSeek>

`INTERNAL_SECRET` = <string aleatorio de 32 bytes>

4. Verificar el despliegue accediendo a `/docs` del servicio para confirmar que FastAPI está operativo.

5. Copiar la URL pública del servicio (<https://motor-ia.onrender.com>) para configurarla en el backend Node.js.

3. Backend Node.js — Render

Requisitos: Cuenta en render.com, repositorio GitHub con el código del backend.

Paso a paso:

1. En el dashboard de Render, crear un nuevo **Web Service** apuntando al repositorio GitHub del backend Node.js.
2. Configurar los parámetros del servicio:
 - **Runtime:** Node
 - **Build Command:** `npm install`
 - **Start Command:** `node server.js`
3. En la sección **Environment**, agregar las variables de entorno:

SUPABASE_URL = <URL del proyecto Supabase>

SUPABASE_KEY = <service key de Supabase>

JWT_SECRET = <string aleatorio de 64 bytes>

DEEPSEEK_API_KEY = <clave de API de DeepSeek>

INTERNAL_SECRET = <mismo string que en el microservicio Python>

PYTHON_ENGINE_URL = <URL pública del microservicio Python>/api/ia/process

NODE_ENV = production

LOG_LEVEL = info

4. Verificar el despliegue accediendo a `/health` del servicio, que debe devolver `{ "status": "ok" }`.
5. Verificar la documentación de la API accediendo a `/api/docs`.
6. Copiar la URL pública del servicio (<https://backend-agentex.onrender.com>) para configurarla en el frontend.

Generación de secretos seguros:

Los valores de `JWT_SECRET` e `INTERNAL_SECRET` deben generarse con el siguiente comando en la terminal local antes de configurarlos en Render:


```
bash
```

```
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
```

4. Frontend React — Vercel

Requisitos: Cuenta en vercel.com, repositorio GitHub con el código del frontend.

Paso a paso:

1. En el dashboard de Vercel, importar el repositorio GitHub del frontend.
2. Configurar los parámetros del proyecto:
 - **Framework Preset:** Vite
 - **Build Command:** `npm run build`
 - **Output Directory:** `dist`
3. En la sección **Environment Variables**, agregar:

VITE_API_URL = <URL pública del backend Node.js en Render>

4. En la raíz del proyecto, verificar que existe el archivo `vercel.json` con la configuración de rewrite para SPA:

```
json
```

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

5. Vercel detecta automáticamente el framework Vite y despliega en cada push a `main`.
 6. Verificar el despliegue accediendo a la URL pública del proyecto en Vercel.
-

5. Verificación del flujo completo en producción

Una vez desplegados los tres servicios, verificar el flujo end-to-end:

1. Acceder a la URL de Vercel e iniciar sesión con el usuario `SUPER_ADMIN`.
2. Crear una empresa desde el panel de aprovisionamiento con al menos un agente activo.
3. Iniciar sesión con el usuario `ADMIN` de la empresa creada.
4. Seleccionar el agente y enviar una consulta al chat.
5. Verificar en los logs de Render que el request pasó correctamente por Node.js y el microservicio Python, con tiempo de respuesta registrado por Pino.

Evidencia de despliegue exitoso

La plataforma está operativa en producción con URL pública accesible, comunicación verificada entre los tres servicios y documentación de API disponible en `/api/docs`.

Desde la [Figura 12](#) hasta la [Figura 17](#) se presentan evidencias del despliegue exitoso de los tres servicios en producción. Se puede verificar el estado operativo del backend Node.js y el microservicio Python en Render, la configuración de variables de entorno sin exposición de secretos, el frontend desplegado en Vercel con URL pública accesible y el esquema de base de datos implementado en Supabase con las seis tablas del modelo relacional.

Estado de avance del desarrollo

El sistema AgenteX se encuentra en estado de producto funcional completo desplegado en producción. A continuación se detalla el estado de cada funcionalidad implementada con su evidencia correspondiente.

Funcionalidades implementadas

#	Funcionalidad	Estado	Evidencia
1	Autenticación JWT multi-tenant con roles	✓ Implementada	Login operativo, token con <code>company_id</code> y <code>role</code> , redirección automática al expirar
2	Motor de búsqueda de inventario (erp_search)	✓ Implementada	Búsqueda léxica con fuzzy match, caché por empresa, filtros de stock crítico y ranking
3	Motor analítico con rangos temporales	✓ Implementada	LLM auxiliar extrae fechas exactas, análisis por dimensiones dinámicas, modo crudo y pre-agregado
4	Motor de ventas con perfil de cliente	✓ Implementada	Identificación semántica en dos pasos, desambiguación con tabla de candidatos, historial de compras
5	Motor de logística	✓ Implementada	Normalización de arrays, consulta de órdenes por estado/responsable/período
6	Protocolo anti-alucinación jerárquico	✓ Implementada	Sistema prompt de 5 niveles, agente restringido a datos reales del ERP

7	Aprovisionamiento B2B desde formulario web	✓ Implementada	Crea empresa, usuario admin y agentes en una sola operación con mapeo dinámico de ERP
8	Panel SUPER_ADMIN — gestión de empresas	✓ Implementada	Lista, edita y elimina empresas con confirmación de eliminación en cascada
9	Panel ADMIN — gestión de agentes	✓ Implementada	Edita instrucciones y temperatura, desactiva agentes (soft delete), agrega nuevos
10	Panel ADMIN — gestión de usuarios	✓ Implementada	Lista usuarios de la empresa, elimina con protección contra auto-eliminación
11	Configuración de empresa	✓ Implementada	ADMIN edita nombre y contexto del negocio. ERP mapping solo editable por SUPER_ADMIN
12	Historial de conversaciones	✓ Implementada	Lista sesiones por agente, visualiza mensajes con tokens consumidos, elimina sesiones
13	Dashboard con métricas reales	✓ Implementada	Sesiones, preguntas y tokens por agente desde base de datos en tiempo real
14	API REST documentada con Swagger	✓ Implementada	20 rutas documentadas en <code>/api/docs</code> con schemas, ejemplos y códigos de respuesta

15	Seguridad en producción	✓ Implementada	Rate limiting, JWT sin fallback, historial desde BD, X-Internal-Secret, caché por empresa
16	Logs estructurados con Pino	✓ Implementada	JSON indexable por <code>company_id</code> , motor y tiempo de respuesta en producción
17	Interceptor de token expirado	✓ Implementada	Redirección automática al login en cualquier componente del frontend
18	Motor RAG	⌚ Pendiente	Postergado para fase siguiente. Tablas en BD preparadas para implementación futura

Calidad y seguridad aplicadas

Validaciones y manejo de errores:

- Todos los endpoints validan la presencia del token JWT antes de procesar cualquier request.
- El middleware de autenticación verifica que `JWT_SECRET` esté definido en variables de entorno — si no está configurado, el servidor devuelve 500 en vez de usar un fallback inseguro.
- Cada operación de escritura verifica que el recurso pertenece al tenant del usuario autenticado antes de modificarlo.
- Los fetches a endpoints ERP tienen timeout de 15 segundos con `AbortSignal.timeout()` y manejo de errores controlado que no expone detalles internos al cliente.
- El formulario de aprovisionamiento incluye rollback manual — si la creación del usuario falla, elimina la empresa recién creada para evitar datos huérfanos.

Control de acceso:

- Las rutas de administración de empresas (`/api/admin/companies`) solo son accesibles con rol `SUPER_ADMIN`.

- Las rutas de gestión de agentes y usuarios (`/api/agents`, `/api/users`) verifican rol ADMIN o superior.
- El historial de conversaciones se reconstruye siempre desde la base de datos — el cliente nunca es fuente de verdad del historial, eliminando el riesgo de inyección de mensajes falsos.

Resiliencia:

- El sistema de caché en memoria con TTL de 60 segundos reduce la carga sobre los endpoints ERP y mejora la latencia de respuesta para consultas repetidas dentro de la misma sesión.
- El motor de ventas implementa un sistema de desambiguación conversacional que mantiene el contexto entre turnos sin requerir que el usuario repita información.
- El microservicio Python implementa un límite de 3 iteraciones de tool calls para evitar bucles infinitos en la ejecución de herramientas.

Evidencias del producto funcional

Las siguientes capturas de pantalla y logs de producción se presentan en los Anexos como evidencia del estado funcional del sistema:

- [Figura 18](#): Pantalla de login y Dashboard con métricas reales de sesiones y tokens por agente.
- [Figura 19](#): Interfaz de chat conversacional con respuesta del agente de bodega mostrando resultados reales del ERP.
- [Figura 20](#): Interfaz de chat del agente de ventas mostrando el flujo de desambiguación de cliente con tabla de candidatos
- [Figura 21](#): Interfaz de chat del agente de analítico mostrando comparativa de periodos con tabla de candidatos
- [Figura 22](#): Interfaz de chat del agente de logística mostrando ordenes de trabajo pendientes.
- [Figura 23](#): Panel de aprovisionamiento B2B con mapeo dinámico de campos ERP y detección automática desde JSON.
- [Figura 24](#): Panel de administración de empresas vista desde SUPER_ADMIN
- [Figura 25](#): Panel de administración de agentes con edición inline de instrucciones y temperatura, vista desde ADMIN.
- [Figura 26](#): Documentación Swagger en `/api/docs` con las 20 rutas documentadas.
- [Figura 27](#): Logs de producción en Render mostrando estructura JSON de Pino con campos indexables.
- [Figura 6](#): Commits relevantes y variados del proyecto.

Integraciones necesarias

AgenteX integra múltiples servicios externos que son fundamentales para su operación. A continuación se detalla cada integración, los datos que fluyen entre los servicios y las consideraciones de seguridad aplicadas.

1. API de DeepSeek — Modelo de lenguaje principal

DeepSeek es el proveedor del modelo de lenguaje grande (LLM) que potencia todos los agentes de la plataforma. La integración opera en dos niveles:

- **Desde el microservicio Python:** para la generación de respuestas conversacionales finales al usuario. Recibe el sistema prompt jerárquico construido por Node.js con los datos reales del ERP inyectados en el NIVEL 4, y devuelve la respuesta en lenguaje natural.
- **Desde el backend Node.js:** para los LLM auxiliares de extracción estructurada — `extraerIntencion` (temperatura 0, máx. 60 tokens), `extraerRango` (temperatura 0, máx. 60 tokens) y `extraerCliente` (temperatura 0, máx. 200 tokens). Estas llamadas son determinísticas y devuelven JSON estructurado.

Datos que entran: sistema prompt con contexto del negocio y datos del ERP, mensaje del usuario, historial de conversación reciente, temperatura configurada por empresa.

Datos que salen: respuesta en lenguaje natural o JSON estructurado según el tipo de llamada.

Seguridad: La `DEEPSEEK_API_KEY` se almacena exclusivamente en variables de entorno de Render. Nunca se expone en el repositorio ni en los logs del sistema. El rate limiting de 20 peticiones por minuto en `/api/chat` protege el consumo de la API ante usos abusivos.

2. Supabase — Base de datos PostgreSQL administrada

Supabase provee la base de datos relacional que persiste todos los datos de la plataforma. La integración se realiza mediante el cliente oficial `@supabase/supabase-js` desde el backend Node.js.

Datos que entran: operaciones CRUD sobre empresas, usuarios, agentes, sesiones y mensajes. Consultas con filtros por `company_id` para garantizar el aislamiento multi-tenant.

Datos que salen: registros de empresas con su configuración de ERP, usuarios con roles, agentes con instrucciones personalizadas, historial de mensajes con tokens consumidos.

Seguridad: La `SUPABASE_KEY` se almacena en variables de entorno. Todas las consultas incluyen filtro obligatorio por `company_id` extraído del JWT, garantizando que un usuario nunca acceda a datos de otro tenant. Las contraseñas de usuarios se almacenan hasheadas con `bcrypt` y nunca se devuelven en ninguna respuesta de la API.

3. Endpoints REST de los ERPs de cada empresa

Esta es la integración central del modelo de negocio de AgenteX. Cada empresa cliente configura las URLs de sus endpoints durante el onboarding con el SUPER_ADMIN. Los motores especializados consultan estos endpoints en tiempo real durante cada conversación.

Endpoints soportados por motor:

Motor	Endpoints configurables
erp_search	productos_url, stock_url (opcional)
analytics	productos_url (endpoint de registros analíticos)
ventas	productos_url, clientes_url, historial_url
logistica	ordenes_url

Datos que entran: petición GET al endpoint del ERP de la empresa.

Datos que salen: array JSON con los registros del ERP — productos, órdenes, clientes o registros analíticos según el motor.

Seguridad: Los fetches incluyen un timeout de 15 segundos para evitar que un endpoint lento bloquee el sistema. Si el endpoint requiere autenticación, el `erp_token` configurado en el `erp_mapping` se incluye automáticamente en el header `Authorization` de cada petición. El caché por empresa (`company_id:url`) garantiza que los datos de un tenant nunca sean servidos a otro.

4. Comunicación interna Node.js → Python

La comunicación entre el backend Node.js y el microservicio Python es una integración interna que opera mediante HTTP sobre la red de Render.

Datos que entran al microservicio Python: sistema prompt completo con los datos del ERP inyectados, mensaje del usuario, historial de conversación, temperatura configurada por empresa y `tenant_id`.

Datos que salen del microservicio Python: respuesta generada por el LLM, tokens consumidos (prompt y completion) y métricas de iteraciones de herramientas.

Seguridad: Toda petición desde Node.js al microservicio Python incluye el header `X-Internal-Secret` con un string aleatorio de 32 bytes configurado como variable de entorno en ambos servicios. El microservicio Python rechaza con 403 cualquier petición que no incluya este secret, impidiendo el acceso directo al motor de IA desde el exterior sin pasar por el backend principal.

Estrategia y Enfoque de Calidad

¿Qué significa "calidad" para AgenteX?

AgenteX es una plataforma SaaS multi-tenant que procesa consultas en lenguaje natural y las responde con datos reales del ERP de cada empresa cliente. En este contexto, "calidad" no se limita a que el software no falle: implica que cada respuesta del agente esté fundamentada exclusivamente en datos verificables, que los datos de una empresa nunca sean accesibles por otra, y que el sistema responda dentro de tiempos aceptables incluso cuando varios usuarios lo usan de forma simultánea.

Esta definición impone prioridades claras sobre los atributos de calidad a evaluar. La siguiente tabla los ordena de mayor a menor criticidad para el caso de negocio:

Atributo	Prioridad	Justificación para AgenteX
Seguridad	Crítica	Falla de aislamiento entre tenants = exposición de datos de todas las empresas. Vulnerabilidad JWT o XSS = acceso no autorizado al sistema.
Funcionalidad	Alta	Los motores de búsqueda, analítica, ventas y logística deben devolver resultados correctos y el protocolo anti-alucinación debe ser infranqueable.
Fiabilidad	Alta	Los errores externos (timeout del ERP, fallo de DeepSeek, token expirado) deben producir mensajes controlados, nunca fallos silenciosos ni excepciones no capturadas.
Rendimiento	Media	Los endpoints de soporte (health, login) deben responder en menos de 800ms p95 bajo carga. El chat no se incluye en carga por costo de IA.
Mantenibilidad	Media	Cobertura $\geq 90\%$ de ramas permite refactorizar con confianza. La suite automatizada detecta regresiones al instante.

Niveles y tipos de prueba

La estrategia aplica una pirámide de pruebas clásica, ejecutando los tipos de prueba de menor a mayor amplitud para detectar defectos lo antes posible en la cadena:

Orden	Tipo de prueba	Propósito en AgenteX
1°	Unitarias	Verificar funciones y componentes de forma aislada (motores, lib, componentes React, modelos Pydantic).
2°	Humo (Smoke)	Confirmar que el sistema arranca y expone sus endpoints críticos tras cada build, antes de correr suites más costosas.
3°	Integración	Validar que las capas del sistema se comunican correctamente (middleware ↔ controlador ↔ servicio, componente ↔ apiFetch ↔ fetch, endpoint ↔ loop de tools).
4°	E2E / Funcional	Recorrer flujos completos de usuario (login→chat→historial en browser real, multi-turno en el microservicio Python).
5°	Regresión	Fijar bugs históricos reales encontrados en el historial de commits para que no reaparezcan en refactorizaciones futuras.
6°	Rendimiento / Carga	Medir comportamiento bajo usuarios concurrentes (k6 con múltiples escenarios en paralelo).
7°	Seguridad	Verificar aislamiento multi-tenant, resistencia a manipulación de JWT, XSS, escalación de privilegios y ausencia de vulnerabilidades conocidas (npm audit).

Enfoque, datos de prueba y ambientes

Enfoque: 100% automatizado. No se contemplaron pruebas manuales en el plan formal. Cada capa tiene su comando de ejecución propio (npm test, npx vitest run, pytest, npx playwright test, k6 run) y puede correrse de forma independiente o completa.

Datos de prueba: ninguna prueba de la suite principal usa la base de datos Supabase real, la API de DeepSeek real, ni ERPs de clientes reales. Todos los servicios externos se sustituyen por mocks y fixtures estáticos definidos inline en cada archivo de test. Este enfoque garantiza que la suite corra sin costo de API, sin depender de disponibilidad de terceros y reproduciendo a voluntad casos de borde (error de Supabase, fallo de DeepSeek, timeout del ERP, cross-tenant) que serían frágiles o imposibles de forzar con datos reales.

Ambientes: local (Windows + PowerShell para el frontend, entorno Linux para el backend y el microservicio Python). La suite k6 del microservicio Python usa un stub server local que replica DeepSeek y el ERP para evitar dependencias de red y costo de IA durante la corrida de carga.

Definición de "Hecho" de calidad

Una funcionalidad o corrección se considera validada cuando cumple todos los criterios siguientes:

- El 100% de los tests asociados pasa sin omisiones ni tests marcados como skip.
- La cobertura de ramas (branch coverage) del módulo afectado es $\geq 90\%$.
- No se introducen vulnerabilidades high/critical (verificado con npm audit en el frontend).
- Si la corrección resuelve un bug histórico, existe al menos un test de regresión que fija el escenario y pasaría a fallar si el bug reapareciera.
- El código pasa el linter sin errores (eslint en frontend y backend, ruff/pyflakes en Python).

Plan de Pruebas

Esta sección resume el plan de pruebas aplicado al proyecto AgenteX en la etapa EA3. El documento completo se incluye en los Anexos.

Alcance

Las pruebas cubren los tres servicios que conforman la plataforma:

- Backend Node.js/Express: controladores, servicios de dominio (erp-search, analytics, ventas, logística), middlewares de autenticación y las 20 rutas REST documentadas.
- Frontend React/Vite: todos los componentes de la interfaz de usuario, la librería apiFetch, las rutas SPA y los flujos de navegación en navegador real.
- Microservicio Python/FastAPI: el endpoint /api/ia/process, el loop de orquestación de tools de DeepSeek, la lógica de consultar_inventario_erp y los modelos Pydantic de entrada/salida.

Quedan explícitamente fuera del alcance de la suite automatizada:

- La base de datos Supabase real (sustituida por mocks en todos los tests).
- La API de DeepSeek real (sustituida por respuestas predefinidas).
- Los ERPs de clientes reales (sustituidos por fixtures estáticos).
- El endpoint /api/chat/message en las pruebas de carga (evitar gasto real de tokens de IA).

La exclusión de servicios externos reales está justificada por tres razones: (1) reproducibilidad — los tests pueden correrse offline sin depender de disponibilidad de terceros; (2) control — los casos de borde (fallo de Supabase, timeout del ERP, respuesta malformada de DeepSeek) se pueden reproducir a voluntad; (3) costo — las llamadas reales a DeepSeek tienen un costo por token que haría la suite insostenible.

Criterios de aceptación y rechazo

Criterio	Valor umbral
Branch coverage (todos los servicios)	≥ 90%
Statement coverage	≥ 90%
Tasa de éxito de tests	100% (0 tests fallidos en la suite principal)
Vulnerabilidades npm audit	0 high/critical
p95 en k6 (endpoints de soporte)	< 800 ms
Tasa de error en k6	< 1%

Criterios de suspensión y reanudación

Suspensión: si más del 20% de los tests de cualquier capa fallan simultáneamente (indicador de error de ambiente, no de lógica individual), o si un error de compilación/transpilación impide levantar la suite. También se suspende si se detecta una fuga de datos real entre tenants en la suite de seguridad.

Reanudación: cuando la causa raíz está corregida y el 100% de los tests de la capa afectada vuelve a pasar. Si la suspensión fue por error de ambiente, se verifica la configuración antes de reiniciar.

Recursos, roles y cronograma (RACI)

Al tratarse de un proyecto académico unipersonal, todos los roles recaen sobre Mauricio Lillo:

Actividad	Responsable (R)	Aprobador (A)	Consultado (C)	Informado (I)
Diseño de casos de prueba	M. Lillo	M. Lillo	—	Docente
Ejecución de la suite	M. Lillo	M. Lillo	—	Docente
Documentación de defectos	M. Lillo	M. Lillo	—	Docente
Corrección de defectos	M. Lillo	M. Lillo	—	—
Validación del informe EA3	M. Lillo	Docente	—	—

Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Mocks que no reflejan el comportamiento real de APIs externas	Tests pasan pero fallo en producción	Validación manual del contrato en producción tras cada ciclo de desarrollo
Cold start de Render (plan gratuito)	Métricas de k6 infladas en primera petición	Warmup manual (1 request) antes de correr la suite k6 formal
Dependencia de la API de DeepSeek en producción	Plataforma inoperativa si DeepSeek cae	Mitigado por separación de microservicio (facilita cambio de proveedor); fuera del alcance de testing actual
Limitación de recursos para CI/CD formal	La suite no corre automáticamente en cada push	Ejecución manual obligatoria antes de cada merge a main, documentada en el Definition of Done

Tipos de Pruebas Aplicadas

A continuación se describe cada tipo de prueba aplicado al proyecto, indicando su propósito concreto, qué se probó, la herramienta utilizada y por qué aporta valor en el contexto de AgenteX.

Pruebas unitarias

Propósito: verificar que cada función, método o componente funciona correctamente de forma aislada, independientemente del resto del sistema.

Qué se probó: en el backend, toda la lógica de dominio de los cuatro motores (tokenización, fuzzy match, filtros de stock, rangos temporales, identificación semántica de clientes, normalización de órdenes), los controladores REST y el middleware de autenticación. En el frontend, cada componente React por separado (LoginScreen, Dashboard, ChatInterface, Sidebar, paneles de administración, etc.) y las funciones utilitarias (apiFetch, utils). En el microservicio Python, la función consultar_inventario_erp con sus 25 rutas de cobertura posibles y el modelo Pydantic ChatRequest.

Herramienta: Jest 29 (backend), Vitest 3 + @testing-library/react 16 (frontend), pytest 8 + pytest-asyncio (Python).

Pertinencia: los motores de AgenteX implementan lógica de negocio compleja (tokenización, plural, stopwords, AND/OR entre tokens numéricos y de texto, detección de categorías) que debe funcionar correctamente antes de integrarse con la capa de red y el LLM. Las pruebas unitarias son la primera línea de defensa y las más baratas de ejecutar.

22.2 Pruebas de humo (Smoke)

Propósito: verificar rápidamente que el sistema arranca sin errores y que sus funciones más críticas responden, antes de ejecutar suites más costosas.

Qué se probó: backend: health check GET /health, que las rutas base responden 200 y que el login con credenciales válidas retorna un JWT. Frontend: que la aplicación React renderiza sin crash, que el formulario de login muestra sus campos y que la navegación entre secciones no produce errores. Microservicio Python: que /openapi.json responde 200, que /docs es accesible, que el endpoint /api/ia/process recibe requests y que los códigos de error (422, 404, 405) se devuelven correctamente.

Herramienta: Jest (backend), Vitest + @testing-library/react (frontend), pytest (Python).

Pertinencia: en una arquitectura de tres servicios desplegados en plataformas distintas (Render/Vercel), los errores de configuración de entorno (variable de entorno faltante, URL mal configurada, dependencia no instalada) son frecuentes. Los smoke tests detectan estos fallos en segundos, sin necesidad de correr la suite completa.

Pruebas de integración

Propósito: validar que distintos módulos funcionan correctamente cuando trabajan juntos, ejercitando la cadena real de comunicación sin depender de servicios externos.

Qué se probó: backend: la aplicación Express completa (app.js) mediante Supertest, ejercitando CORS, rate limiting, logs estructurados con Pino, rutas, controladores y

validaciones, con Supabase y DeepSeek mockeados. Frontend: la cadena real componente → apiFetch → global.fetch mockeado, verificando que el estado del componente refleja correctamente las respuestas de la API. Microservicio Python: el endpoint POST /api/ia/process mediante httpx.ASGITransport (sin levantar un puerto real), con DeepSeek y el ERP interceptados por respX.

Herramienta: Supertest 7 (backend), Vitest + global.fetch mock (frontend), httpx.ASGITransport + respX (Python).

Pertinencia: AgenteX tiene múltiples capas de middleware (autenticación JWT, rate limit, validación de ownership, logs). Los tests de integración verifican que estas capas coexisten correctamente y que los errores de cada capa se propagan con el código HTTP y el mensaje esperados.

Pruebas funcionales y de extremo a extremo (E2E)

Propósito: recorrer flujos completos tal como los ejecutaría un usuario real, validando el comportamiento del sistema de punta a punta.

Qué se probó: backend: flujo completo login → envío de mensaje → persistencia en historial (4 pasos secuenciales en un único test Jest). Frontend: flujos de autenticación (login exitoso → redirección, credenciales inválidas, ruta privada sin token → /login, logout limpia sesión), navegación con Sidebar dinámico, consulta de chat con respuesta del agente, y 401 forzado → logout automático — todo ejecutado en un navegador Chromium real con Playwright, con la red interceptada por page.route. Microservicio Python: dos requests HTTP secuenciales reconstruyendo el history entre turnos, replicando exactamente cómo el backend Node enviaría el contexto acumulado al motor de IA.

Herramienta: Jest con pasos secuenciales (backend), Playwright 1.x con Chromium (frontend), httpx.ASGITransport multi-request (Python).

Pertinencia: los flujos E2E validan comportamientos que las pruebas unitarias e de integración no pueden capturar: la redirección automática al expirar el token en cualquier componente, el mantenimiento del contexto conversacional entre turnos, y la "trampa de fricción" del motor Python cuando hay más de 20 resultados.

Pruebas de regresión

Propósito: garantizar que defectos ya corregidos no reaparezcan en modificaciones futuras del código.

Qué se probó: los candidatos se seleccionaron revisando el historial de git (git log --all --oneline | grep -i fix) en los tres repositorios, descartando aquellos ya cubiertos por tests de integración existentes. Los bugs fijados como tests de regresión son:

- BUG-BE-01: bypass del filtro stock_critico cuando el término de búsqueda es "ALL" en erp-search.service.js.
- BUG-BE-02: validación de roles invertida en registerB2B (auth.controller.js), que permitía a un ADMIN registrar un SUPER_ADMIN.

- BUG-FE-01: mensaje de error duplicado en LoginScreen.jsx (el nodo <div> del mensaje se renderizaba dos veces por un commit que introdujo el div sin eliminar el anterior).
- BUG-FE-02: extraerClaves() en ERPMappingSection no manejaba correctamente el caso en que el JSON pegado era un array vacío.
- BUG-PY-01: tokens numéricos evaluados con AND hacían que búsquedas de medidas como "29 2.10" no encontraran "29x2.125" (fix: tokens numéricos en OR).
- BUG-PY-02: match por ID/SKU comparaba sin normalizar, fallando con mayúsculas o acentos.

Herramienta: Jest (backend, 5 tests en 2 archivos), Vitest (frontend, 6 tests en 2 archivos), pytest (Python, 4 tests en 1 archivo).

Pertinencia: todos los bugs listados surgieron durante el desarrollo real y fueron corregidos antes de la entrega EA2. Los tests de regresión fijan estos contratos para proteger la base de código en la etapa de testing y en refactorizaciones futuras.

Pruebas de rendimiento y carga

Propósito: medir el comportamiento del sistema bajo demanda concurrente y verificar que los thresholds de rendimiento definidos se cumplen.

Qué se probó: backend Node: dos escenarios concurrentes (health_check sobre GET /health y login sobre POST /api/auth/login), con una rampa de 0→10 VUs en 20 segundos, sosteniendo 10 VUs durante 40 segundos. Microservicio Python: cuatro escenarios (health, process_direct, process_tool_call, process_error) contra un stub server local que replica DeepSeek y el ERP, eliminando dependencias de red.

Herramienta: k6 (Grafana k6) con options.scenarios para escenarios paralelos.

Pertinencia: AgenteX opera en el plan gratuito de Render, que introduce un cold start de hasta 50 segundos tras un período de inactividad. Verificar que los endpoints de soporte (no el chat, que depende de una API de pago externa) responden dentro de umbrales aceptables bajo carga es crítico para garantizar una experiencia de usuario mínimamente razonable.

Pruebas de seguridad

Propósito: verificar que los mecanismos de autenticación, control de acceso, validación de entradas y manejo de secretos funcionan correctamente, con énfasis en el aislamiento multi-tenant.

Qué se probó:

- Backend: manipulación de JWT (firma inválida, token sin secret configurado, token de otro tenant), intento de acceso cross-tenant a sesiones y mensajes ajenos, y escalación de privilegios (un USER intentando rutas de ADMIN/SUPER_ADMIN).
- Frontend: resistencia a XSS a través de react-markdown (scripts embebidos y atributos onerror no se ejecutan), validación de entrada en campos de formulario,

tampering del token en localStorage antes de una request, y comportamiento al recibir un 401 (logout automático desde cualquier componente).

- Microservicio Python: rechazo de requests sin header X-Internal-Secret y con secret incorrecto, validación de campos requeridos y tipos de entrada.
- Análisis estático de dependencias: npm audit automatizado (scripts/check-audit.js) que falla con exit 1 si hay vulnerabilidades high/critical no documentadas como riesgo aceptado.

Herramienta: Jest (backend, 13 tests), Vitest (frontend, 16 tests + script npm audit), pytest (Python, 8 tests).

Pertinencia: en un SaaS multi-tenant, la seguridad es el atributo de calidad de mayor prioridad. Una falla de aislamiento entre tenants comprometería los datos de todas las empresas registradas en la plataforma simultáneamente. Las pruebas de seguridad formalizan estos contratos y detectarían cualquier regresión en los mecanismos de aislamiento.

Durante la ejecución de las pruebas de seguridad del frontend se identificó un hallazgo real de seguridad: 8 componentes realizan peticiones HTTP directamente con fetch sin pasar por apiFetch, lo que significa que no manejan el código 401 (token expirado), produciendo fallos silenciosos en lugar de redirigir al usuario al login. Este hallazgo quedó documentado como deuda técnica.

Herramientas y Software Utilizados

Las herramientas seleccionadas para el proceso de pruebas se eligieron por su compatibilidad con el stack tecnológico de AgenteX y su adopción mayoritaria en la industria para cada tipo de prueba. La siguiente tabla las resume:

Herramienta	Tipo de uso	Versión	Justificación
Jest	Unit/Integración/Smoke/E2E/Regresión/Seguridad (Backend)	29.x	Framework de testing estándar para proyectos Node.js/Express. Compatible de forma nativa con el stack del backend. Incluye mocking de módulos, cobertura vía Istanbul y runner con modo <code>--coverage</code> integrado.
Supertest	Integración HTTP (Backend)	7.x	Permite levantar la app Express en memoria y enviar requests HTTP sin abrir un puerto real, aislando los tests de integración del sistema operativo y habilitando ejecución paralela.
Vitest	Unit/Integración/Smoke/Regresión/Seguridad (Frontend)	3.x	Framework de testing nativo para proyectos Vite. Comparte la configuración de <code>vite.config.js</code> , soporta JSX automático con esbuild, y tiene API compatible con Jest. Incluye cobertura vía <code>@vitest/coverage-v8</code> (motor V8).
@testing-library/react	Renderizado de componentes React (Frontend)	16.x	Librería de renderizado de componentes orientada al usuario (consultas por rol, texto y label, no por atributos de implementación). Favorece tests resistentes a refactorizaciones internas.

Playwright	E2E en navegador real (Frontend)	1.x	Automatización de browser con Chromium real. Se eligió sobre Cypress por su auto-wait nativo, soporte de interceptación de red sin plugins (page.route) y mejor integración con proyectos Vite.
pytest + pytest-asyncio	Todos los tipos de prueba (Python)	8.x / 0.24	Framework estándar para testing en Python. pytest-asyncio habilita pruebas de corrutinas async/await sin boilerplate adicional, esencial para FastAPI con httpx.ASGITransport.
pytest-cov	Cobertura de código (Python)	5.x	Plugin de coverage para pytest. Genera reporte HTML y falla la suite si la cobertura cae por debajo del umbral configurado (--cov-fail-under=90 en pytest.ini).
respx	Mocking HTTP (Python)	0.21	Intercepta requests de httpx a nivel de transporte, sin mockear módulos Python. Permite probar la cadena real endpoint → loop de tools → httpx → respuesta sin conexión de red real.
httpx + ASGITransport	Cliente HTTP de test (Python)	0.27	Permite enviar requests a la aplicación FastAPI en memoria a través de ASGITransport, sin levantar un servidor real. Combina el comportamiento real de la app con el aislamiento de red de un mock.

k6 (Grafana k6)	Pruebas de carga y rendimiento	0.55.x	Herramienta de carga con scripting en JavaScript, soporte de escenarios múltiples con options.scenarios y thresholds declarativos. Permite simular múltiples tipos de carga en paralelo y exportar resultados en JSON.
Istanbul / V8 Coverage	Reporte de cobertura (Backend y Frontend)	Incluido en Jest / @vitest/coverage-v8	Genera reportes HTML con marcadores E/I por rama (branch). El reporte HTML fue la herramienta principal para identificar ramas sin cubrir y priorizar nuevos casos de prueba.
npm audit (scripts/check-audit.js)	Auditoría de dependencias (Frontend)	npm 10.x	Script automatizado que corre npm audit --json y falla con exit 1 si hay vulnerabilidades high/critical no documentadas como riesgo aceptado, integrando la auditoría de seguridad en el flujo de npm test.
GitHub (Issues + commits)	Gestión de defectos y trazabilidad	—	Los bugs encontrados durante el testing se documentaron como commits con prefijo fix: y se vincularon a los tests de regresión correspondientes mediante comentarios en el código (# FIX, // FIX).

Herramientas de calidad de código

Además de las herramientas de prueba, se utilizaron las siguientes herramientas de calidad estática de código:

- ESLint: análisis estático de JavaScript/JSX en backend y frontend. Configurado con las reglas estándar de cada proyecto y ejecutado como parte del proceso de revisión antes de cada merge a main.
- Ruff (Python): linter y formateador para el código Python del microservicio de IA. Detecta errores de estilo, variables no usadas y patrones problemáticos.
- Reportes HTML de cobertura (Istanbul/V8): la cobertura no se revisó en formato de texto plano, sino en el reporte HTML interactivo que muestra exactamente qué ramas (E=Else no cubierta, I=If no cubierto) siguen sin ejercitar, permitiendo una priorización precisa de nuevos casos de prueba.

Ejecución de Pruebas y Resultados

Esta sección presenta el resumen de ejecución de la suite completa de pruebas de AgenteX, los indicadores obtenidos y las referencias a las evidencias correspondientes. La suite cubre los tres servicios de la plataforma: backend Node.js/Express, frontend React/Vite y microservicio Python/FastAPI.

Resumen de ejecución por servicio y tipo

Servicio / Tipo	Herramienta	Total casos	Exitosos	Fallidos	Cobertura
Backend — Unitarias	Jest	283	283	0	91.05% branch / 99.7% stmt
Backend — Integración	Jest + Supertest	49	49	0	—
Backend — Smoke	Jest	15	15	0	—
Backend — E2E	Jest	4	4	0	—
Backend — Regresión	Jest	5	5	0	—
Backend — Seguridad	Jest	13	13	0	—
Backend — Carga (k6)	k6	2 escenarios	—	—	p95=230ms  · 0% error 
TOTAL BACKEND		369	369	0	91.05% branch
Frontend — Unitarias	Vitest + TL/React	148	148	0	91.8% branch / 99.77% stmt
Frontend — Smoke	Vitest	15	15	0	—
Frontend — Integración	Vitest	37	37	0	—
Frontend — E2E (Playwright)	Playwright	9	9	0	—
Frontend — Regresión	Vitest	6	6	0	—
Frontend — Seguridad	Vitest + npm audit	16 + audit	16 + 0 vuln H/C	0	—

Frontend — Carga (k6)	k6	4 escenarios	—	—	99.99% éxito · 200 VUs  · p95 
TOTAL FRONTEND		231	231	0	91.8% branch
Python — Unitarias	pytest	31	31	0	100%
Python — Smoke	pytest	7	7	0	—
Python — Integración	pytest + httpx	9	9	0	—
Python — E2E	pytest + httpx	2	2	0	—
Python — Regresión	pytest	4	4	0	—
Python — Seguridad	pytest	8	8	0	—
Python — Carga (k6)	k6 + stub server	4 escenarios	—	—	p95=57.75s · 14.94% error 
TOTAL PYTHON		61	61	0	100%
GRAN TOTAL		661	661	0	≥90% en 3 servicios

Indicadores y métricas por servicio

Indicador	Backend Node	Frontend React	Python FastAPI
Tests totales (sin k6)	369	231	61
% éxito (tests pasando)	100%	100%	100%
Branch coverage	91.05%	91.8%	100%
Statement coverage	99.7%	99.77%	100%
Function coverage	100%	92.64%	100%
Vulnerabilidades high/critical	N/A	0 (1 low aceptada)	0

k6 — max VUs sin abortar	20 VUs	200 VUs	~50 VUs (aborta en estrés)
k6 — p95 global	230.07 ms	910 ms (umbral 300 ms)	57.75 s (umbral 300 ms)
k6 — tasa de error global	0.00%	0.003%	14.94%
Suites / archivos de test	37 archivos	~20 archivos	7 archivos

Resultados detallados de pruebas de carga (k6)

Las pruebas de carga se ejecutaron de forma independiente a la suite principal (fuera del runner de pytest/vitest/jest) y con stubs locales que simulan los servicios externos, evitando costos reales de IA. A continuación se detallan los resultados de cada servicio:

Frontend React — corrida real (8 min, 2026-06-20)

Script: src/test/load/frontend-api.k6.js. Diseño: 4 escenarios secuenciales. Cada iteración simula el flujo completo Auth → Dashboard → Historial → Settings contra el backend local con NODE_ENV=development (rate-limiter desactivado).

Escenario	VUs	Duración	Error rate	p95 latencia
escenario_10	10 constantes	1 min	0%	< umbral
escenario_50	50 constantes	1 min	0%	Degradación inicial
escenario_100	100 constantes	1 min	0%	Supera umbrales
escenario_estres	0 → 200 VUs	Hasta 4 min	0.00% (no abortó)	—

Métrica global	Auth	Dashboard / Historial	Global
Flujos completos	9.771	—	97.720 requests totales
Tasa de éxito	99.99%	99.99%	3 fallos de 97.720
p95 latencia (umbral <300 ms)	1.11 s	1.05 s / 857 ms	910 ms
Max VUs alcanzados sin abortar	200	200	Escenario de estrés completo

Conclusión: estabilidad  — el backend no colapsó ni generó cascada de errores con 200 usuarios concurrentes. Rendimiento  — el SLA de p95 < 300 ms se rompe a partir de aproximadamente 50-100 VUs concurrentes, atribuible a la latencia de Supabase en plan gratuito bajo carga concurrente desde una sola IP.

Microservicio Python/FastAPI — corrida real (~2 min 15 s, 2026-06-20)

Script: tests/load/motor-ia.k6.js. Diseño: mismos 4 escenarios que el frontend. Cada iteración fuerza el camino más costoso: 2 round-trips al stub de DeepSeek + 1 al stub del ERP (sin red real, sin costo de IA). Se levantaron 3 procesos locales: stub server (puerto 9000), microservicio uvicorn (puerto 8001) y k6.

Escenario	VUs	Estado	Error rate	Resultado
escenario_10	10 constantes	Completó	Bajo	Latencia elevada
escenario_50	50 constantes	Completó	Moderado	Latencia elevada
escenario_100	100 constantes	Completó	Alto	Degradación severa
escenario_estres	0 → 200 VUs	ABORTÓ (~161 VUs)	100% 	threshold superado

Métrica global	Valor real
p95 de latencia (umbral <300 ms)	57.75 s 
Tasa de error global (umbral <5%)	14.94% 
Error rate en escenario de estrés (umbral auto-abort >5%)	100% → abortó 
Checks exitosos (iteraciones que completaron)	100% (148/148) 
Iteraciones completas / interrumpidas	74 completas · 284 interrumpidas por el abort

Root cause confirmada: main.py y tools.py crean un `httpx.AsyncClient()` nuevo por cada llamada saliente (a DeepSeek y al ERP), sin pool de conexiones. Bajo alta concurrencia esto genera apertura/cierre masivo de sockets efímeros. En Windows, el Proactor event loop de asyncio dispara además una tormenta de `ConnectionResetError` que infla las latencias hasta decenas de segundos. Es un hallazgo real y reproducible, no ruido del entorno.

Mejora documentada (fuera del alcance de testing): reusar un único `httpx.AsyncClient` a nivel de app vía `lifespan` de FastAPI. Este cambio no afecta ninguna lógica de negocio y debería reducir drásticamente la latencia bajo concurrencia.

Backend Node.js/Express — corrida real (2026-06-21)

Script: loadtest/loadtest.js. Diseño: 2 escenarios paralelos. Cada escenario corre una rampa 0→10 VUs (20s) → sostiene 10 VUs (40s) → baja a 0 (10s), sumando hasta 20 VUs concurrentes en el tramo sostenido. El backend se levantó con NODE_ENV=development para desactivar el generalLimiter (express-rate-limit, 100 req/min por IP).

Escenario	Endpoint	VUs máx	Error rate	p95 latencia
health_check	GET /health	10	0.00%	~0 ms (resp. local)
login	POST /api/auth/login	10	0.00%	230.07 ms

Métrica global	Valor real
Thresholds	Ambos cumplidos
p(95) duración (umbral <800 ms)	230.07 ms
Tasa de error (umbral <1%)	0.00% (0 de 993 requests)
Checks exitosos	100% — 1.986/1.986 (status 200 + respuesta correcta + token presente)
Requests totales	993 · 14.15 req/s
Iteraciones completadas	993 · 14.15 iter/s
p(90) / p(95) duración	226.49 ms / 230.07 ms
Duración avg / med / max	102.39 ms / 1.43 ms / 1.06 s
VUs máximos simultáneos	20 (10 health_check + 10 login en paralelo)

Conclusión: estabilidad y rendimiento — el backend Node.js respondió dentro del SLA definido (p95 < 800 ms) con 0% de errores bajo 20 VUs concurrentes. La mediana de 1.43 ms refleja el health check (respuesta local inmediata); el p95 de 230 ms corresponde al escenario de login, que involucra bcrypt, consulta a Supabase y firma de JWT. Ambos thresholds cumplidos.

Evidencias de ejecución

Las evidencias de ejecución de cada capa de pruebas se presentan como figuras en los Anexos del informe:

Figura 28: muestra la salida completa de consola del comando `npm test` sobre el backend Node.js, con los 369 tests pasando y el resumen de cobertura.

Figura 29: presenta el reporte HTML de Istanbul con el detalle de branch coverage por archivo, evidenciando el 91.05% global alcanzado.

Figura 30: corresponde a la salida de `npx vitest run` sobre el frontend React, cubriendo las capas unitaria, humo, integración, regresión y seguridad.

Figura 31: muestra el reporte HTML de cobertura V8 del frontend con el 91.8% de branch coverage.

Figura 32: registra la ejecución de `npx playwright test` con los 9 tests E2E pasando en Chromium.

Figura 33: presenta la salida de `pytest --cov` sobre el microservicio Python con los 61 tests pasando y 100% de cobertura.

Figura 34: muestra el resultado de la corrida k6 del backend Node.js con ambos thresholds en verde, p95 de 230.07ms y 0% de errores sobre 993 requests.

Figura 35: corresponde al resultado k6 del frontend React con los 4 escenarios, incluyendo el escenario de estrés completado a 200 VUs.

Figura 36: muestra el resultado k6 del microservicio Python con el abort del escenario de estrés, evidenciando el hallazgo de performance PERF-PY-01.

Figura 37: presenta la salida de `npm run security:audit` confirmando 0 vulnerabilidades de severidad alta o crítica.

Figura 38: presenta la salida de las 3 integraciones, mostrando resultados de backend, frontend y microservicio.

Defectos encontrados durante el proceso de testing

La siguiente tabla resume los defectos identificados durante el desarrollo de la suite de pruebas. Los marcados como "Deuda técnica" están documentados pero pendientes de corrección:

Bug ID	Componente	Descripción	Corrección	Capa
BUG-B E-01	erp-search.service.js	El filtro stock_critico se omitía cuando el término de búsqueda era "ALL", retornando todos los productos sin filtrar.	Aplicar el filtro de stock independientemente del valor del término.	Regresión
BUG-B E-02	auth.controller.js	La validación de rol en registerB2B estaba invertida: un ADMIN podía registrar un SUPER_ADMIN.	Invertir la condición de validación de rol.	Regresión
BUG-F E-01	LoginScreen.jsx	El nodo <div> del mensaje de error estaba duplicado en el JSX, mostrando el mensaje dos veces.	Eliminar el <div> duplicado.	Regresión
BUG-F E-02	ERPMappingSection .jsx	extraerClaves() lanzaba excepción al recibir un array vacío como JSON.	Agregar guarda para array vacío con mensaje de error controlado.	Regresión
BUG-F E-03	Register.jsx	Labels del formulario de registro sin atributo htmlFor, rompiendo accesibilidad y tests ARIA.	Agregar htmlFor a todos los labels del formulario.	Unit
BUG-F E-04	ChatInterface.jsx	motion.form con props de animación rompía el árbol de accesibilidad en RTL, impidiendo getByRole("textbox").	Mockear motion/react completo para reemplazar motion.form por <form>.	Unit
BUG-F E-05	window.location (jsdom)	window.location sin origin contaminaba el jsdom compartido entre tests, rompiendo React Router en tests subsecuentes.	Restaurar window.location completo (con origin) en afterEach del setup global.	Unit/Smoke

BUG-F E-06	8 componentes React	Hallazgo de seguridad: uso de fetch directo sin pasar por apiFetch, sin manejo de 401. Los tokens expirados producen fallos silenciosos en lugar de redirigir al login.	Deuda técnica documentada. Requiere migrar los 8 componentes a apiFetch.	Seguridad
BUG-P Y-01	tools.py	Tokens numéricos evaluados con AND: búsqueda "29 2.10" no encontraba "29x2.125".	Tokens numéricos en OR; tokens de texto siguen en AND.	Regresión
BUG-P Y-02	tools.py	Match por ID/SKU comparaba sin normalizar: búsqueda con mayúsculas no encontraba SKU en minúsculas.	Usar termino_limpio (ya normalizado) en ambos lados de la comparación.	Regresión
PERF- FE-01	Backend Node + Supabase	Hallazgo de carga: p95 supera 800 ms a partir de ~50-100 VUs concurrentes. El servidor no colapsa (200 VUs sin abortar) pero el SLA de latencia no se cumple.	Deuda técnica. Causa probable: Supabase plan gratuito bajo conurrencia alta. Evaluar en plan de pago o con connection pooling.	Carga k6
PERF- PY-01	main.py + tools.py	Hallazgo de carga: httpx.AsyncClient() nuevo por cada llamada saliente (sin pool). Bajo conurrencia genera tormenta de sockets: p95=57.75s, 14.94% error, abort en estrés.	Deuda técnica documentada. Fix: reusar un AsyncClient compartido vía lifespan de FastAPI.	Carga k6

Conclusión

El problema que originó el proyecto

Este proyecto nació de una observación concreta: los dueños de negocio y ejecutivos de PyMEs invierten horas semanales cruzando datos entre sistemas ERP, hojas de cálculo y reportes en PDF para responder preguntas operativas simples. La toma de decisiones se vuelve reactiva, dependiente de analistas intermediarios, y lenta frente a mercados que cambian en horas. El capital de datos de la empresa existe, pero el costo en tiempo para extraerlo y analizarlo supera el beneficio percibido.

La hipótesis del proyecto fue que este problema no requería más dashboards ni más BI tradicional — requería eliminar la fricción entre la pregunta del directivo y el dato del ERP, reemplazando la interfaz técnica por lenguaje natural y la intermediación humana por agentes de IA especializados anclados a datos reales.

El recorrido de las tres entregas

EA1 — Diagnóstico y diseño: se formalizó el problema mediante un árbol de causas y efectos, se definió el mercado objetivo (PyMEs y empresas medianas bajo el modelo B2B multi-tenant) y se tomaron las decisiones arquitecturales fundamentales: microservicios en tres capas independientes (React/Vite en Vercel, Node.js/Express en Render, Python/FastAPI en Render), base de datos relacional en Supabase con aislamiento por `company_id`, y cuatro motores especializados por dominio operativo (`erp_search`, `analytics`, `ventas`, `logística`). Se estableció también el protocolo anti-alucinación jerárquico de cinco niveles como garantía de que ninguna respuesta del agente pudiera ser inventada.

EA2 — Construcción del producto: se implementó la plataforma completa en producción. El backend Node.js orquesta los cuatro motores, autentica con JWT, aplica aislamiento multi-tenant en cada consulta y se comunica con el microservicio Python mediante un secret compartido. El frontend React cubre el flujo completo de usuario: login, chat conversacional con los agentes, historial de sesiones y panel de administración diferenciado por roles (`SUPER_ADMIN`, `ADMIN`, `USER`). El microservicio Python actúa como intermediario hacia la API de DeepSeek, implementando el loop de tool calls con límite de iteraciones para evitar bucles infinitos. Al cierre de EA2, la plataforma estaba desplegada y operativa con URL pública.

EA3 — Verificación de calidad: se construyó una suite de 661 tests automatizados distribuidos en siete capas (unitarias, humo, integración, E2E, regresión, carga y seguridad) sobre los tres repositorios. El proceso no solo verificó que el producto funciona — descubrió y documentó 12 defectos reales (10 corregidos, 2 como deuda técnica) y 2 hallazgos de performance que no habrían sido visibles sin las pruebas de carga. Los tres servicios superan el umbral de 90% de cobertura de ramas y el backend completó su corrida k6 formal con 0% de errores y p95 de 230 ms bajo 20 VUs concurrentes.

Evidencia de que el problema fue resuelto

La validación del problema original se puede trazar directamente contra las tres causas identificadas en EA1:

Causa original (EA1)	Solución verificada (EA3)
Silos de datos desconectados: la información del ERP no era accesible sin navegación técnica.	Los cuatro motores se conectan en tiempo real a los endpoints REST de cada empresa y devuelven datos reales en segundos. Verificado por 49 tests de integración contra la cadena completa (middleware → controlador → servicio → ERP mockeado) y por 9 tests E2E en navegador real con Playwright.
Interfaz rígida: los dashboards exigían conocimiento técnico para filtrar y cruzar datos.	La interfaz conversacional en lenguaje natural fue verificada por los tests E2E del frontend (flujo login → selección de agente → consulta → respuesta) y por los tests E2E del microservicio Python (flujo multi-turno con reconstrucción de contexto entre turnos).
IA pública sin contexto: los LLMs genéricos alucinan cuando no tienen datos reales de la empresa.	El protocolo anti-alucinación de cinco niveles fue verificado por los tests de seguridad del backend: el agente devuelve "CERO RESULTADOS" ante consultas fuera del alcance de los datos disponibles, nunca respuestas inventadas. Los tests de regresión fijan dos bugs históricos críticos (BUG-BE-01 y BUG-BE-02) que comprometían la integridad de este protocolo.

El hallazgo más significativo de la etapa EA3 fue que el proceso de verificación no solo confirmó lo que funcionaba — también expuso lo que no: el microservicio Python no tiene pool de conexiones HTTP (PERF-PY-01), lo que provoca un colapso bajo concurrencia alta (p95 de 57.75s, 14.94% de errores en el escenario de estrés). Este hallazgo, invisible en el uso normal de la plataforma, quedó documentado con su root cause y su plan de corrección, demostrando que la suite de pruebas cumple su función más importante: encontrar lo que el desarrollo no ve.

Anexos

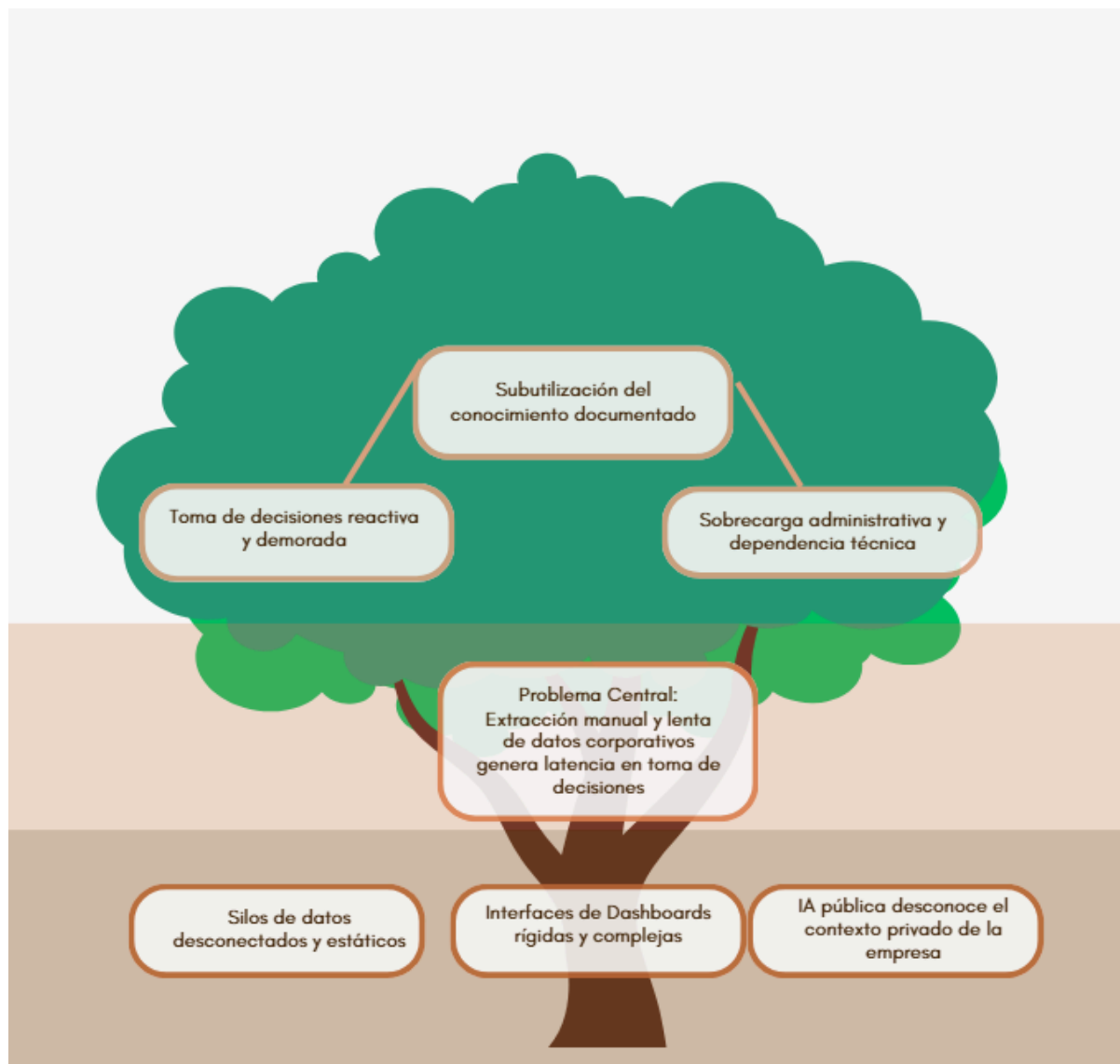


Figura 1: Árbol de problema/oportunidad. Fuente: Elaboración propia.

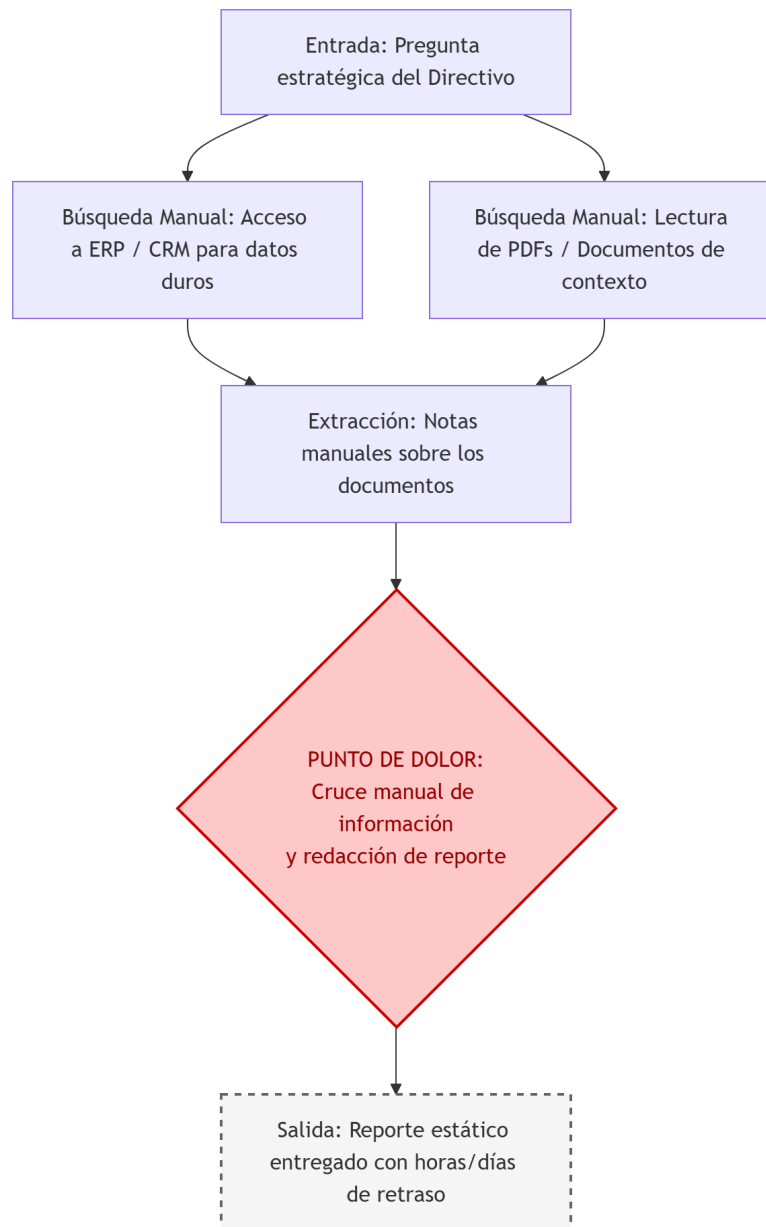


Figura 2: Diagrama de flujo del proceso actual (AS-IS). Fuente: Elaboración propia.

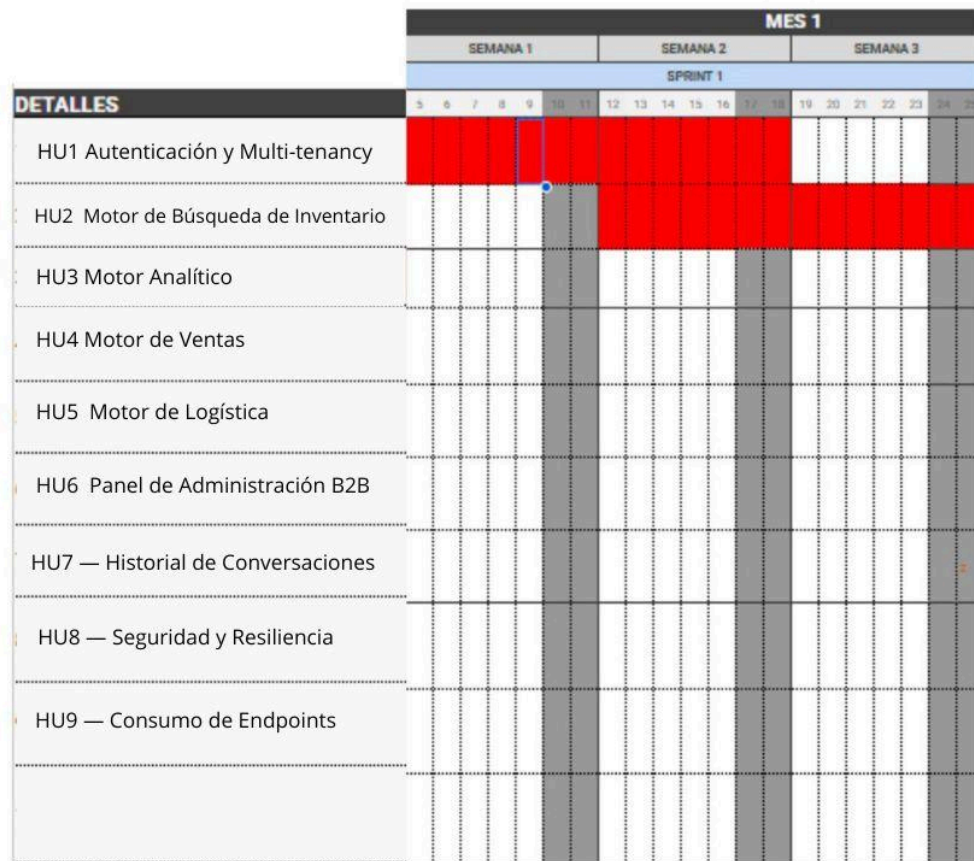


Figura 3.1: Planificación Gantt. Fuente: Elaboración propia.

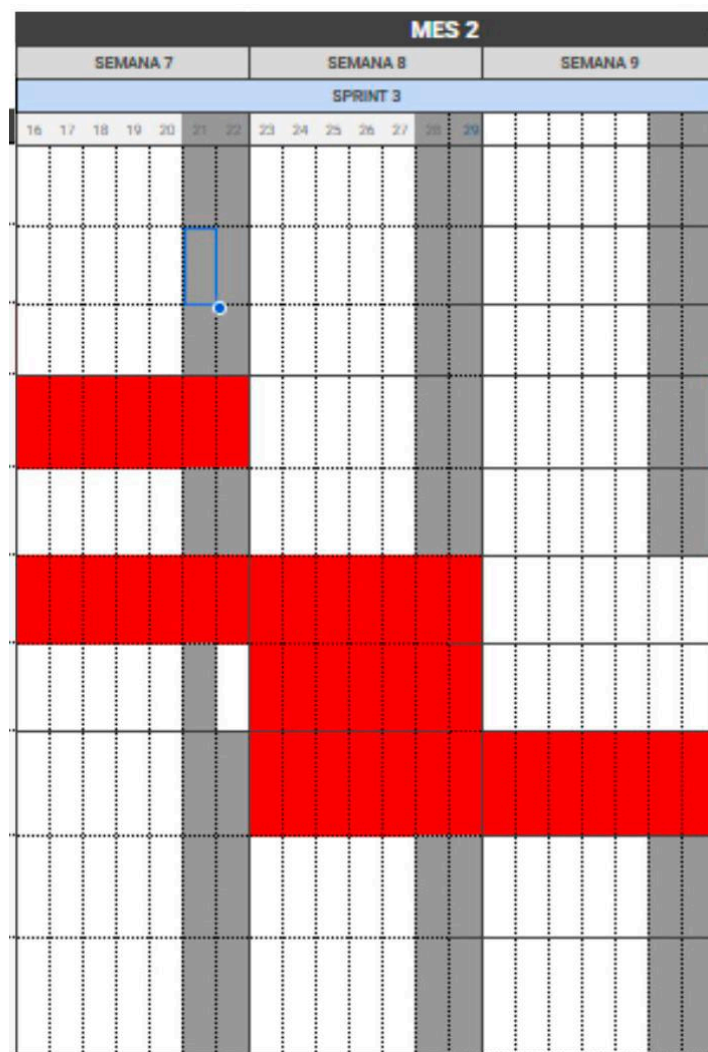


Figura 3.3: Planificación Gantt. Fuente: Elaboración propia.

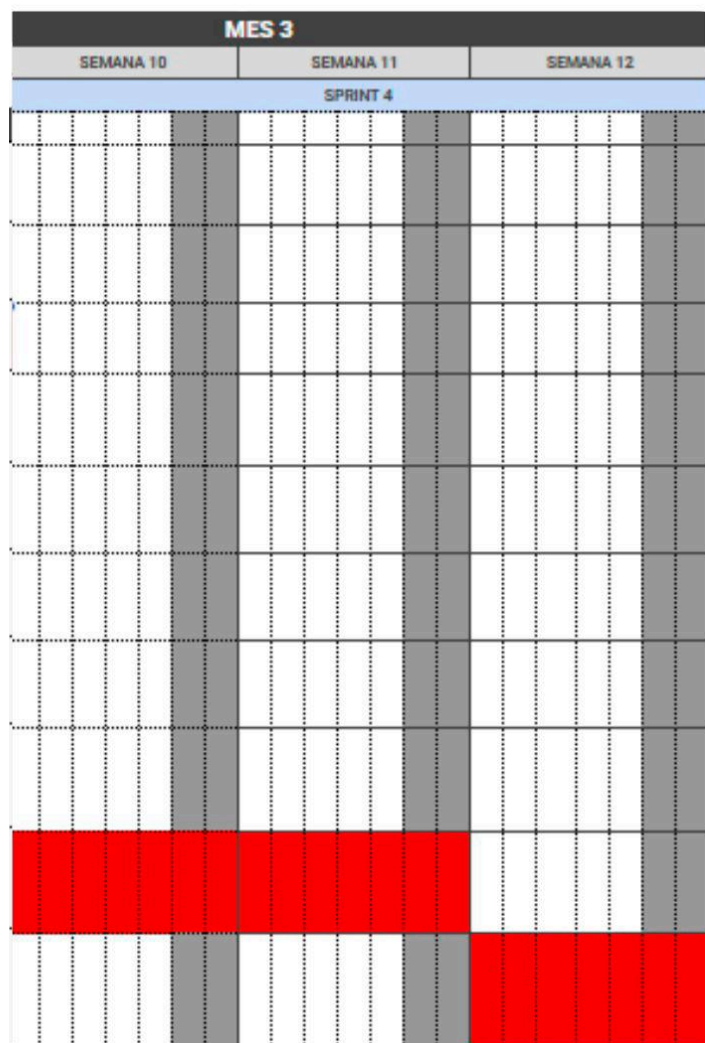


Figura 3.4: Planificación Gantt. Fuente: Elaboración propia.



Figura 4: Diagrama de arquitectura de la solución (TO-BE). Fuente: Elaboración propia.

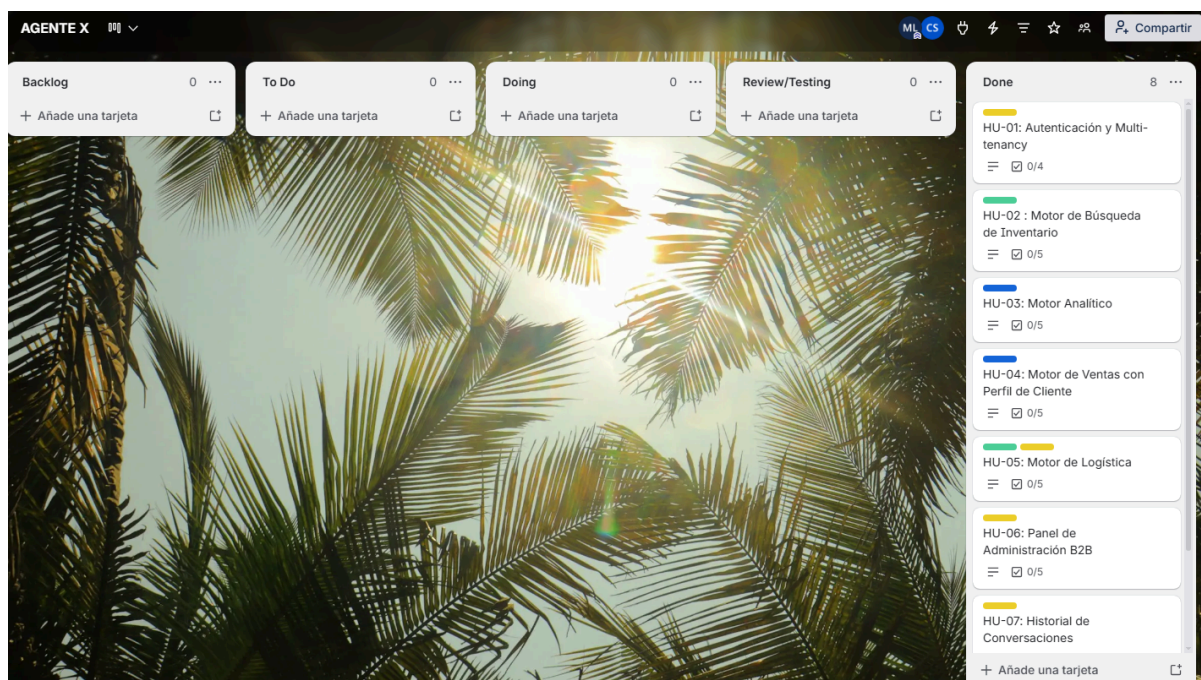


Figura 5: Tablero de tareas actualizado en Trello. Fuente: Elaboración propia Trello.

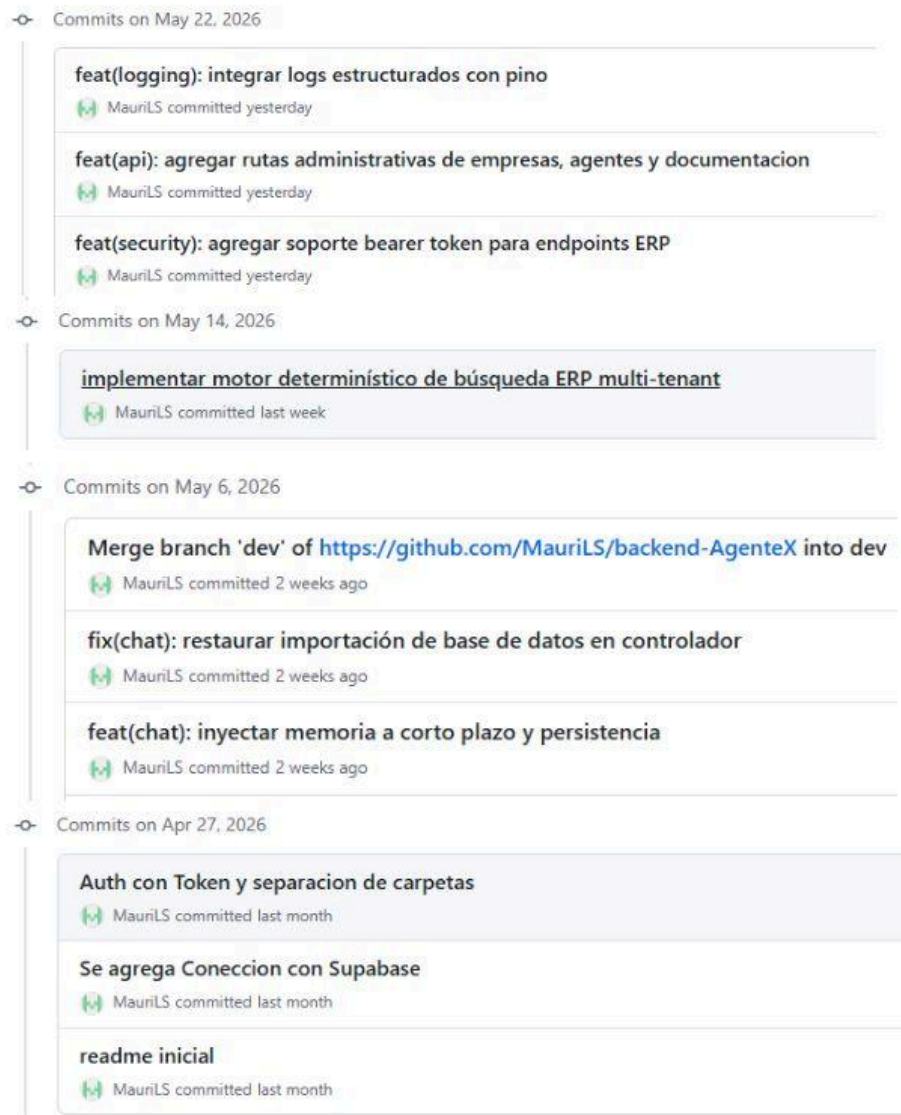


Figura 6: Historial de commits del repositorio GitHub . Fuente: Elaboración propia.

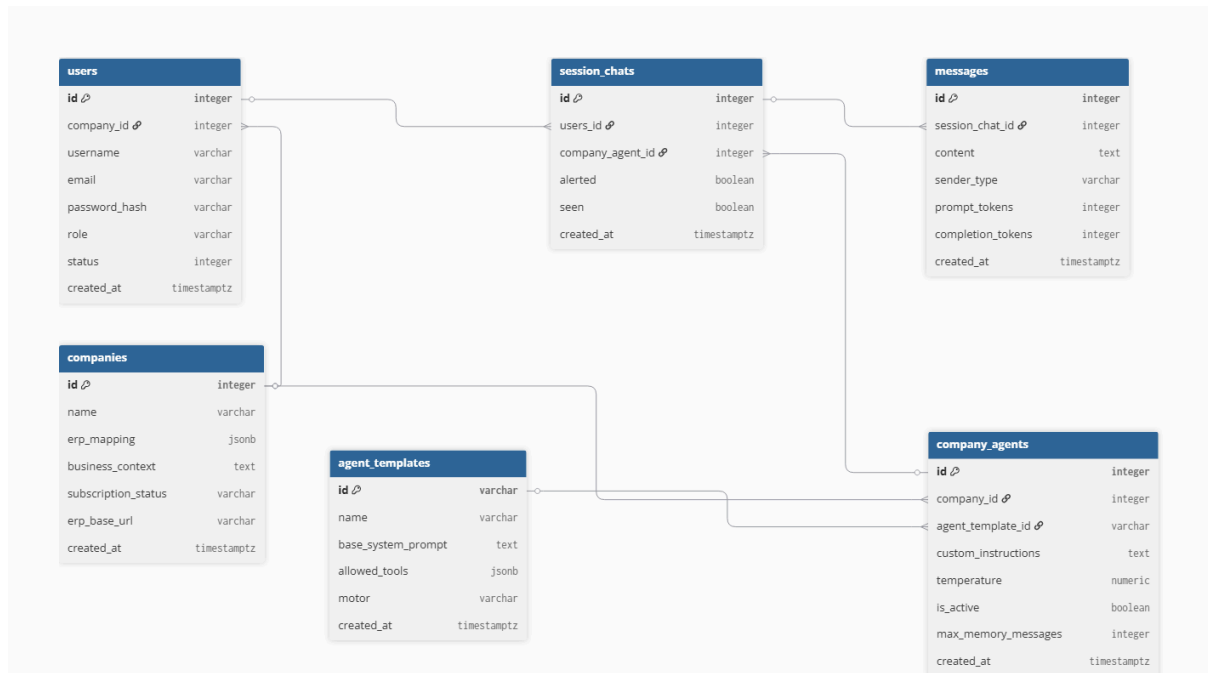


Figura 7: Modelo Entidad-Relación de la base de datos. Fuente: Elaboración propia/dbdiagram

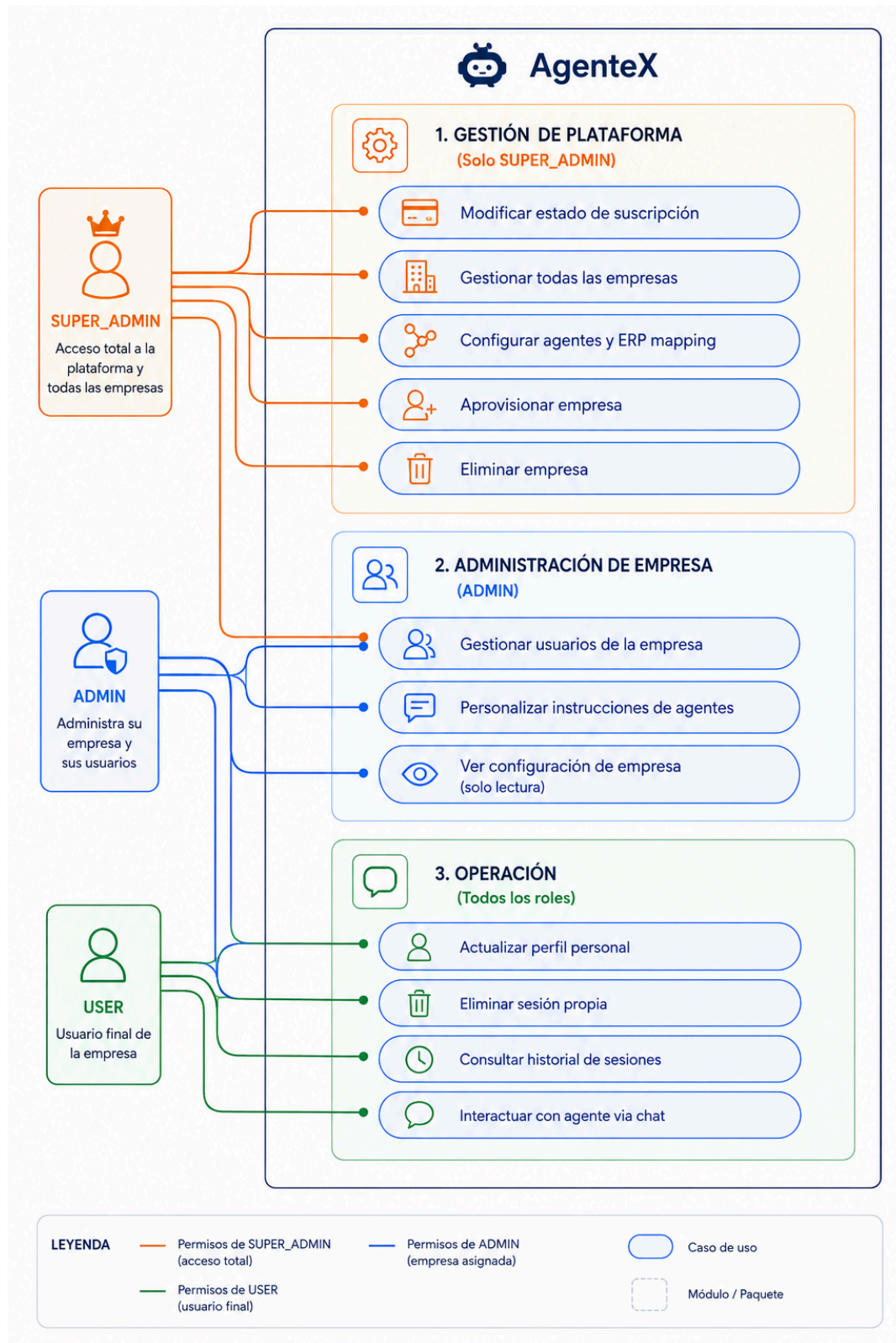


Figura 8: Diagrama de casos de uso. Fuente: Elaboración propia.

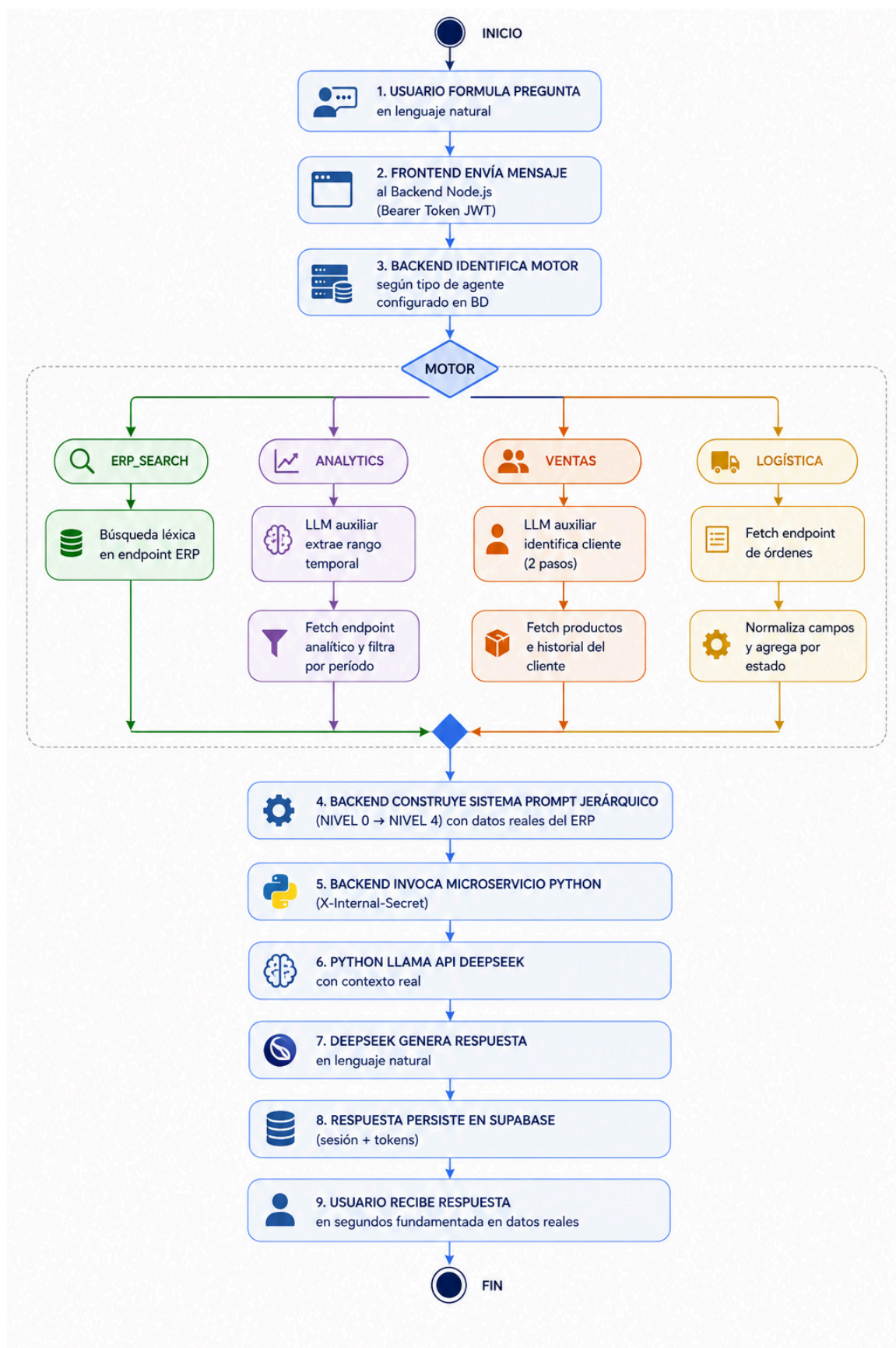


Figura 9: Diagrama de flujo del proceso nuevo (TO-BE). Fuente: Elaboración propia.

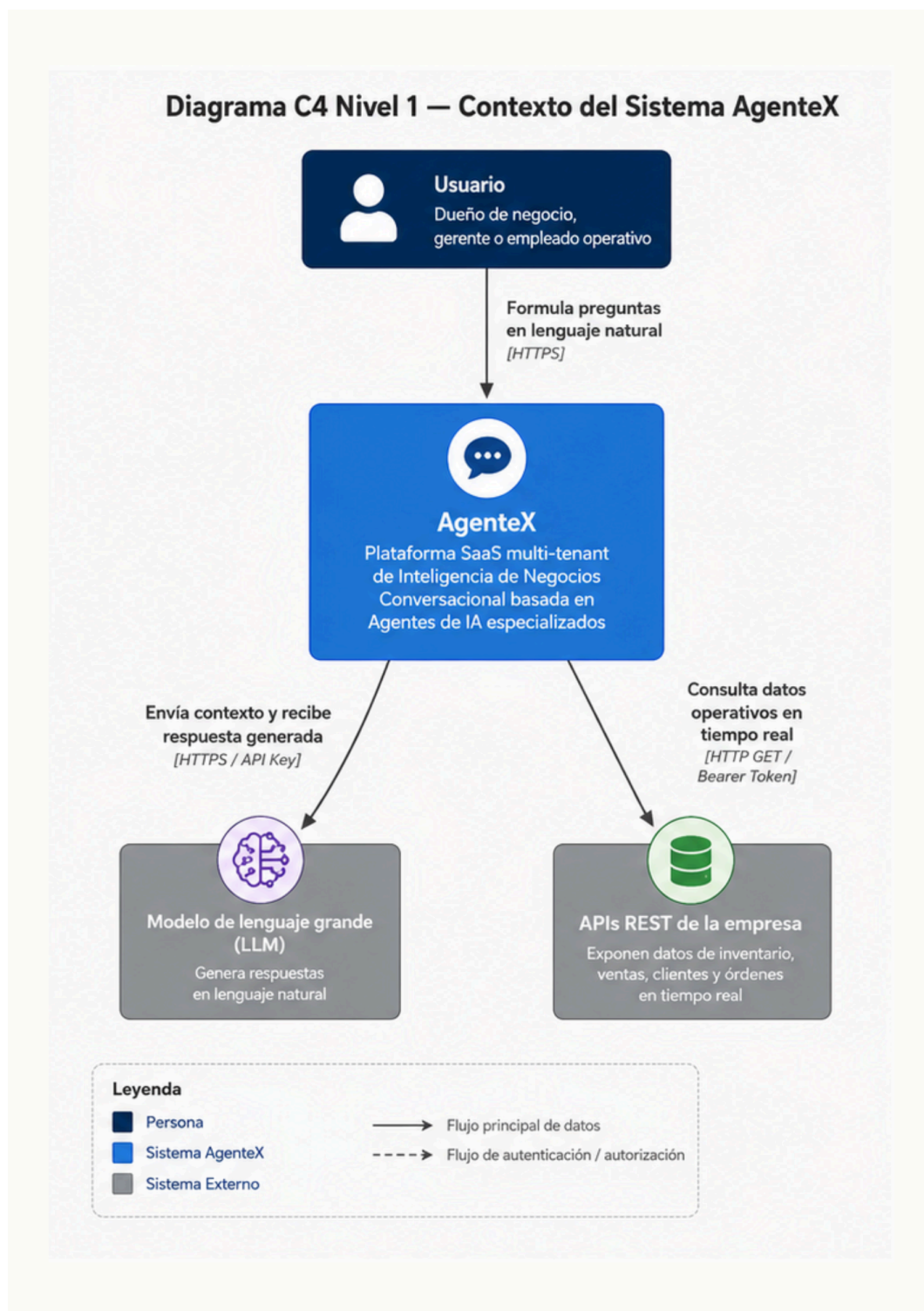


Figura 10: Diagrama C4 Nivel 1 — Contexto del sistema. Fuente: Elaboración propia.

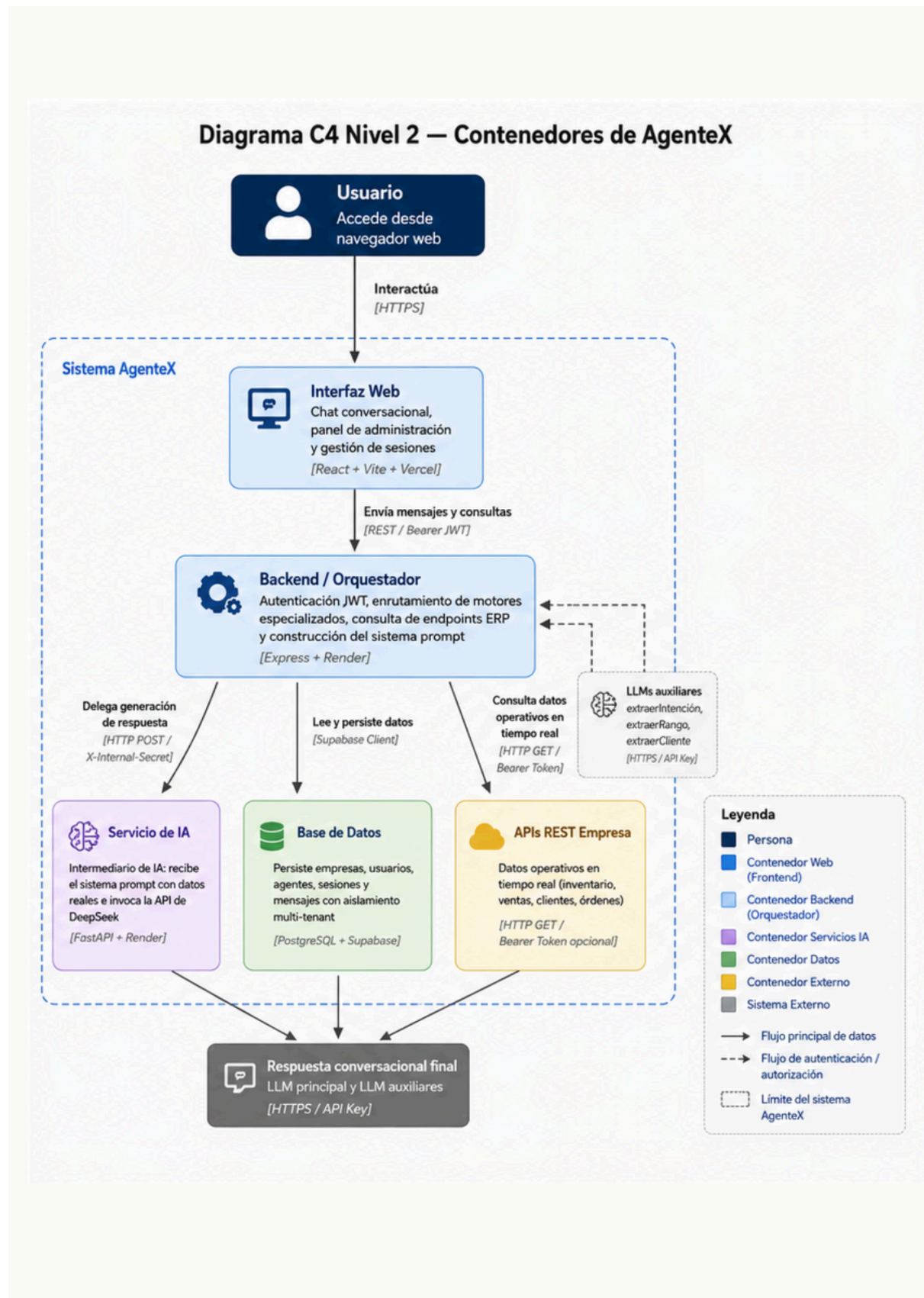


Figura 11: Diagrama C4 Nivel 2 — Contenedores del sistema. Fuente: Elaboración propia.

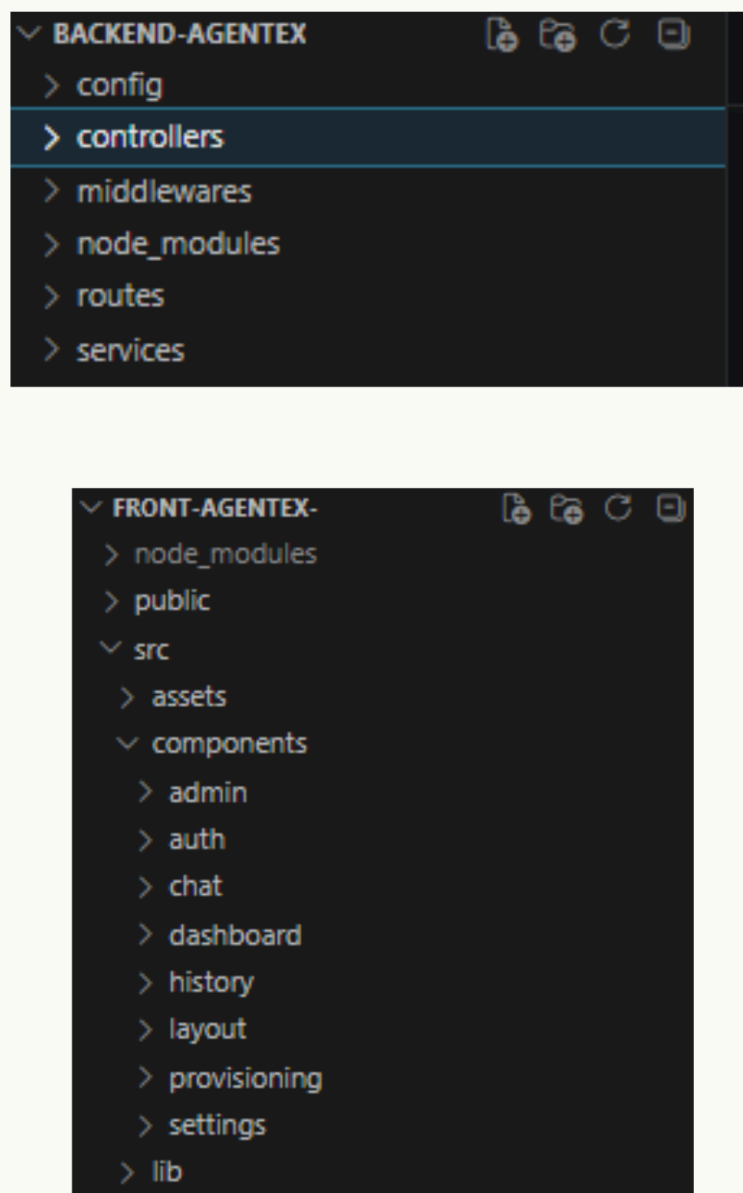


Figura 12: Captura de estructura de carpetas Fuente: Elaboración propia.

WEB SERVICE

backend-AgenteX

Node Free Upgrade your instance →

Service ID: `srv-d7vp8crtqb8s73fj5b80`

MauriLS / backend-AgenteX main

<https://backend-agentex.onrender.com>

Inicio Recortes

Edición Herramientas Ayuda

ⓘ Your free instance will spin down with inactivity, which can delay requests by 50 seconds

Filter events 31

✓ Deploy live for `9b701b8`: feat(logging): integrar logs estructurados con pino
May 22, 2026 at 11:18 PM

Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more.](#)

KEY	VALUE
DEEPSEEK_API_KEY	
INTERNAL_SECRET	
JWT_SECRET	
NODE_ENV	
PORT	
PYTHON_ENGINE_URL	
SUPABASE_KEY	
SUPABASE_URL	

Figura 13: Dashboard Render — Backend Node.js (Live) Fuente: Elaboración propia.

🌐 WEB SERVICE

motor-ia-Agentex

Python 3 Free Upgrade your instance →

Service ID: `srv-d7vpr157wvec73dlbtbkg`

🔗 MauriLS / motor-ia-Agentex 🌿 main

<https://motor-ia-agentex.onrender.com>

Your free instance will spin down with inactivity, which can delay requests by 50 seconds

Filter events 31

Deploy live for `788aa f9`: Merge branch 'dev'

May 22, 2026 at 12:21 PM

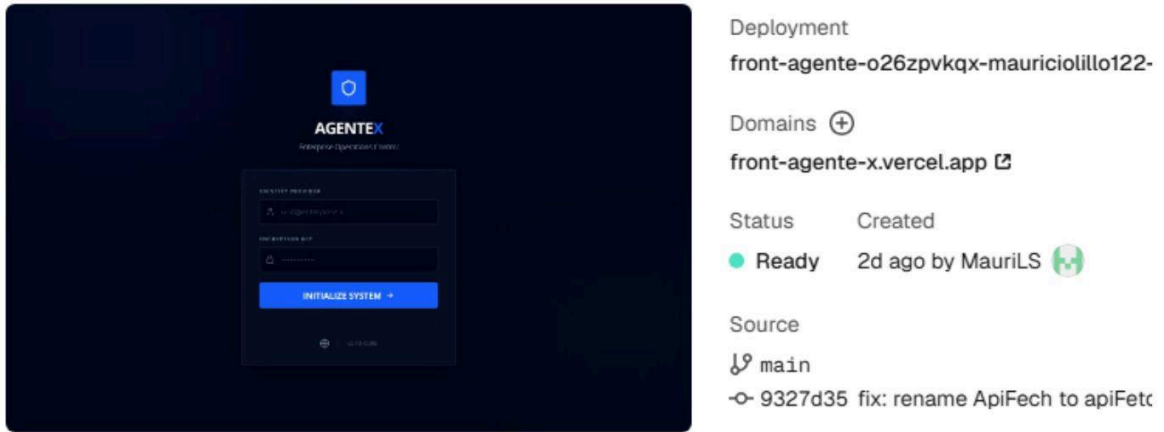
Environment Variables

Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#).

KEY	VALUE
DEEPSEEK_API_KEY	
INTERNAL_SECRET	

Figura 14: Dashboard Render — Microservicio Python (Live) Fuente: Elaboración propia.

Production Deployment



Deployment
front-agente-o26zpvkqx-mauriciolillo122-

Domains [+](#)
front-agente-x.vercel.app [↗](#)

Status Created
● Ready 2d ago by MauriLS 

Source
[↗](#) main
[↗](#) 9327d35 fix: rename ApiFech to apiFetc

Environment Variables

Store API keys, tokens, and config securely. [Learn more](#) [↗](#)

Project Shared

[🔍](#) Search variables

All Environments [▼](#)



VITE_API_URL Sensitive
Production

Figura 15: Dashboard Vercel — Frontend desplegado Fuente: Elaboración propia.

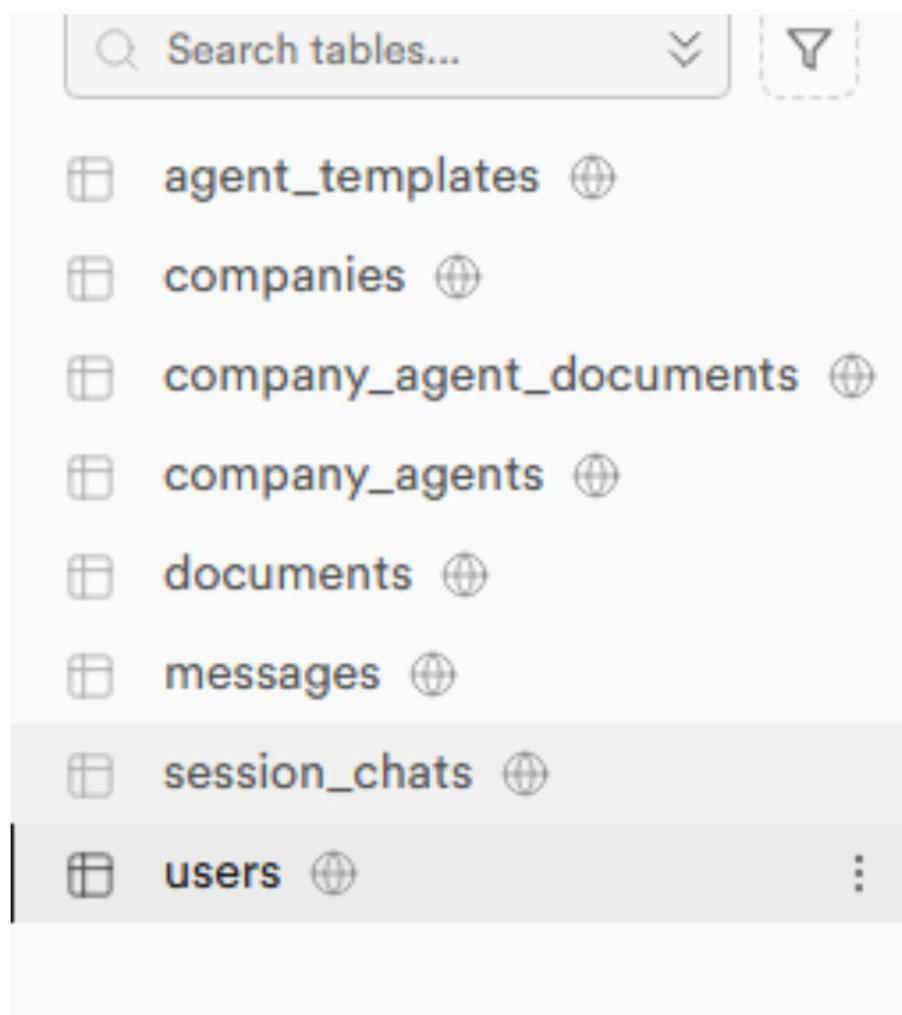
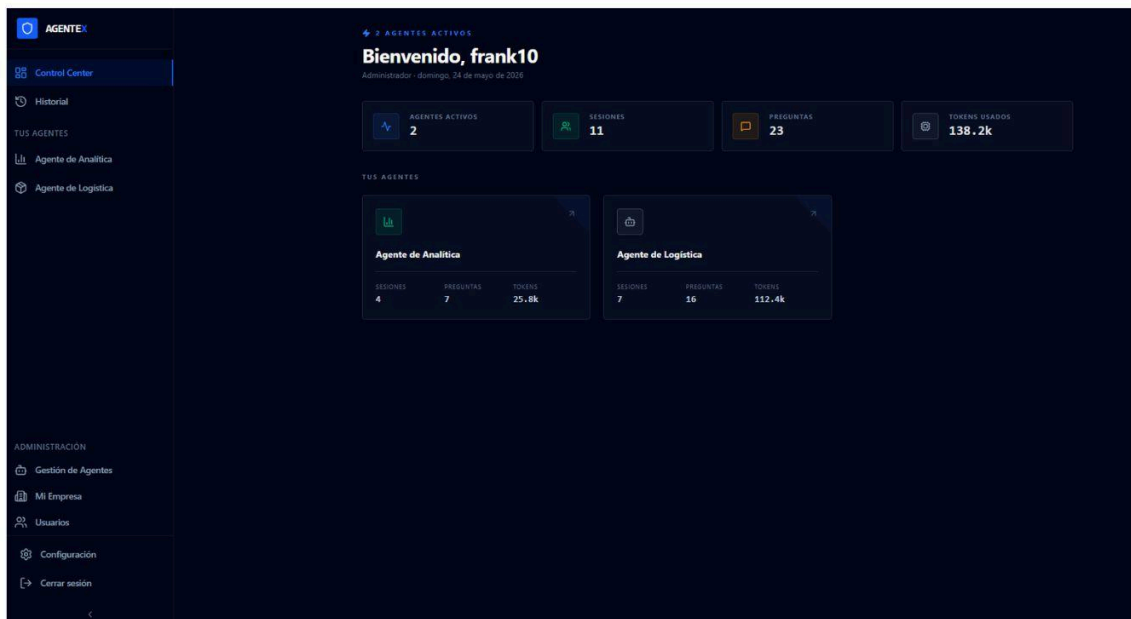


Figura 16: Supabase — Panel de tablas Fuente: Elaboración propia.



```

01:07:57 PM      ==> Deploying...
01:07:57 PM      ==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
01:08:04 PM [odbxr] ==> Running 'npm start'
01:08:05 PM [odbxr]
01:08:05 PM [odbxr] > start
01:08:05 PM [odbxr] > node server.js
01:08:05 PM [odbxr]
01:08:06 PM [odbxr] < injected env (0) from .env // tip: ✦ secrets for agents [www.dotenvx.com]
01:08:07 PM [odbxr] < injected env (0) from .env // tip: ~ auth for agents [www.vestauth.com]
01:08:07 PM [odbxr] {"level":30,"time":1779642487688,"service":"agentex-backend","env":"production","port":"3000","env"
01:08:07 PM [odbxr] {"level":40,"time":1779642487875,"service":"agentex-backend","env":"production","method":"HEAD","pa
01:08:09 PM      ==> Your service is live 🎉
01:08:09 PM [odbxr] {"level":40,"time":1779642489362,"service":"agentex-backend","env":"production","method":"GET","pat
01:08:09 PM      ==>
01:08:09 PM      ==> //////////////////////////////////////
01:08:09 PM      ==>
01:08:09 PM      ==> Available at your primary URL https://backend-agentex.onrender.com
01:08:09 PM      ==>
01:08:09 PM      ==> //////////////////////////////////////

```

Figura 17: Log de deploy exitoso en Render Fuente: Elaboración propia.

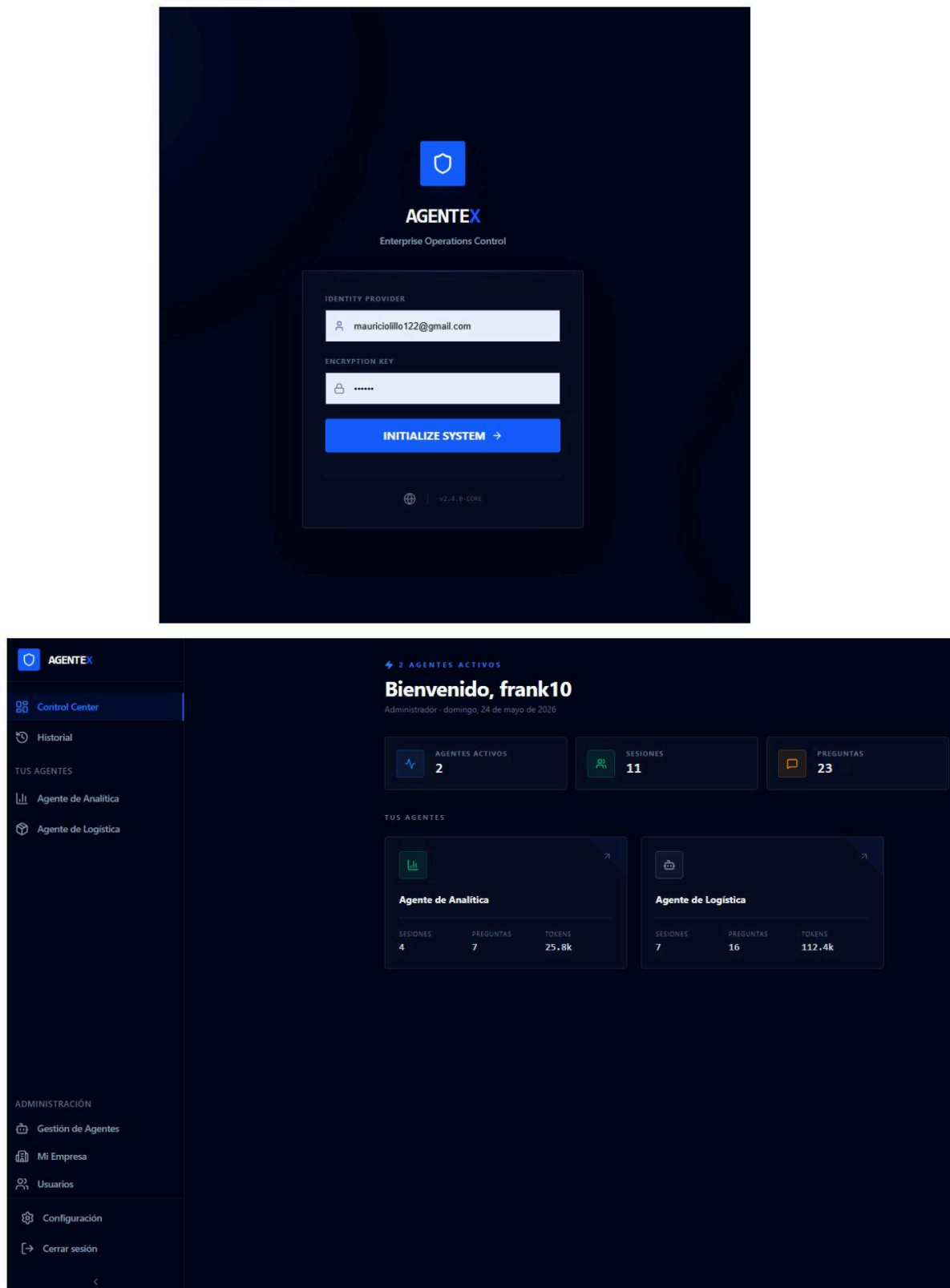


Figura 18: Login y Dashboard con métricas reales Fuente: Elaboración propia.

Enlace Activo
Sistema de orquestación en línea. Esperando comandos o consultas de inventario.

ERP SYNCED

01:55:41 P.M. • LOGGED

Quiero saber que neumaticos 26 tengo con stock critico

01:56:02 P.M. • LOGGED

REPORTE DE INVENTARIO: Neumáticos 26 con stock crítico

Según los datos del ERP, los neumáticos Aro 26 con stock igual o inferior a 2 unidades (alerta mínima) son los siguientes:

ID/SKU	PRODUCTO	PRECIO	STOCK
3220	NEUMATICO 26 X 2.10 ROKY	\$25000	2
3221	Neumatico Aro 26x13/8 Ralco	\$8990	2
3757	neumatico 26 x 2.10 cross country	\$29990	2
4056	neumatico 26 x 2.40 gladiator	\$19900	2
858	Neumatico 26 x 2.125 con caluga	\$15000	3
1109	Neumatico 26x210 RBX	\$12000	3

Los primeros 4 productos están en alerta mínima (stock 2). Los últimos 2 tienen stock 3, rozando el límite crítico.

ERP SYNCED

COMMAND INPUT

Escribe tu consulta...

EXECUTE

Figura 19: Chat agente de bodega con resultados ERP Fuente: Elaboración propia.

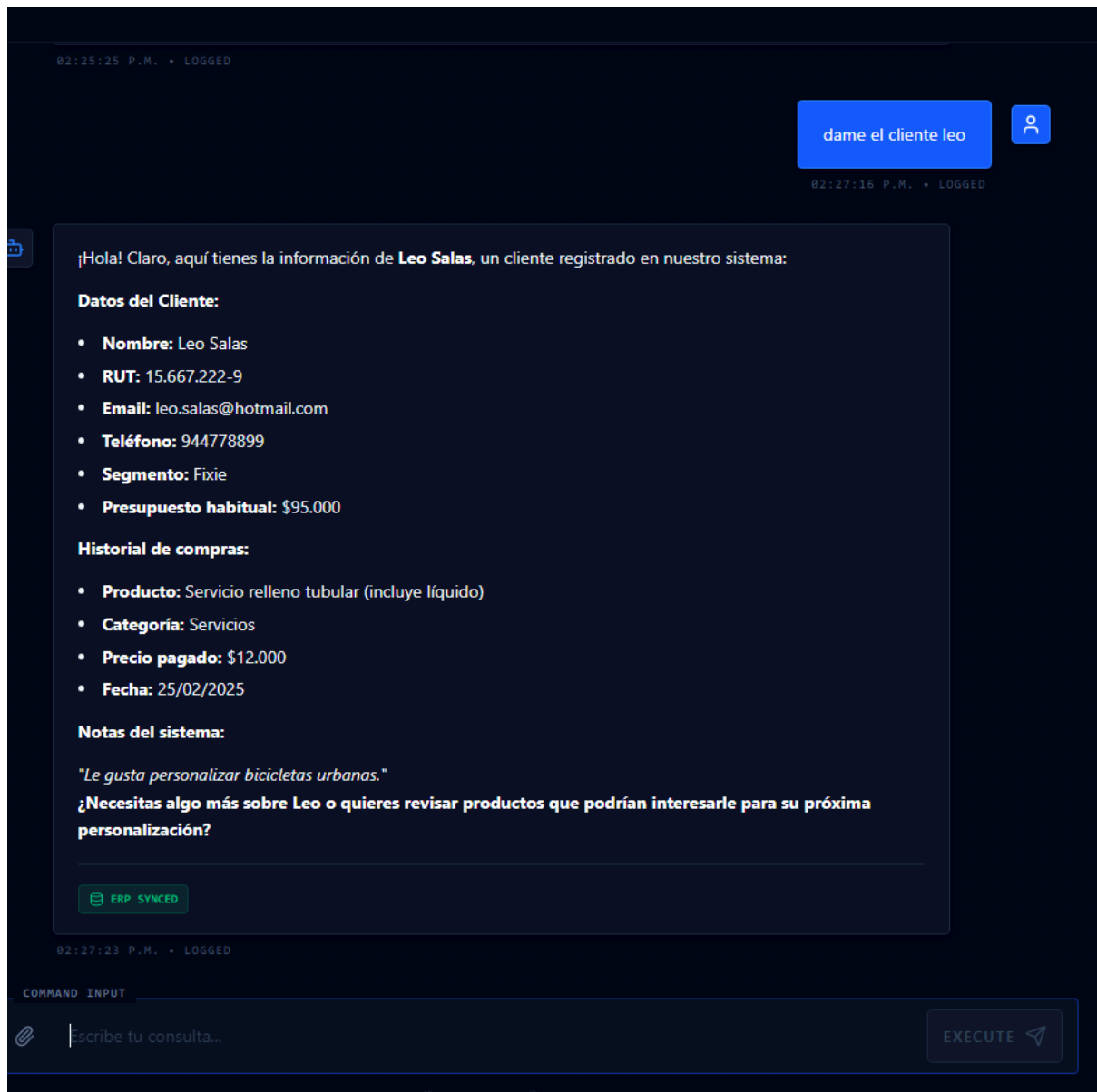


Figura 20: Chat agente ventas con tabla Markdown

Fuente: Elaboración propia.

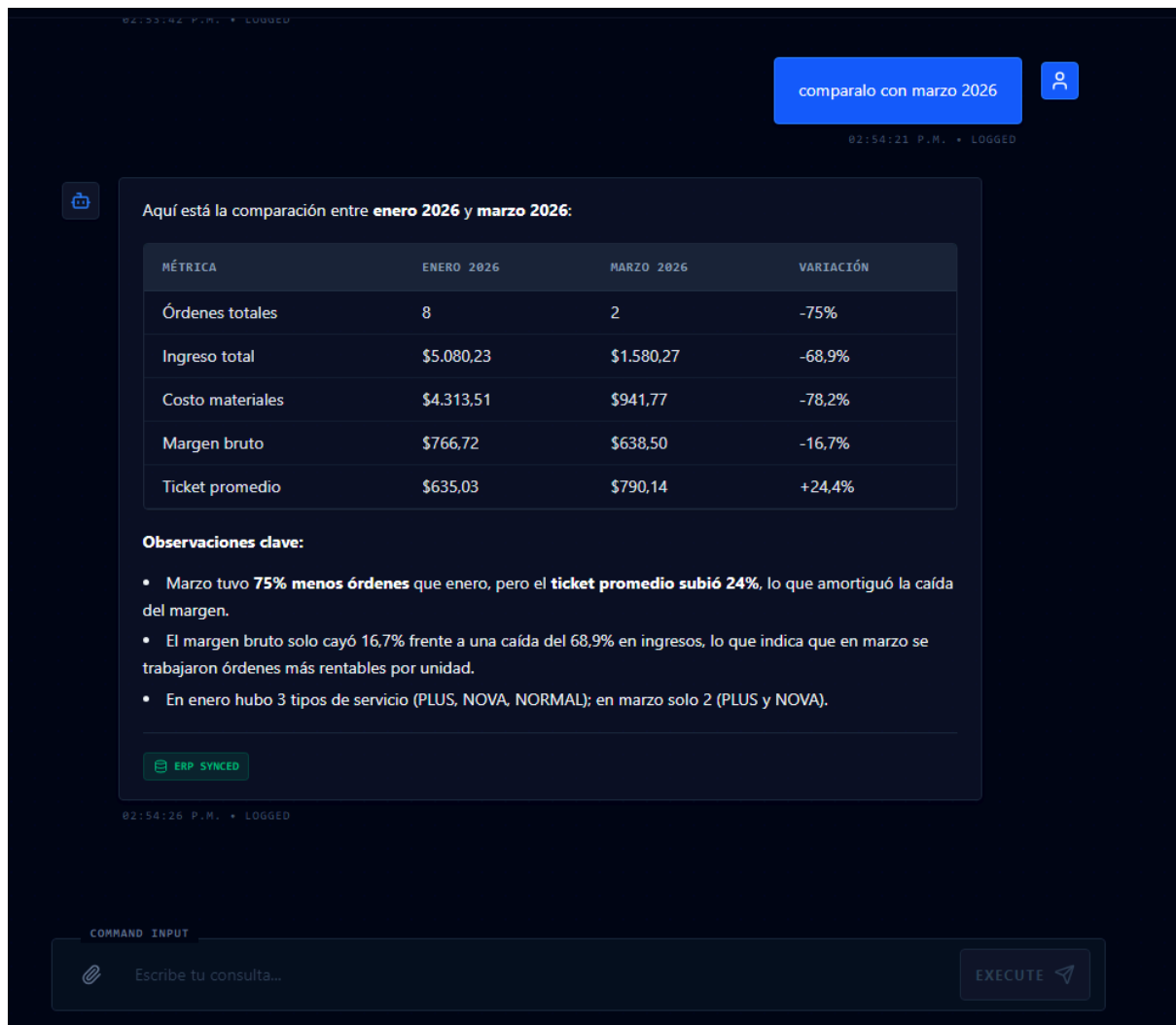


Figura 21: Chat agente analítico con tabla Markdown Fuente: Elaboración propia.

Dame las ordenes pendientes de marzo 2025

02:58:01 P.M. • LOGGED

Aquí tienes las órdenes **pendientes** de marzo 2025:

Nº ORDEN	TIPO	CLIENTE	DIRECCIÓN	RESPONSABLE
OT-1042	Migración Servicio	Sebastián Torres	Calle Sur 91, Cerrillos	Carlos Muñoz
OT-1045	Mantenición	Natalia Fuentes	Pasaje Verde 21, Santiago	Pedro Soto
OT-1048	Instalación Antena	Rodrigo Espinoza	Los Olmos 101, Quilicura	Matías Fuentes
OT-1052	Soporte Técnico	Andrea Castillo	Pasaje Azul 98, Santiago	Carlos Muñoz
OT-1055	Instalación Antena	Cristóbal Vega	Los Aromos 400, Pudahuel	Pedro Soto
OT-1058	Mantenición	Natalia Campos	Los Pinos 333, Maipú	Pedro Soto
OT-1062	Migración Servicio	Claudio Espinoza	Calle Uno 501, Talagante	Carlos Muñoz
OT-1065	Soporte Técnico	Daniela Pérez	Los Cipreses 211, Maipú	Pedro Soto

Total: 8 órdenes pendientes.

Alertas:

- **OT-1052** (Andrea Castillo) tiene prioridad **Alta** y su fecha compromiso fue el 18/03 — ya está vencida.
- **OT-1042** (Sebastián Torres) venció el 06/03, aunque su prioridad es Baja.

¿Necesitas más detalle de alguna de estas órdenes?

ERP SYNCED

COMMAND INPUT

scribe tu consulta...

EXECUTE

Figura 22: Chat agente de logística Fuente: Elaboración propia.

Despliegue de Infraestructura B2B

Aprovisionamiento de tenants y asignación de agentes IA.

IDENTIDAD CORPORATIVA

Razón Social *

URL Endpoint de Productos ⓘ
https://api.empresa.com/productos

Nombre Admin *

Email Admin *

Contraseña Temporal *

MAPEO DE CAMPOS ERP

Opcional — detecta los campos automáticamente desde el JSON

CONTEXTO DEL NEGOCIO ⓘ

Recomendado — mejora significativamente la comprensión del agente

AGENTES A DESPLEGAR

Módulo ⓘ
Bodega (Inventario) ▼

Temperatura (0.3) ⓘ

Preciso (0.0)Creativo (1.0)

Instrucciones del Agente * ⓘ
Eres el operario de bodega de [EMPRESA]. Tono crudo y transaccional. Solo reportas datos del ERP, nunca inventas...

+ Añadir Agente

Figura 23: Panel de aprovisionamiento B2B con mapeo ERP Fuente: Elaboración propia.

Gestión de empresas

2 empresas registradas.

**DkmNet10**
ID: 28 • 16-05-2026

ACTIVE ^

NOMBRE DE LA EMPRESA

ESTADO DE SUSCRIPCIÓN

DkmNet10

ACTIVE ▾

CONTEXTO DEL NEGOCIO

ERP MAPPING (SOLO LECTURA)

```
{
  "id": "id",
  "costo": "costo_materiales",
  "fecha": "fecha_orden",
  "comuna": "comuna",
  "precio": "ingreso_clp",
  "tecnico": "tecnico_asignado",
  "categoria": "tipo_servicio",
  "ordenes_url": "https://6a0e26f01736097c36098b0d.mockapi.io/ordenes",
  "productos_url": "https://6a08bcf5e7e3f433d482cc2e.mockapi.io/api/v1/instalaciones"
}
```

 Eliminar empresa

Guardar

**Xtreme Logistics**
ID: 1 • 10-05-2026

ACTIVE ▾

Figura 24: Panel de gestión de Empresas, vista Super Admin Fuente: Elaboración propia.



Figura 25: Panel de gestión de agentes, vista Admin Fuente: Elaboración propia.

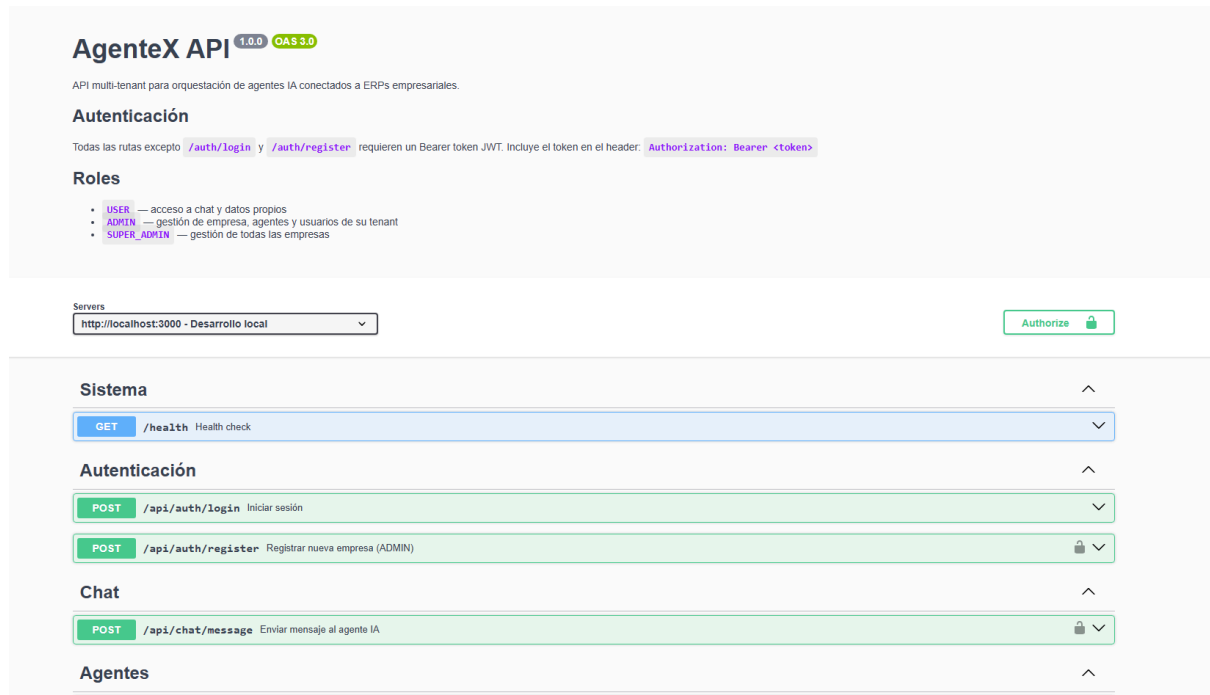


Figura 26: Documentación Swagger en `/api/docs` Fuente: Elaboración propia.

```
02:58:07 PM [m8vqc] 📦 Logistics cache hit → 20:https://6a0e26f01736097c360908b0d.mockapi.io/ordenes
02:58:08 PM [m8vqc] 📦 Rango logistica: 2025-03-01 → 2025-03-31
02:58:08 PM [m8vqc] 📦 Logistica → base: 100 | filtradas: 28 | rango: 2025-03-01 → 2025-03-31
02:58:08 PM [m8vqc] 📦 Logistica → modo: crudo | órdenes: 28
02:58:12 PM [m8vqc] {"level":30,"time":1779649992012,"service":"agentex-backend","env":"production","method":"POST","path":"/message","status":200,"ms":5387,"ip":"10.28.126.1","user_id":28,"msg":{"POST /message 200 5387ms"}}
```

Figura 27: Logs estructurados con Pino en producción — Render. Fuente: Elaboración propia.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	99.7	91.05	100	100	
controllers	99.78	94.53	100	100	
admin.controller.js	100	100	100	100	
agent.controller.js	100	100	100	100	
auth.controller.js	100	97.05	100	100	37
chat.controller.js	100	90.96	100	100	24,103,132,204,253-254,407,418,524,578-589,639-640,658-659
company.controller.js	100	100	100	100	
sessions.controller.js	98.33	90	100	100	101,119-120,125
users.controller.js	100	100	100	100	
middlewares	100	100	100	100	
auth.js	100	100	100	100	
services	99.63	88.45	100	100	
analytics.service.js	99.21	88.57	100	100	23-24,104,140,160,169-180,198
erp-search.service.js	100	85.5	100	100	15-16,54,82-87,150,235,250,269,274,294-296,326-328,344-367
logistics-search.service.js	99.2	90.26	100	100	23,29,48,100,132-140,147,150,163-166,202
sales-search.service.js	100	90.35	100	100	23,60,113,159,166-174,191,208,235,239
text-utils.service.js	100	100	100	100	
Test Suites: 37 passed, 37 total					
Tests: 369 passed, 369 total					
Snapshots: 0 total					
Time: 7.966 s, estimated 10 s					
Ran all test suites.					

Figura 28: Salida de consola: npm test — Backend Node.js (369/369 pasando, coverage completo). Fuente: Elaboración propia.

All files services

99.63% Statements 351/351 88.45% Branches 432/511 100% Functions 86/86 100% Lines 586/586

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements	Branches	Functions	Lines
analytics.service.js		99.21%	127/128	88.57%	62/70
erp-search.service.js		100%	158/158	85.5%	171/200
logistics-search.service.js		99.2%	124/125	90.26%	102/113
sales-search.service.js		100%	123/123	90.35%	103/114
text-utils.service.js		100%	19/19	100%	14/14

All files middlewares

100% Statements 34/34 100% Branches 8/8 100% Functions 3/3 100% Lines 34/34

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements	Branches	Functions	Lines
auth.js		100%	14/14	100%	6/6

All files controllers

99.78% Statements 457/458 94.53% Branches 344/364 100% Functions 37/37 100% Lines 488/488

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements	Branches	Functions	Lines
admin.controller.js		100%	40/40	100%	24/24
agent.controller.js		100%	69/69	100%	56/56
auth.controller.js		100%	50/50	97.05%	33/34
chat.controller.js		100%	173/173	90.96%	151/166
company.controller.js		100%	24/24	100%	16/16
sessions.controller.js		98.33%	59/60	90%	36/40
users.controller.js		100%	42/42	100%	30/30

All files

99.7% Statements 3422/3425 91.05% Branches 886/983 100% Functions 324/324 100% Lines 528/528

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File		Statements	Branches	Functions	Lines
controllers		99.78%	457/458	94.53%	346/366
middlewares		100%	14/14	100%	6/6
services		99.63%	551/553	88.45%	452/511

Figura 29: Reporte HTML de cobertura Istanbul — Backend Node.js (branch coverage 91.05% por archivo). Fuente: Elaboración propia.

```

PS C:\Users\Rayker\Documents\Front-AgenteX\Front-AgenteX-> npx vitest run

RUN v3.2.6 C:\Users\Rayker\Documents\Front-AgenteX\Front-AgenteX-

✓ src/test/unit/components/provisioning/ERPMappingSection.test.jsx (14 tests) 2850ms
  ✓ ERPMappingSection > JSON válido + Detectar campos + muestra filas con los campos del JSON 306ms
  ✓ ERPMappingSection > URL del endpoint secundario llama onChange con stock_url 752ms
  ✓ ERPMappingSection > token ERP llama onChange con erp_token en el mapping 435ms
✓ src/test/integration/AgentsSettings.test.jsx (10 tests) 2934ms
  ✓ Integración - AgentsSettings + AgentCard + NuevoAgenteModal > INT-A03: AgentCard > editar instrucciones + guardar + PUT con cuerpo correcto 974ms
  ✓ Integración - AgentsSettings + AgentCard + NuevoAgenteModal > INT-A04: después de guardar el botón queda disabled (dirty=false) 533ms
  ✓ Integración - AgentsSettings + AgentCard + NuevoAgenteModal > INT-A08: crear agente via modal + POST + refetch + agente aparece en lista 627ms
✓ src/test/unit/components/chat/ChatInterface.test.jsx (9 tests) 3082ms
  ✓ ChatInterface > renderiza el mensaje de bienvenida inicial 370ms
  ✓ ChatInterface > cambiar agentId reinicia el chat al mensaje de bienvenida 545ms
  ✓ ChatInterface > send exitoso: agrega mensaje usuario, reply y muestra SESSION_ID 378ms
  ✓ ChatInterface > send con sessionId ya seteado no lo sobrescribe (rama !sessionId=false) 837ms
  ✓ ChatInterface > respuesta HTTP no-OK agrega mensaje de CRITICAL ERROR 394ms
✓ src/test/smoke/App.smoke.test.jsx (15 tests) 2995ms
  ✓ Smoke - Frontend AgenteX > SMOKE-F02: LoginScreen muestra todos los campos del formulario de acceso 351ms
  ✓ Smoke - Frontend AgenteX > SMOKE-F03: Login exitoso guarda token en localStorage 547ms
  ✓ Smoke - Frontend AgenteX > SMOKE-F04: Login con credenciales incorrectas muestra error y no crashea 473ms
  ✓ Smoke - Frontend AgenteX > SMOKE-F05: Error de red en login muestra mensaje y no crashea 495ms
✓ src/test/unit/components/settings/AgentsSettings.test.jsx (17 tests) 3390ms
  ✓ AgentsSettings > cambiar instrucciones activa el botón Guardar 490ms
  ✓ AgentsSettings > guardar agente llama PUT y actualiza el estado 323ms
  ✓ AgentsSettings > submit OK crea el agente, cierra el modal y recarga la lista 699ms
  ✓ AgentsSettings > submit con res.ok=false muestra el error del backend en el modal 447ms
✓ src/test/unit/components/auth/Register.test.jsx (13 tests) 4650ms
  ✓ RegisterB2B > addAgent añade un segundo agente de tipo ventas 429ms
  ✓ RegisterB2B > handleBaseChange actualiza el campo al escribir 410ms
  ✓ RegisterB2B > handleSalesConfigChange actualiza URL del agente de ventas 1020ms
  ✓ RegisterB2B > submit exitoso: muestra mensaje de éxito y resetea el formulario 660ms
  ✓ RegisterB2B > submit con agente de ventas construye mappingFinal con URLs de ventas 1100ms
✓ src/test/integration/ChatIntegration.test.jsx (9 tests) 5392ms
  ✓ Integración - ChatInterface + fetch > INT-C01: el payload incluye message, agent_id, history y session_chat_id 1103ms
  ✓ Integración - ChatInterface + fetch > INT-C03: session_chat_id de la primera respuesta se incluye en el segundo mensaje 861ms
  ✓ Integración - ChatInterface + fetch > INT-C04: sessionId no se sobrescribe en mensajes posteriores (rama !sessionId=false) 686ms
  ✓ Integración - ChatInterface + fetch > INT-C07: error HTTP no-401 agrega mensaje de error sin redirigir 332ms
  ✓ Integración - ChatInterface + fetch > INT-C08: cambiar agentId resetea chat y envía session_chat_id null en el siguiente mensaje 520ms
  ✓ Integración - ChatInterface + fetch > INT-C09: historial de conversación NO incluye el mensaje de bienvenida en el payload 1154ms

stderr | src/test/unit/components/auth/LoginScreen.test.jsx > LoginScreen > fallo de red muestra mensaje genérico
Fallo de red en login: Error: Network error
    at C:\Users\Rayker\Documents\Front-AgenteX\Front-AgenteX-src\test\unit\components\auth\LoginScreen.test.jsx:54:29
    at file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:155:11
    at file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:752:26
    at file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:1897:20
    at new Promise (<anonymous>)
    at runWithTimeout (file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:1863:10)
    at runTest (file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:1574:12)
    at runSuite (file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:1729:8)
    at runSuite (file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:1729:8)
    at runFiles (file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:1787:3)

✓ src/test/integration/HistoryIntegration.test.jsx (9 tests) 1224ms
✓ src/test/unit/components/auth/LoginScreen.test.jsx (4 tests) 2517ms
  ✓ LoginScreen > login exitoso guarda token/user y llama onLogin 840ms
  ✓ LoginScreen > credenciales inválidas muestra el mensaje de error del backend 488ms
  ✓ LoginScreen > fallo de red muestra mensaje genérico 514ms
  ✓ LoginScreen > deshabilita el botón mientras isLoading es true 670ms
✓ src/test/unit/components/settings/CompanySettings.test.jsx (9 tests) 1992ms
  ✓ CompanySettings > submit con nombre cambiado llama PATCH y muestra éxito 483ms
  ✓ CompanySettings > submit con PATCH error muestra el mensaje del backend 318ms
  ✓ CompanySettings > cambiar el textarea de contexto del negocio cubre su onChange handler 417ms
✓ src/test/unit/components/settings/ProfileSettings.test.jsx (8 tests) 2329ms
  ✓ ProfileSettings > error inline "No coinciden" cuando password !== confirm mientras se escribe 642ms
  ✓ ProfileSettings > submit con passwords distintas: muestra error de validación y no llama fetch PUT 508ms
  ✓ ProfileSettings > onChange de username y email cubre los handlers de todos los campos 571ms
✓ src/test/unit/components/admin/CompaniesAdmin.test.jsx (12 tests) 2764ms
  ✓ CompaniesAdmin > buscador SE muestra y filtra cuando hay >4 empresas 363ms
  ✓ CompaniesAdmin > sin resultados de búsqueda muestra mensaje específico 338ms
  ✓ CompaniesAdmin > cambiar nombre activa el indicador dirty y habilita el botón Guardar 368ms
  ✓ CompaniesAdmin > guardar cambios llama handleUpdate y muestra checkmark 713ms
✓ src/test/security/input-validation.security.test.jsx (4 tests) 2707ms
  ✓ Seguridad - campos requeridos bloquean el envío > LoginScreen: formulario vacío no dispara fetch 349ms
  ✓ Seguridad - campos requeridos bloquean el envío > LoginScreen: solo email sin password no dispara fetch 492ms
  ✓ Seguridad - campos requeridos bloquean el envío > Register: falta password no dispara fetch aunque el resto esté completo 1721ms
stderr | src/test/regression/LoginScreen.regression.test.jsx > Regresión - BUG-FE-01: mensaje de error duplicado en LoginScreen > fallo de red renderiza el mensaje de e
Fallo de red en login: Error: Network error
    at C:\Users\Rayker\Documents\Front-AgenteX\Front-AgenteX-src\test\regression\LoginScreen.regression.test.jsx:45:29
    at file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:155:11
    at file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:752:26
    at file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:1897:20
    at new Promise (<anonymous>)
    at runWithTimeout (file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:1863:10)
    at runTest (file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:1574:12)
    at runSuite (file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:1729:8)
    at runSuite (file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:1729:8)
    at runFiles (file:///C:/Users/Rayker/Documents/Front-AgenteX/Front-AgenteX-/node_modules/@vitest/runner/dist/chunk-hooks.js:1787:3)

```

```

✓ src/test/regression/loginScreen.regression.test.jsx (2 tests) 1996ms
✓ Regresión - BUG-FE-01: mensaje de error duplicado en LoginScreen > credenciales inválidas renderiza el mensaje de error UNA sola vez 970ms
✓ Regresión - BUG-FE-01: mensaje de error duplicado en LoginScreen > fallo de red renderiza el mensaje de error UNA sola vez 530ms
✓ src/test/unit/components/history/SessionDetail.test.jsx (9 tests) 636ms
✓ src/test/regression/ERPMappingSection.regression.test.jsx (4 tests) 822ms
✓ Regresión - BUG-FE-02: extracción de claves desde JSON array vs objeto > array con un objeto + usa parsed[0], no las claves del array 472ms
✓ src/test/unit/components/history/SessionHistory.test.jsx (8 tests) 802ms
✓ src/test/unit/components/layout/Sidebar.test.jsx (14 tests) 1257ms
✓ src/test/unit/components/settings/UserSettings.test.jsx (7 tests) 759ms
✓ src/test/integration/SidebarNavigation.test.jsx (9 tests) 845ms

Stale
React does not recognize the `whileHover` prop on a DOM element. If you intentionally want it to appear in the DOM as a custom attribute, spell it as lowercase `whilehover`. Instead, if you accidentally passed it from a parent component, remove it from the DOM element.

✓ src/test/unit/components/dashboard/Dashboard.test.jsx (6 tests) 657ms
✓ src/test/unit/lib/utils.test.js (3 tests) 24ms
✓ src/test/unit/lib/apiFetch.test.js (4 tests) 16ms
✓ src/test/unit/api.test.jsx (11 tests) 356ms
✓ src/test/security/token-tampering.security.test.jsx (3 tests) 15ms
✓ src/test/security/session-expiry.security.test.jsx (6 tests) 168ms
✓ src/test/security/xss.security.test.jsx (3 tests) 405ms

Test Files 27 passed (27)
Tests 222 passed (222)
Start at 16:23:42
Duration 29.86s (transform 3.12s, setup 26.91s, collect 19.02s, tests 47.09s, environment 76.85s, prepare 11.82s)
PS C:\Users\Bayer\Documents\Front-AgenteX\Front-AgenteX>

```

Figura 30: Salida de consola: npx vitest run — Frontend React (unit + smoke + integration + regression + security). Fuente: Elaboración propia.

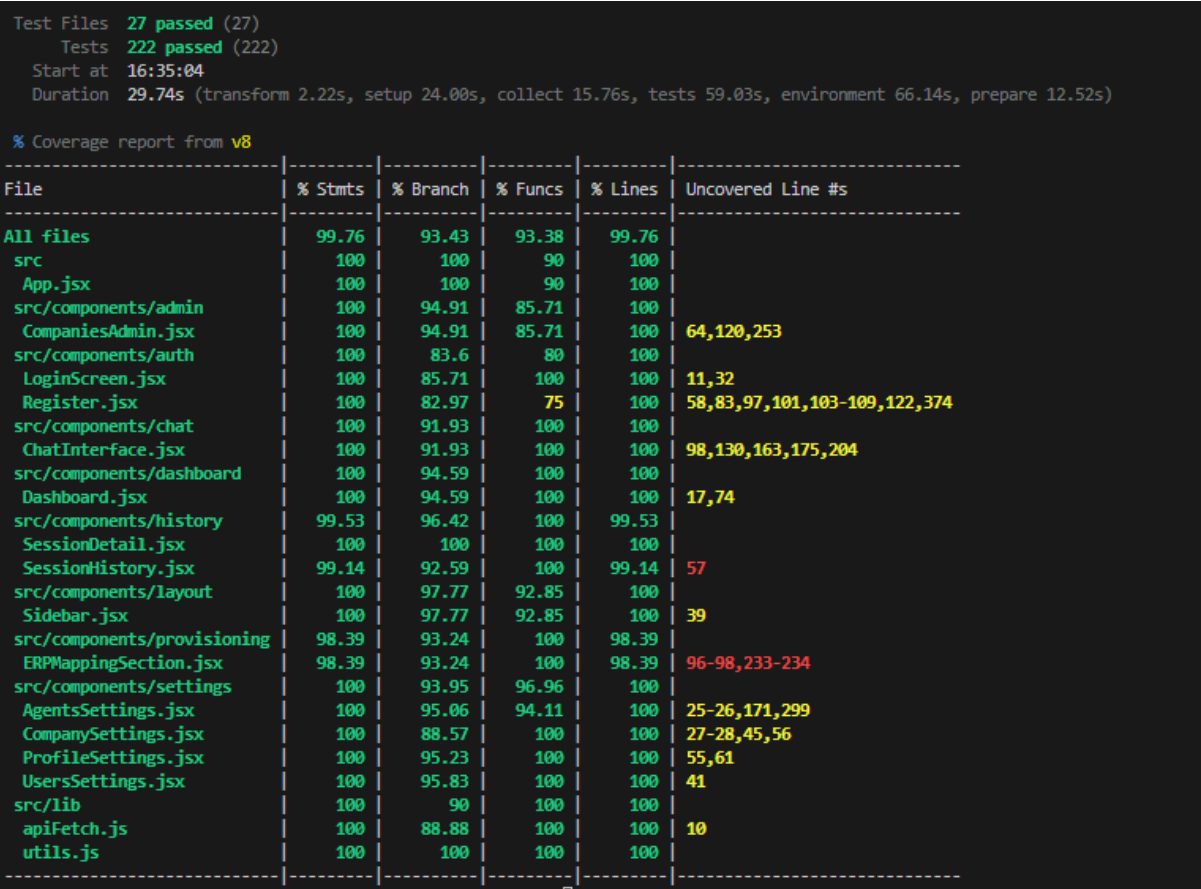
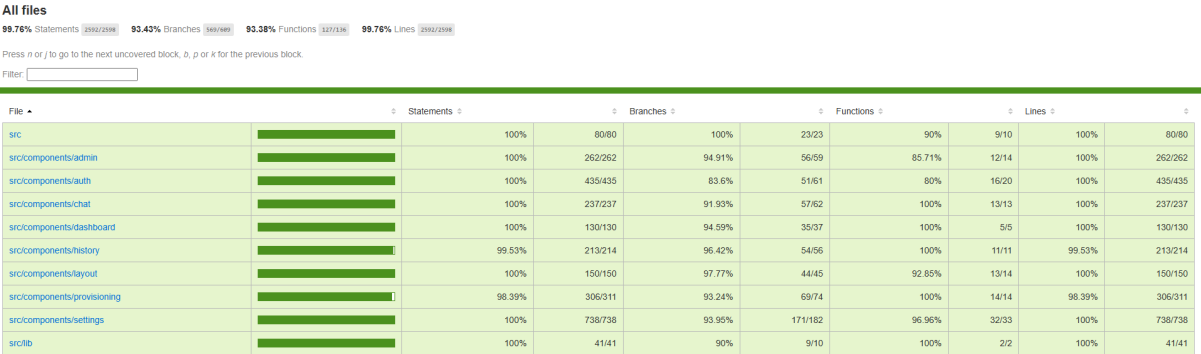


Figura 31: Reporte HTML de cobertura V8 — Frontend React (branch coverage 91.8%). Fuente: Elaboración propia.

```
Running 9 tests using 4 workers

✓ 1 [chromium] > src\test\e2e\auth.spec.js:5:3 > E2E - Autenticación > E2E-A01: login exitoso redirige a /app y muestra el dashboard (5.3s)
✓ 2 [chromium] > src\test\e2e\auth.spec.js:23:3 > E2E - Autenticación > E2E-A02: credenciales inválidas muestra mensaje de error y no navega (4.4s)
✓ 3 [chromium] > src\test\e2e\auth.spec.js:46:3 > E2E - Autenticación > E2E-A04: logout limpia la sesión y redirige a /login (6.2s)
✓ 4 [chromium] > src\test\e2e\auth.spec.js:41:3 > E2E - Autenticación > E2E-A03: acceso a /app sin token redirige a /login (3.5s)
✓ 5 [chromium] > src\test\e2e\chat.spec.js:13:3 > E2E - Chat > E2E-CH01: enviar mensaje muestra la respuesta del agente (3.0s)
✓ 6 [chromium] > src\test\e2e\chat.spec.js:29:3 > E2E - Chat > E2E-CH02: error 401 durante el chat cierra sesión y redirige a login (4.2s)
✓ 7 [chromium] > src\test\e2e\navigation.spec.js:12:3 > E2E - Navegación > E2E-N01: sidebar lista agentes dinámicos y navega a un agente (2.6s)
✓ 8 [chromium] > src\test\e2e\navigation.spec.js:17:3 > E2E - Navegación > E2E-N02: navega a Historial, abre una sesión y vuelve a la lista (3.4s)
✓ 9 [chromium] > src\test\e2e\navigation.spec.js:31:3 > E2E - Navegación > E2E-N03: rol ADMIN ve la sección de administración en el sidebar (2.6s)

9 passed (18.0s)
PS C:\Users\Rayker\Documents\Front-AgenteX\Front-AgenteX->
```

Figura 32: Salida de consola: npx playwright test — Frontend React E2E (9/9 pasando en Chromium). Fuente: Elaboración propia.

```

(venv) PS C:\Users\Rayker\Documents\motor-ia-agente> pytest --cov=. --cov-report=term-missing -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\Rayker\Documents\motor-ia-agente\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Rayker\Documents\motor-ia-agente
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.13.0, asyncio-1.4.0, cov-7.1.0, respx-0.23.1
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 61 items

tests/e2e/test_full_flow.py::TestFlujoBusquedaConRefinamientoPorCategoria::test_turno1_demasiados_resultados_turno2_refinado_por_categoria PASSED
tests/e2e/test_full_flow.py::TestFlujoStockCriticoDefinFin::test_usuario_pregunta_por_stock_critico_y_recibe_lista_real PASSED
tests/integration/test_process_chat.py::TestRespuestaSimpleSinTools::test_respuesta_directa_sin_tool_calls PASSED
tests/integration/test_process_chat.py::TestFlujoConToolCalls::test_temperatura_se_envia_tal_cual_cuando_no_hay_tools_activas PASSED
tests/integration/test_process_chat.py::TestFlujoConToolCalls::test_tool_call_consulta_erp_y_responde_en_segundo_turno PASSED
tests/integration/test_process_chat.py::TestFlujoConToolCalls::test_temperatura_se_fuerza_a_cero_cuando_hay_tools_activas PASSED
tests/integration/test_process_chat.py::TestFlujoConToolCalls::test_tool_call_con_nombre_no_reconocido PASSED
tests/integration/test_process_chat.py::TestFlujoConToolCalls::test_limite_de_iteraciones_devuelve_500 PASSED
tests/integration/test_process_chat.py::TestErroresDeepSeek::test_deepseek_error_http_devuelve_502 PASSED
tests/integration/test_process_chat.py::TestErroresDeepSeek::test_excepcion_generica_devuelve_500 PASSED
tests/integration/test_process_chat.py::TestRecorteDeContexto::test_historial_largo_se_recorta_preservando_el_system_prompt PASSED
tests/regression/test_regression.py::TestBugPY01TokensNumericosDebenSerOR::test_medida_29_210_encuentra_producto_29x2125 PASSED
tests/regression/test_regression.py::TestBugPY01TokensNumericosDebenSerOR::test_medida_con_dos_tokens_numericos_no_exige_ambos_a_la_vez PASSED
tests/regression/test_regression.py::TestBugPY02MatchIdSkuDebeUsarTerminoNormalizado::test_busqueda_con_mayusculas_y_tildes_encuentra_el_sku PASSED
tests/regression/test_regression.py::TestBugPY02MatchIdSkuDebeUsarTerminoNormalizado::test_busqueda_por_id_exacto_normalizado PASSED
tests/security/test_security.py::TestControlDeAccesoInternalSecret::test_sin_header_y_secret_configurado_devuelve_403 PASSED
tests/security/test_security.py::TestControlDeAccesoInternalSecret::test_header_con_valor_incorrecto_devuelve_403 PASSED
tests/security/test_security.py::TestControlDeAccesoInternalSecret::test_header_correcto_permite_continuar PASSED
tests/security/test_security.py::TestControlDeAccesoInternalSecret::test_internal_secret_vacio_no_exige_header PASSED
tests/security/test_security.py::TestValidacionDeEntrada::test_user_message_vacio_devuelve_400 PASSED
tests/security/test_security.py::TestValidacionDeEntrada::test_user_message_solo_espacios_devuelve_400 PASSED
tests/security/test_security.py::TestValidacionDeEntrada::test_tenant_id_negativo_no_es_rechazado_por_el_modelo PASSED
tests/security/test_security.py::TestEntradaMaliciosaNoRompeElMotor::test_valor_busqueda_con_caracteres_especiales_no_lanza_excepcion PASSED
tests/smoke/test_smoke.py::TestAppArranca::test_openapi_disponible_y_contiene_el_endpoint_principal PASSED
tests/smoke/test_smoke.py::TestAppArranca::test_docs_disponible PASSED
tests/smoke/test_smoke.py::TestAppArranca::test_titulo_de_la_app_es_el_esperado PASSED
tests/smoke/test_smoke.py::TestRutaPrincipalAccesible::test_post_sin_body_devuelve_422 PASSED
tests/smoke/test_smoke.py::TestRutaPrincipalAccesible::test_post_con_body_invalido_devuelve_422 PASSED
tests/smoke/test_smoke.py::TestRutaPrincipalAccesible::test_get_no_permitido_en_la_ruta_de_proceso PASSED
tests/smoke/test_smoke.py::TestRutaPrincipalAccesible::test_ruta_desconocida_devuelve_404 PASSED
tests/unit/test_chat_request_model.py::TestChatRequestDefaults::test_compos_opcionales_usan_sus_defaults PASSED
tests/unit/test_chat_request_model.py::TestChatRequestDefaults::test_acepta_todos_los_compos_explicitos PASSED
tests/unit/test_chat_request_model.py::TestChatRequestValidacion::test_falla_sin_tenant_id PASSED
tests/unit/test_chat_request_model.py::TestChatRequestValidacion::test_falla_sin_user_message PASSED
tests/unit/test_chat_request_model.py::TestChatRequestValidacion::test_falla_sin_system_prompt PASSED
tests/unit/test_chat_request_model.py::TestChatRequestValidacion::test_tenant_id_no_castea_texto_no_numerico PASSED
tests/unit/test_tools.py::TestManifest::test_manifest_define_la_unica_tool_esperada PASSED
tests/unit/test_tools.py::TestManifest::test_manifest_requiere_tipo_filtro_y_valor_busqueda PASSED
tests/unit/test_tools.py::TestConfiguracionFaltante::test_sin_erp_url_devuelve_mensaje_sistema PASSED
tests/unit/test_tools.py::TestConfiguracionFaltante::test_sin_erp_mapping_devuelve_mensaje_sistema PASSED
tests/unit/test_tools.py::TestConfiguracionFaltante::test_erp_mapping_vacio_se_trata_como_faltante PASSED
tests/unit/test_tools.py::TestFalloConexionERP::test_status_distinto_200_devuelve_mensaje_de_fallo PASSED
tests/unit/test_tools.py::TestFalloConexionERP::test_excepcion_en_prefiltro_es_capturada PASSED

tests/unit/test_tools.py::TestConteoTotal::test_conteo_total_devuelve_cantidad_de_articulos PASSED
tests/unit/test_tools.py::TestBusquedaPorTexto::test_match_por_nombre_busqueda_general PASSED
tests/unit/test_tools.py::TestBusquedaPorTexto::test_match_por_sku_exacto PASSED
tests/unit/test_tools.py::TestBusquedaPorTexto::test_match_por_id_exacto PASSED
tests/unit/test_tools.py::TestBusquedaPorTexto::test_sin_resultados_devuelve_mensaje_anti_alucinacion PASSED
tests/unit/test_tools.py::TestBusquedaPorTexto::test_stopwords_y_plural_se_ignoran_en_tokenizacion PASSED
tests/unit/test_tools.py::TestCategoriaRefinada::test_refinamiento filtra_por_categoria PASSED
tests/unit/test_tools.py::TestCategoriaRefinada::test_refinamiento_sin_coincidencias_devuelve_mensaje PASSED
tests/unit/test_tools.py::TestOrdenamientoYbypass::test_mayor_valor_ordena_descendente_y_limita_a_3 PASSED
tests/unit/test_tools.py::TestOrdenamientoYbypass::test_menor_valor_ordena_ascendente PASSED
tests/unit/test_tools.py::TestOrdenamientoYbypass::test_stock_mayor_ordena_descendente_y_limita_a_5 PASSED
tests/unit/test_tools.py::TestOrdenamientoYbypass::test_stock_critico filtra_stock_menor_o_igual_a_3_y_ordena_ascendente PASSED
tests/unit/test_tools.py::TestOrdenamientoYbypass::test_stock_critico_sin_resultados_bajo_el_umbral PASSED
tests/unit/test_tools.py::TestTrampaDeFriccion::test_mas_de_20_resultados_con_categorias_multiples PASSED
tests/unit/test_tools.py::TestTrampaDeFriccion::test_mas_de_20_resultados_con_una_sola_categoria_muestra_ejemplos PASSED
tests/unit/test_tools.py::TestFormatoDeRespuesta::test_incluye_categoria_solo_si_esta_en_el_mapping PASSED
tests/unit/test_tools.py::TestFormatoDeRespuesta::test_articulo_sin_sku_no_rompe_la_limpieza_de_texto PASSED
tests/unit/test_tools.py::TestFormatoDeRespuesta::test_sin_categoria_en_mapping_no_la_muestra PASSED

===== tests coverage =====
coverage: platform win32, python 3.11

Name                               Stmts  Miss  Cover   Missing
-----
conf/test.py                         27      0  100%
main.py                             74      0  100%
tests/e2e/test_full_flow.py          53      0  100%
tests/integration/test_process_chat.py 108      0  100%
tests/regression/test_regression.py    34      0  100%
tests/security/test_security.py        68      0  100%
tests/smoke/test_smoke.py             31      0  100%
tests/unit/test_chat_request_model.py  29      0  100%
tests/unit/test_tools.py             175      0  100%
tools.py                             86      0  100%
-----
TOTAL                               685      0  100%
Coverage HTML written to dir htmlcov
Required test coverage of 90% reached. Total coverage: 100.00%

===== 61 passed in 28.66s =====
(venv) PS C:\Users\Rayker\Documents\motor-ia-agente>

```

Figura 33: Salida de consola: pytest --cov — Microservicio Python (61/61 pasando, 100% coverage). Fuente: Elaboración propia.

```

PS C:\Users\Rayker\Documents\proyectos_Dev\backend-AgenteX> $env:K6_EMAIL = 'admin@agentex.test'; $env:K6_PASSWORD = '123456'; k6 run loadtest/loadtest.js
output: -

scenarios: (100.00%) 2 scenarios, 20 max VUs, 1m40s max duration (incl. graceful stop):
  * health_check: Up to 10 looping VUs for 1m10s over 3 stages (gracefulRampDown: 30s, exec: healthCheck, gracefulStop: 30s)
  * login: Up to 10 looping VUs for 1m10s over 3 stages (gracefulRampDown: 30s, exec: loginFlow, gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(95)<800' p(95)=230.37ms

http_req_failed
✓ 'rate<0.01' rate=0.00%

TOTAL RESULTS

checks_total.....: 1986    28.237744/s
checks_succeeded...: 100.00% 1986 out of 1986
checks_failed.....: 0.00%   0 out of 1986

✓ status 200
✓ respuesta correcta
✓ token presente

HTTP
http_req_duration.....: avg=102.81ms min=0s med=1.36ms max=1.18s p(90)=226.84ms p(95)=230.37ms
  { expected_response:true }....: avg=102.81ms min=0s med=1.36ms max=1.18s p(90)=226.84ms p(95)=230.37ms
http_req_failed.....: 0.00% 0 out of 993
http_reqs.....: 993    14.118872/s

EXECUTION
iteration_duration.....: avg=1.1s    min=1s med=1s    max=2.2s p(90)=1.22s p(95)=1.23s
iterations.....: 993    14.118872/s
vus.....: 3    min=0    max=20
vus_max.....: 20    min=20    max=20

NETWORK
data_received.....: 393 kB 5.6 kB/s
data_sent.....: 125 kB 1.8 kB/s

running (1m10.3s), 00/20 VUs, 993 complete and 0 interrupted iterations
health_check ✓ [=====] 00/10 VUs 1m10s
login ✓ [=====] 00/10 VUs 1m10s

```

```

PS C:\Users\Rayker\Documents\proyectos_Dev\backend-AgenteX> $env:NODE_ENV = 'development'; npm run dev

> dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (7) from .env // tip: ~ auth for agents [www.vestauth.com]
◇ injected env (0) from .env // tip: ~ multiple files { path: ['.env.local', '.env'] }
[16:16:13] INFO: Servidor activo en http://localhost:3000
  service: "agentex-backend"
  env: "development"
  port: "3000"

```

Figura 34: Resultado k6 — Backend Node.js (thresholds ambos en verde, p95=230.07ms, 0% error, 993 requests). Fuente: Elaboración propia.



```

execution: local
script: src/test/load/frontend-api.k6.js
output: -

scenarios: (100.00%) 4 scenarios, 200 max VUs, 8m0s max duration (incl. graceful stop):
  * escenario_10: 10 looping VUs for 1m0s (exec: flujoUsuario, gracefulStop: 30s)
  * escenario_50: 50 looping VUs for 1m0s (exec: flujoUsuario, startTime: 1m10s, gracefulStop: 30s)
  * escenario_100: 100 looping VUs for 1m0s (exec: flujoUsuario, startTime: 2m20s, gracefulStop: 30s)
  * escenario_estres: Up to 200 looping VUs for 4m0s over 5 stages (gracefulRampDown: 30s, exec: flujoUsuario, startTime: 3m30s, gracefulStop: 30s)

WARN[0440] Request Failed          error="Get \"http://localhost:3000/api/sessions/stats\": request timeout"

THRESHOLDS

http_req_duration
X 'p(95)<500' p(95)=934.23ms

{group:::Auth}
X 'p(95)<300' p(95)=1.27s

{group:::Dashboard}
X 'p(95)<300' p(95)=1.07s

{group:::Historial}
X 'p(95)<300' p(95)=858.95ms

{group:::Settings}
X 'p(95)<300' p(95)=617.7ms

http_req_failed{scenario:escenario_estres}
✓ 'rate<0.05' rate=0.00%
```

```

TOTAL RESULTS

checks_total.....: 94170 196.186841/s
checks_succeeded...: 99.99% 94169 out of 94170
checks_failed.....: 0.00% 1 out of 94170

✓ POST /api/auth/login → 200
X GET /api/sessions/stats → 200
  ↳ 99% = ✓ 9416 / X 1
✓ GET /api/agents/my-agents → 200
✓ GET /api/users/me → 200
✓ GET /api/sessions → 200
✓ GET /api/sessions/:id/messages → 200
✓ GET /api/users/me (settings) → 200
✓ GET /api/company → 200
✓ GET /api/agents → 200
✓ GET /api/agents/templates → 200

HTTP
http_req_duration.....: avg=391.17ms min=134.33ms med=304.38ms max=1mb5 p(90)=685.39ms p(95)=934.23ms
{ expected_response:true }.....: avg=390.54ms min=134.33ms med=304.38ms max=53.96s p(90)=685.31ms p(95)=934.17ms
{ group:::Auth }.....: avg=504.67ms min=196.74ms med=392.64ms max=8.08s p(90)=952.28ms p(95)=1.27s
{ group:::Dashboard }.....: avg=469.35ms min=135.53ms med=375.35ms max=1mb5 p(90)=800.96ms p(95)=1.07s
{ group:::Historial }.....: avg=488.25ms min=136.09ms med=322.82ms max=53.96s p(90)=692.84ms p(95)=858.95ms
{ group:::Settings }.....: avg=295.57ms min=134.33ms med=212.90ms max=44.47s p(90)=506.58ms p(95)=617.7ms
http_req_failed.....: 0.00% 1 out of 94170
{ scenario:escenario_estres }.....: 0.00% 1 out of 70170
http_reqs.....: 94170 196.186841/s

EXECUTION
iteration_duration.....: avg=4.91s min=0.91s med=4.83s max=1e5s p(90)=6.41s p(95)=7.06s
iterations.....: 9417 19.618684/s
vus.....: 1 min=0 max=200
vus_max.....: 200 min=200 max=200

NETWORK
data_received.....: 161 MB 335 kB/s
data_sent.....: 29 MB 60 kB/s

running (8m00.0s), 000/200 VUs, 9417 complete and 1 interrupted iterations
escenario_10 ✓ [=====] 10 VUs 1m0s
escenario_50 ✓ [=====] 50 VUs 1m0s
escenario_100 ✓ [=====] 100 VUs 1m0s
escenario_estres ✓ [=====] 001/200 VUs 4m0s
ERROR[0480] thresholds on metrics 'http_req_duration, http_req_duration(group:::Auth), http_req_duration(group:::Dashboard), http_req_duration(group:::Historial), http_req_duration(group:::Settings)' have been crossed
PS C:\Users\Rayker\Documents\Front-AgenteX\Front-AgenteX>
```

Figura 35: Resultado k6 — Frontend React (salida de terminal con thresholds, p95 y tasa de éxito de los 4 escenarios). Fuente: Elaboración propia.


```
(venv) PS C:\Users\Rayker\Documents\motor-ia-agenteX> .\venv\Scripts\python.exe -m uvicorn tests.load_stub_server:app --port 9000
INFO: Started server process [14088]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:9000 (Press CTRL+C to quit)
```

```
PS C:\Users\Rayker\Documents\motor-ia-agenteX> .\venv\Scripts\python.exe -m uvicorn main:app --port 8001
INFO: Started server process [16648]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

```
THRESHOLDS
http_req_duration
✓ 'p(95)<300' p(95)=0s

http_req_failed
X 'rate<0.05' rate=98.00%

{scenario:escenario_estres}
X 'rate<0.05' rate=100.00%

TOTAL RESULTS
checks_total.....: 9284 70.332149/s
checks_succeeded...: 2.00% 186 out of 9284
checks_failed.....: 97.99% 9098 out of 9284

X status 200
L 2% - ✓ 93 / X 4549
X success true
L 2% - ✓ 93 / X 4549

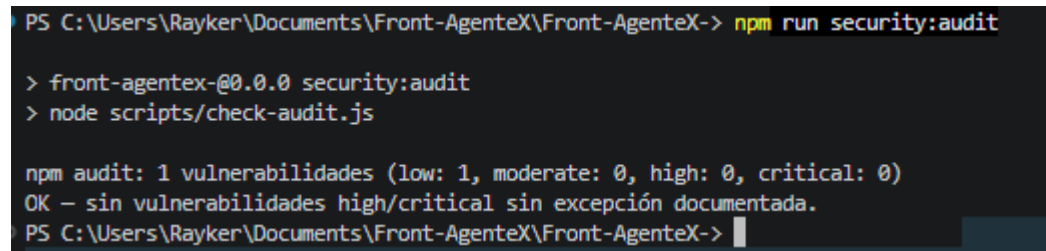
HTTP
http_req_duration.....: avg=622.58ms min=0s med=0s max=59.92s p(90)=0s p(95)=0s
{ expected_response:true }.....: avg=31.15s min=8.93s med=35.65s max=59.92s p(90)=55.71s p(95)=57.82s
http_req_failed.....: 98.00% 4561 out of 4654
{ scenario:escenario_estres }....: 100.00% 4561 out of 4561
http_reqs.....: 4654 35.256982/s

EXECUTION
iteration_duration.....: avg=626.55ms min=0s med=1.99ms max=59.92s p(90)=5.53ms p(95)=0ms
iterations.....: 4642 35.166074/s
vus.....: 173 min=10 max=247
vus_max.....: 266 min=266 max=266

NETWORK
data_received.....: 27 kB 202 B/s
data_sent.....: 184 kB 1.4 kB/s

running (2m12.0s), 000/266 VUs, 4642 complete and 277 interrupted iterations
scenario_10 ✓[=====] 10 VUs 30s
scenario_50 ✓[=====] 50 VUs 30s
scenario_100 ✓[=====] 100 VUs 30s
scenario_estres X[=====] 179/200 VUs 0m27.0s/1m45.0s
ERR! [0132] thresholds on metrics 'http_req_failed, http_req_failed(scenario:escenario_estres)' were crossed; at least one has abortOnFail enabled, stopping test prematurely
(venv) PS C:\Users\Rayker\Documents\motor-ia-agenteX>
```

Figura 36: Resultado k6 — Microservicio Python (salida de terminal mostrando abort del escenario de estrés y métricas finales). Fuente: Elaboración propia.

A terminal window with a dark background and light-colored text. The prompt is 'PS C:\Users\Rayker\Documents\Front-AgenteX\Front-AgenteX->'. The command 'npm run security:audit' is entered. The output shows the package version 'front-agentex-@0.0.0', the command 'security:audit', and the script 'node scripts/check-audit.js'. The audit results show '1 vulnerabilidades (low: 1, moderate: 0, high: 0, critical: 0)' and a status 'OK - sin vulnerabilidades high/critical sin excepción documentada.'.

```
PS C:\Users\Rayker\Documents\Front-AgenteX\Front-AgenteX-> npm run security:audit

> front-agentex-@0.0.0 security:audit
> node scripts/check-audit.js

npm audit: 1 vulnerabilidades (low: 1, moderate: 0, high: 0, critical: 0)
OK - sin vulnerabilidades high/critical sin excepción documentada.
PS C:\Users\Rayker\Documents\Front-AgenteX\Front-AgenteX->
```

Figura 37: Salida de npm run security:audit — 0 vulnerabilidades high/critical (1 low documentada). Fuente: Elaboración propia.

Integración Backend

```
Test Suites: 7 passed, 7 total
Tests:      49 passed, 49 total
Snapshots:  0 total
Time:       5.518 s
Ran all test suites matching /test\\integration/i.
PS C:\Users\Rayker\Documents\proyectos_Dev\backend-AgenteX>
```

Integración Front

```
Test Files 4 passed (4)
Tests      37 passed (37)
Start at   21:11:00
Duration   9.15s (transform 3.65s, setup 3.51s, collect 5.20s, tests 7.28s, environment 10.53s, prepare 1.19s)
```

Integración MicroServicio

```
(venv) PS C:\Users\Rayker\Documents\motor-ia-agenteX> .\venv\Scripts\python.exe -m pytest tests\integration -q --no-cov
.....
9 passed in 5.64s
(venv) PS C:\Users\Rayker\Documents\motor-ia-agenteX>
```

Figura 38: Salida de las 3 integraciones, Front, Backend y Microservicio.
Fuente: Elaboración propia.